Pontificia Universidad Católica del Perú
Facultad de Ciencias e Ingeniería
Departamento Académico de Ingeniería



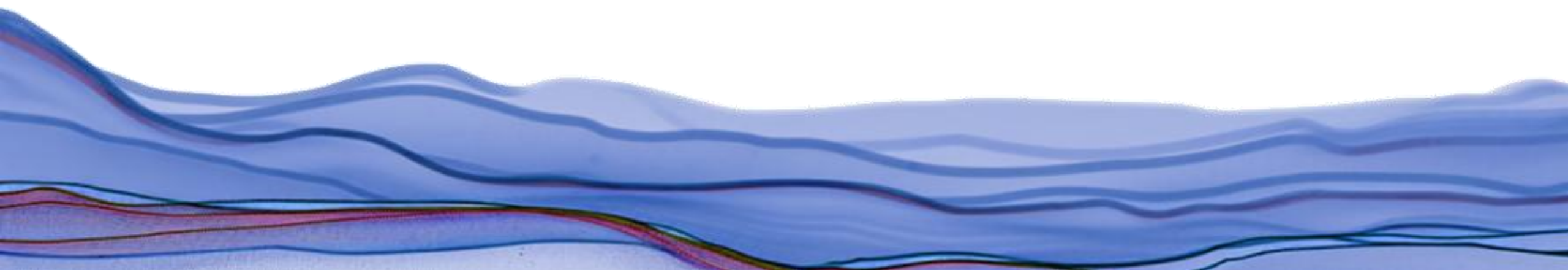
INF659 TÉCNICAS AVANZADAS DE DATA MINING Y SISTEMAS INTELIGENTES

Modelos Autorregresivos y Transformadores

Prof. Rodrigo Uribe

Outline

- Costo computacional y paralelismo de las RNN
- Modelos autorregresivos convolucionales
- RNN con mecanismos de atención
- Self-attention y transformadores



Respuestas del Vibe Quest

- ¿Cómo afectan los gradientes que se desvanecen al rendimiento de una RNN simple en una tarea con dependencias a largo plazo? ¿Puedes demostrarlo con un ejemplo de código cómo esas gradientes explotan vs un modelo normal?
- ¿Puedes diseñar y entrenar una RNN para generar texto coherente a partir de un conjunto de datos pequeño? ¿Cómo afecta los tokens o embeddings utilizados para entrenar este modelo?
- ¿Qué información almacenan realmente las neuronas del estado oculto de una RNN entrenada para modelado de lenguaje? ¿Puedes visualizar las activaciones internas y encontrar neuronas que representen conceptos como comillas abiertas/cerradas, niveles de indentación o estructuras sintácticas?
- ¿Cómo evoluciona la capacidad de una RNN para aprender patrones de corto y largo plazo durante el entrenamiento de un modelo generador de texto a nivel de caracteres? ¿Puedes demostrarlo entrenando una RNN/LSTM y comparando muestras de texto generadas en distintas etapas del entrenamiento, además de analizar qué tipos de dependencias (ortografía, sintaxis, estructura y contexto) se aprenden primero y cuáles aparecen más tarde?

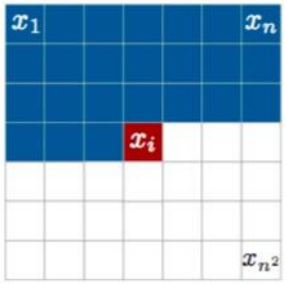


RNN: Costos computacionales y paralelismo

Modelos autorregresivos para generar secuencias largas

- Modelos autorregresivos (generan resultados de un elemento a la vez, donde cada nuevo elemento depende de elementos generados previamente)

- ¿Podemos mejorarlos para generar secuencias muy largas?





- Problema: entrenar una RNN para generar estas secuencias requiere > 10 000 pasos de tiempo en un bucle, que es computacionalmente costoso

Caracterización de los costos computacionales de las RNN

Podemos analizar los costos computacionales de RNN de varias maneras.

• **Flops:** número de operaciones de suma-multiplicación para los pases de ida y vuelta, determinado por el número de conexiones.

•**Requisitos de memoria:**

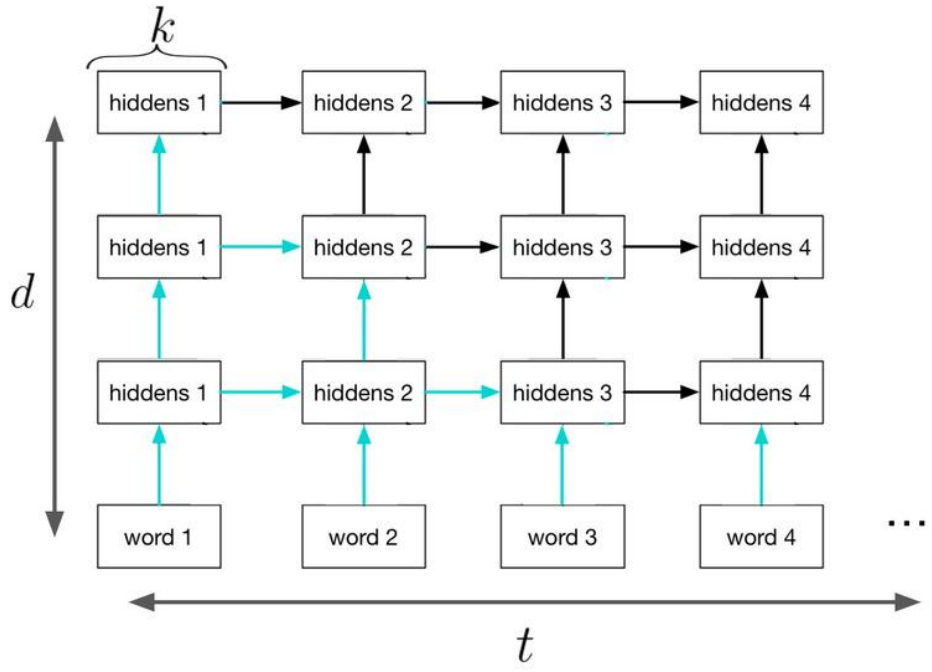
- Tiempo de entrenamiento: determinado por la cantidad de activaciones, ya que es necesario almacenarlas para la retropropagación.
- Tiempo de evaluación: determinado por el número de parámetros. Eficiente gracias a la distribución del peso a través de pasos de tiempo.

•**Complejidad secuencial:** número de operaciones secuenciales necesarias para realizar alguna tarea, suponiendo un paralelismo ilimitado.

Flops (Número de conexiones)

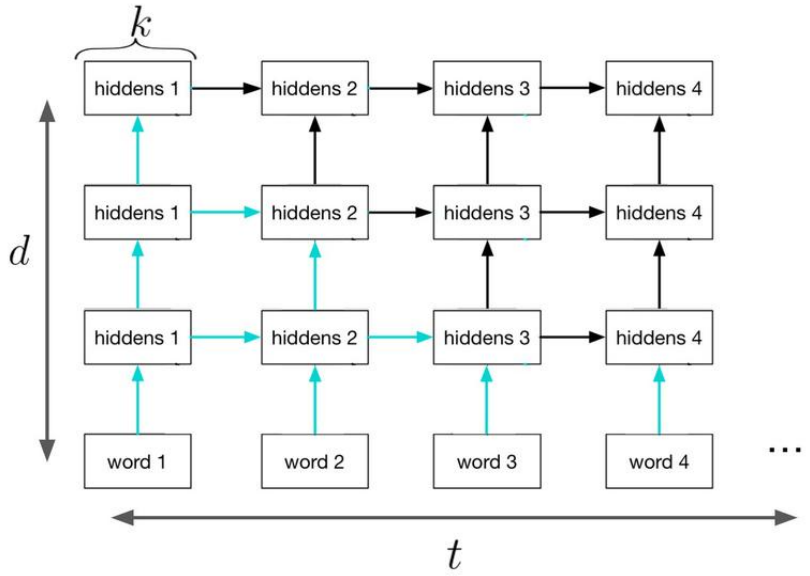
Considere una RNN con d capas ocultas, k unidades ocultas por capa, entrenada en en una secuencia de longitud t

¿Cuál es el número de conexiones?



Memoria de tiempo de generación RNN

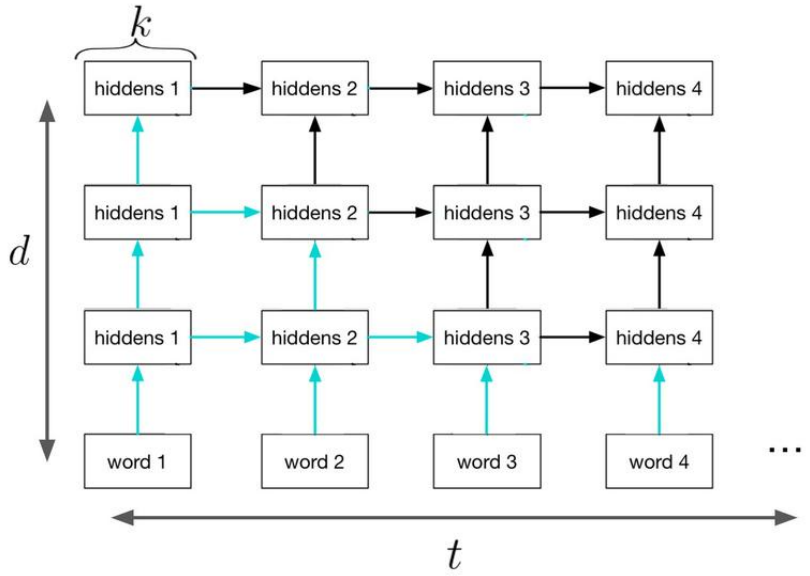
¿Cual es el número de parámetros?



Memoria del tiempo de entrenamiento de RNN

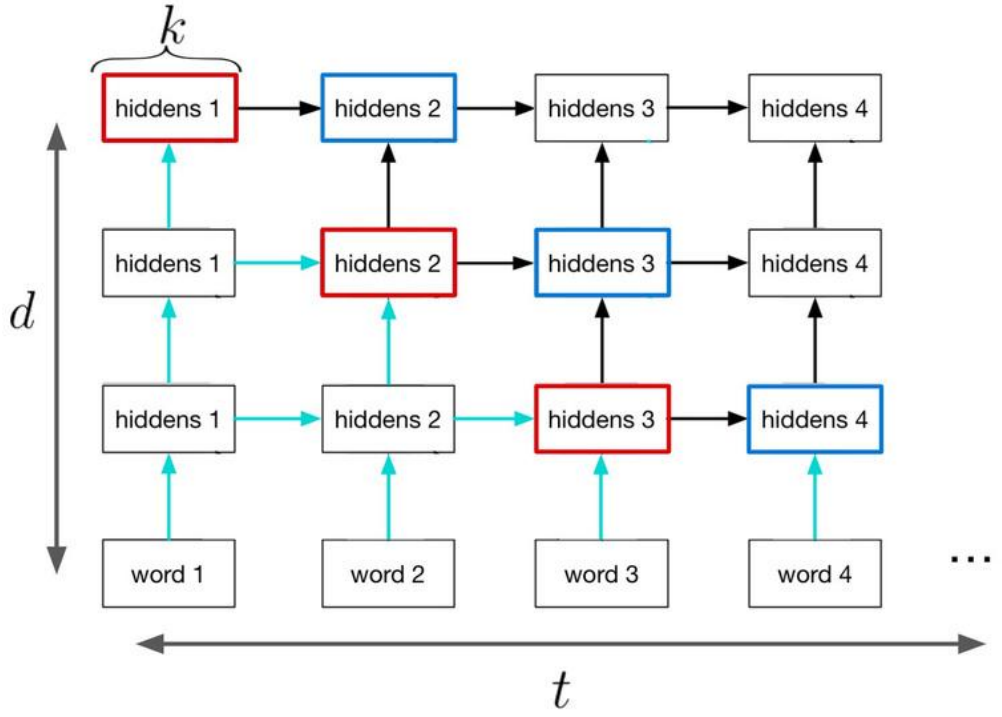
Backprop requiere almacenar todas las activaciones en la memoria.

¿Cual es el número de activaciones?



Actualización de la complejidad secuencial de un gradiente de RNN

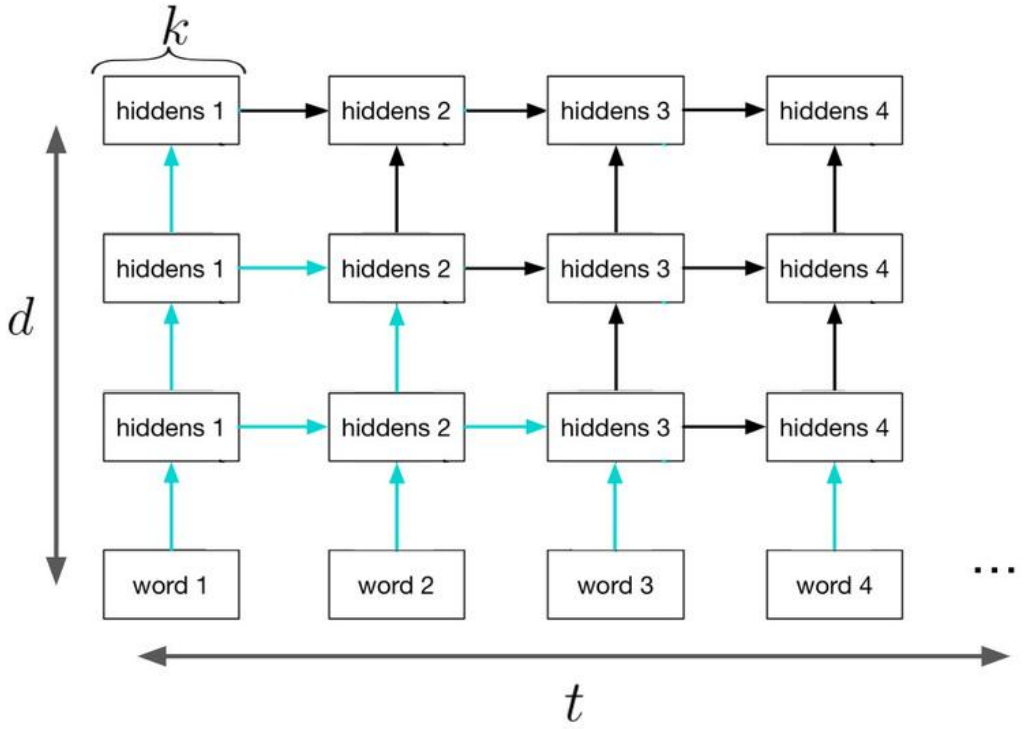
¿Cuántas operaciones secuenciales se requieren para realizar una actualización de gradiente?



Complejidad secuencial de generación de RNN

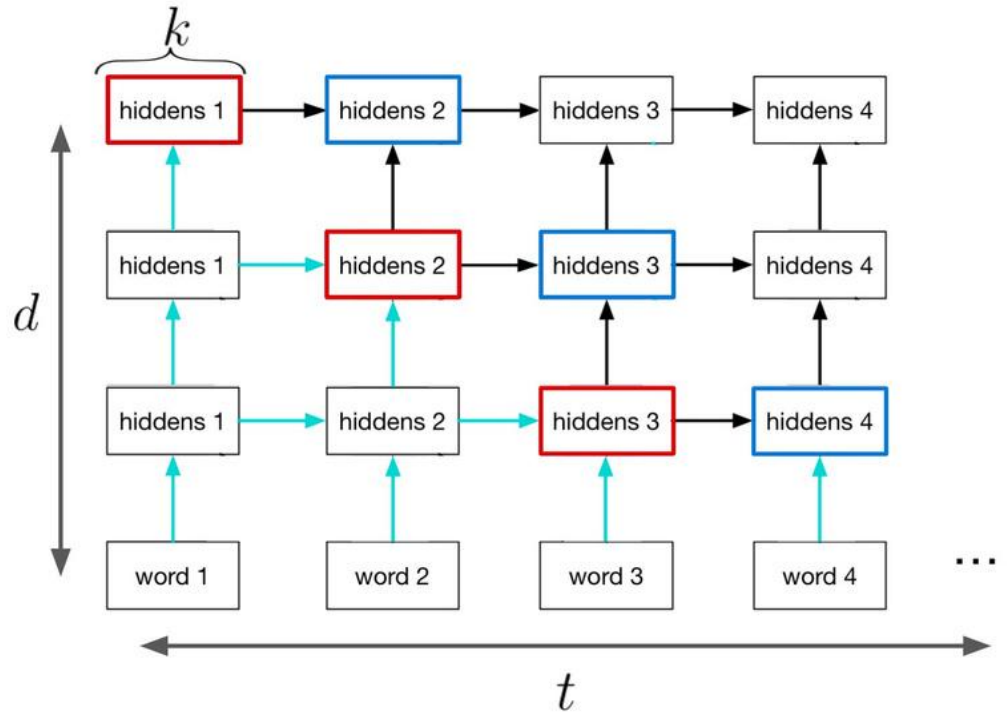
¿Cuántas operaciones secuenciales se requieren para generar datos?

Cada $k \times k$ matmul cuenta como una operación.



Resumen: Costos computacionales y paralelismo de RNN

	RNN
connections	$k^2 * d * t$
parameters	$k^2 * d$
activations	$k * d * t$
Seq. complexity (training)	$d + t$
Seq. complexity (generation)	dt



Resumen: Costo computacional y paralelismo de RNN

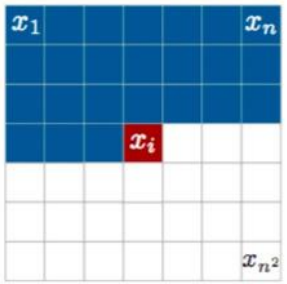
- La complejidad secuencial es perjudicial para secuencias largas.
- Los estados ocultos deben almacenar toda la información crítica sobre la secuencia de entrada, de manera redundante a través de muchos pasos de tiempo.



Modelos autorregresivos convolucionales

Modelos autorregresivos para generar secuencias largas

- ¿Podemos mejorar los modelos autorregresivos (por ejemplo, las RNN) para generar secuencias muy largas?





- Problema: entrenar una RNN para generar estas secuencias requiere > 10 000 pasos de tiempo en un bucle, lo cual es computacionalmente costoso

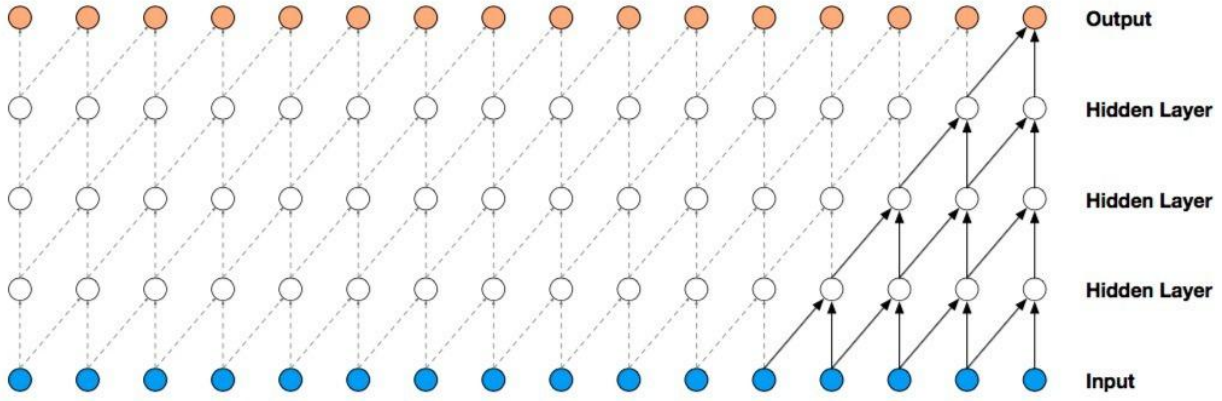
- Soluciones: (1) Convolución causal (2) Convolución dilatada

Solución 1: Convolución causal

•RNN mantiene la causalidad: cada predicción depende únicamente de entradas anteriores en la secuencia.

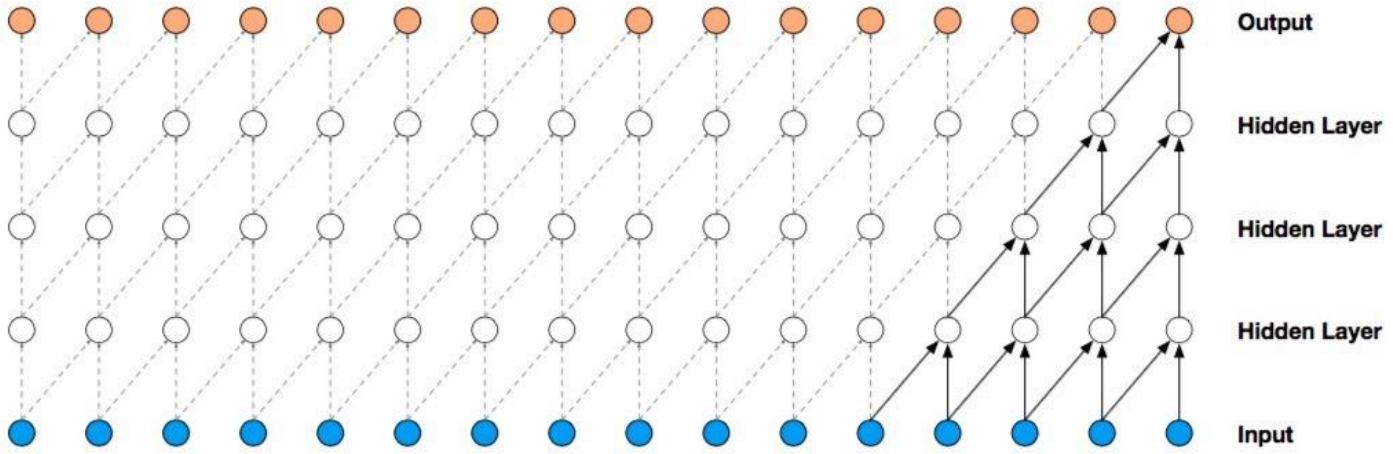
•La propiedad causal se puede lograr utilizando una arquitectura convolucional.

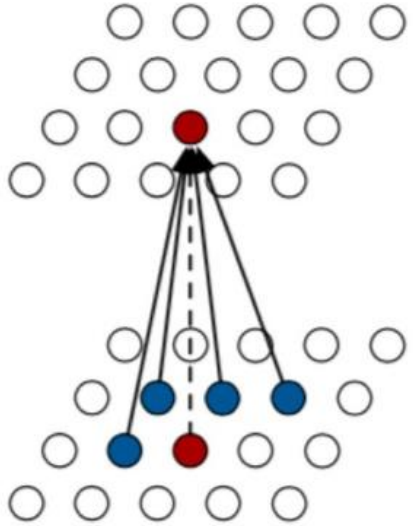
•Las convoluciones no requieren bucles for y son más eficientes para manejar bucles largos de secuencias.



Convolución causal

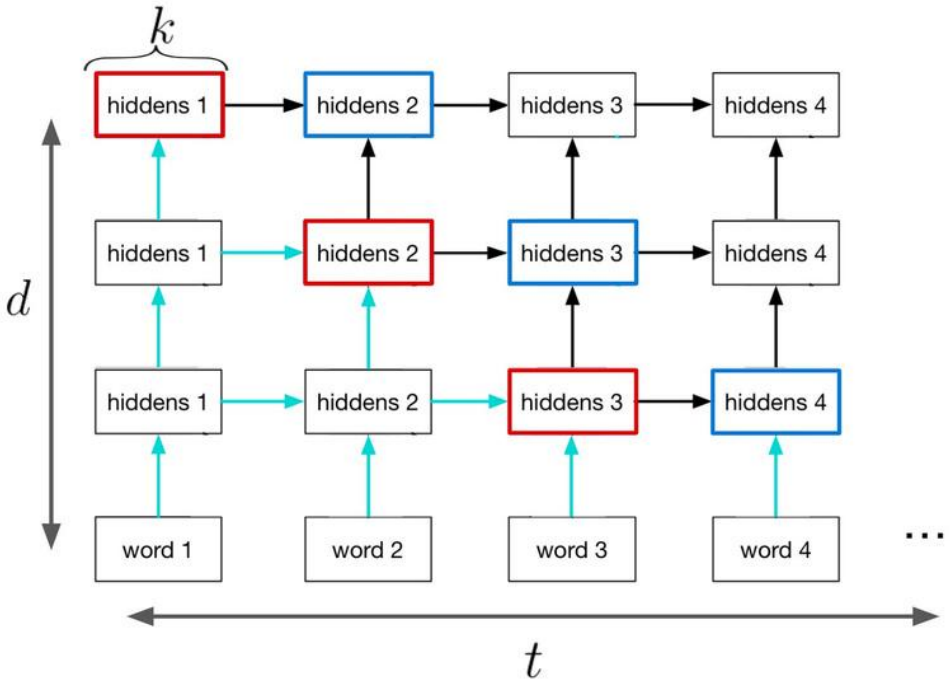
Propiedad causal: cada predicción depende únicamente de las entradas anteriores de la secuencia. Se implementa enmascarando o estableciendo ponderaciones cero en las conexiones con entradas futuras.



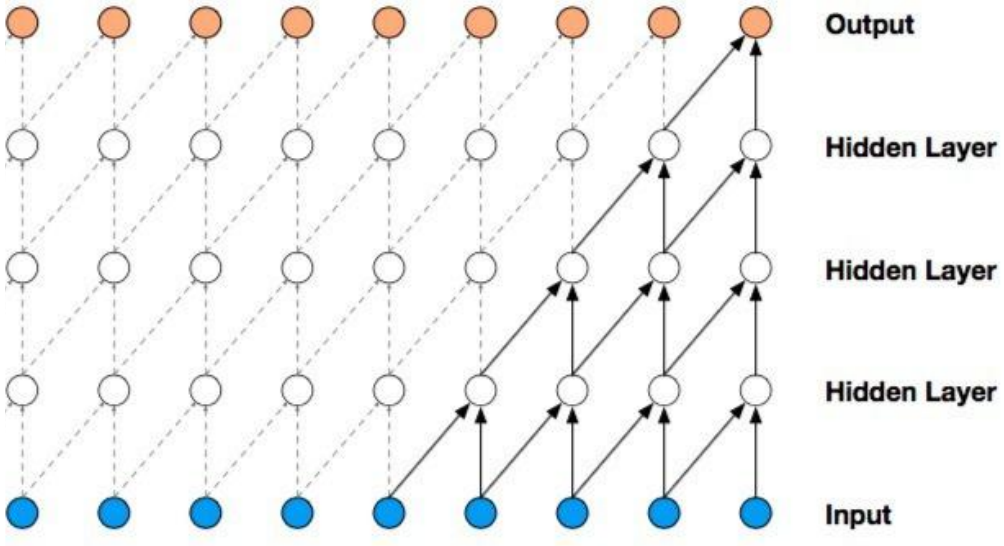


Comparación de la complejidad de RNN y CNN

RNN



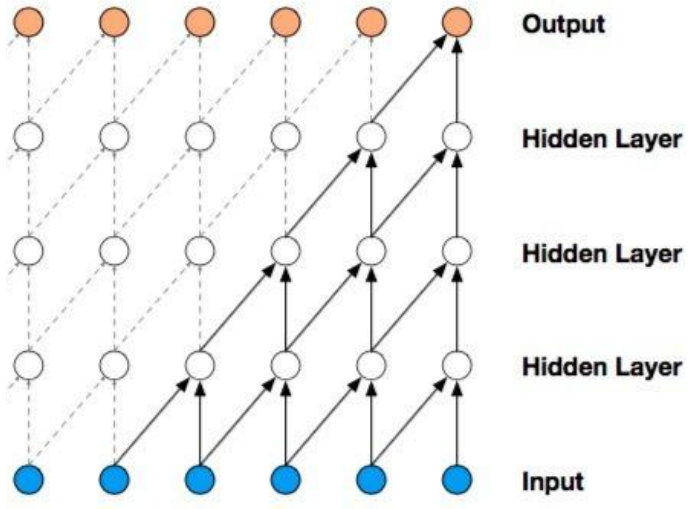
CNN



Análisis de la complejidad de CNN

t denota la longitud de la secuencia, d denota el número de capas, k denota el número de características por capa, y w denota el ancho del kernel.

	RNN	CNN
Conexiones Parámetros	$k^2 * d * t$	
Activaciones Seq.	$k^2 * d$	
Complejidad	$k * d * t$	
(entrenamiento) Seq.	$d + t$	
Complejidad (generación)	dt	



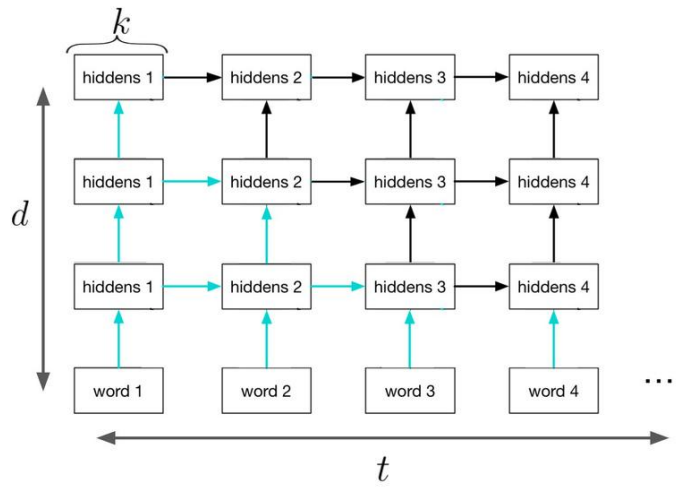
Comparación de la complejidad de RNN y CNN

	RNN	CNN
Conexiones Parámetros	$k^2 * d * t$	$k^2 * w * d * t$
Activaciones Seq.	$k^2 * d$	$k^2 * w * d$
Complejidad	$k * d * t$	$k * d * t$
(entrenamiento) Seq.	$d + t$	d
Complejidad (generación)	dt	dt

Complejidad secuencial: RNN versus CNN

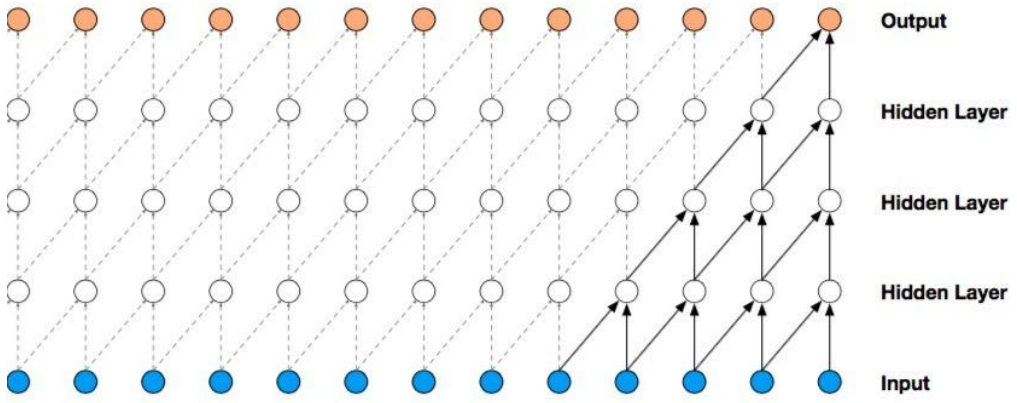
Complejidad de secuencia RNN: d+t

- Dependencias de capa a capa (d)
- Dependencias temporales (t)



Complejidad de secuencia de CNN: d

- Dependencias de capa a capa (d)
- Sin dependencias temporales

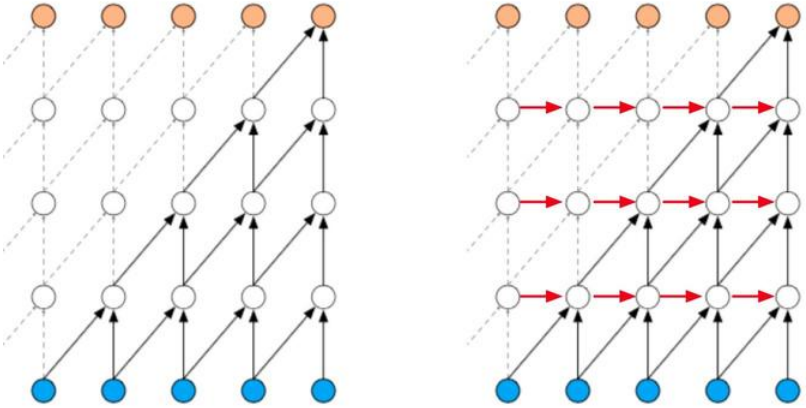


Compensación: CNN causal

Podemos convertir una CNN causal en una RNN agregando conexiones recurrentes. ¿Cuál es el compromiso?

- La RNN tiene memoria y puede usar información de pasos anteriores, mientras que la CNN tiene contexto limitado.
- La RNN es costosa para secuencias largas debido a su gran complejidad secuencial.
- La CNN puede explotar mejor el paralelismo.

Generar desde ambos modelos es muy costoso ya que ambos tienen gran complejidad secuencial.





PixelCNN y PixelRNN

Van Den Oord et al. "Redes neuronales recurrentes de píxeles". ICML 2016. Introdujeron dos modelos autorregresivos de imágenes.

Generaron imágenes de resolución impresionantemente alta mediante la predicción de una distribución.

Pixel Recurrent Neural Networks

Aäron van den Oord AVDNOORD@GOOGLE.COM
Nal Kalchbrenner NALK@GOOGLE.COM
Koray Kavukcuoglu KORAYK@GOOGLE.COM

Google DeepMind

Abstract

Modeling the distribution of natural images is a landmark problem in unsupervised learning. This task requires an image model that is at once expressive, tractable and scalable. We present a deep neural network that sequentially predicts the pixels in an image along the two spatial dimensions. Our method models the discrete probability of the raw pixel values and encodes the complete set of dependencies in the image. Architectural novelties include fast two-dimensional recurrent layers and an effective use of residual connections in deep recurrent networks. We achieve log-likelihood scores on natural images that are considerably better than the previous state of the art. Our main results also provide benchmarks on the diverse ImageNet



Figure 1. Image completions sampled from a PixelRNN.

cling is building complex and expressive models that are also tractable and scalable. This trade-off has resulted in a large variety of generative models, each having their advantages. Most work focuses on stochastic latent variable models such as VAE's (Rezende et al., 2014; Kingma & Welling, 2013) that aim to extract meaningful representations, but often come with an intractable inference step that

Completar una imagen usando PixelRNN

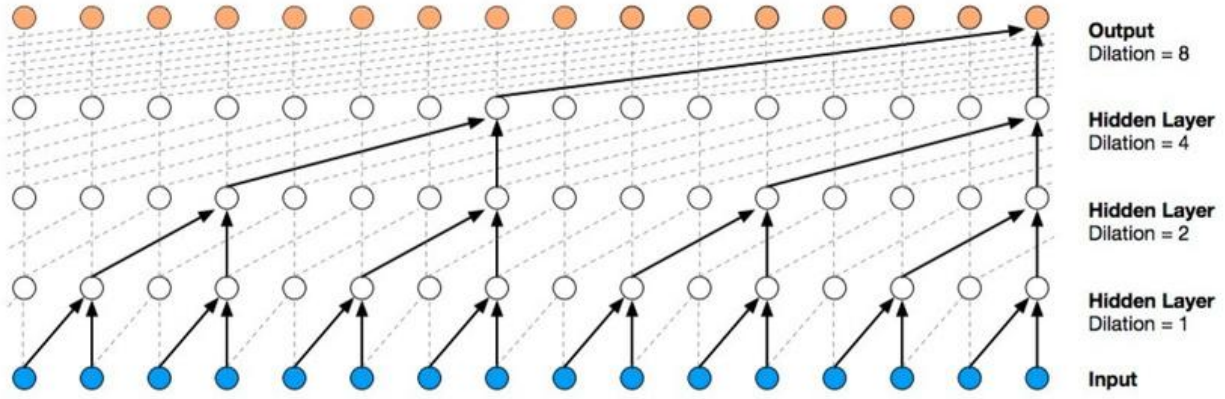


Muestras de PixelRNN entrenado en ImageNet



Solución 2: Convolución dilatada

- La ventaja de las RNN sobre las CNN es que su memoria les permite aprender dependencias arbitrarias de larga distancia.
- Pero podemos aumentar dramáticamente el campo receptivo de una CNN usando convolución dilatada.



Red de ondas

Modelo autorregresivo para audio crudo de alta calidad usando convoluciones dilatadas causales.

La generación de audio requiere secuencias muy largas

- Debe muestrear 16,000 marcos por segundo para audio de buena calidad.
- WaveNet usa dilataciones de 1, 2, ...,512.
- Cada bloque tiene un campo receptivo de 1024 marcos.
- 1024 marcos corresponden a 1024/16000 = 0.064s o 64ms de audio.
- WaveNet apila varios de estos bloques. La longitud total del contexto es ~300ms.

https://deepmind.google/discover/blog/wavenet-a-generative-model-for-raw-audio/

Van Den Oord, Aaron, et al. Wavenet: A generative model for raw audio. Google DeepMind. (2016).



RNN basadas en atención

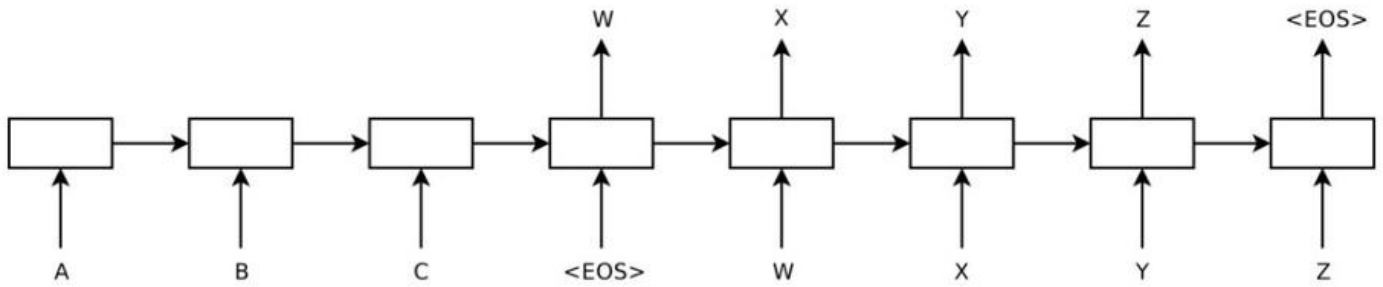
Atención Aborda los Cuellos de Botella de la Información

La arquitectura tradicional del codificador-descodificador comprime la oración de entrada en un conjunto de unidades ocultas.

Problema: las oraciones largas contienen demasiado información para ser comprimidas eficazmente en las unidades ocultas

Solución: dejar que el decodificador acceda directamente a la oración de entrada.

Éste es el concepto fundamental detrás de los mecanismos de atención.



De la compresión a la selección

Las RNN tradicionales

- comprimen toda la información en un solo vector
- creando un cuello de botella de información.

Modelos basados en la atención

- permiten que el modelo acceda directamente y se centre selectivamente en partes relevantes de la entrada

Modelo de traducción de Bahdanau et al., 2015

Bahdanau, et al. "Neural machine translation by jointly learning to align and translate."International Conference on Learning Representations (ICLR), 2015.

Concepto fundamental:

Cada palabra de la traducción de salida se deriva de una sola palabra o de un pequeño conjunto de palabras del texto de entrada.

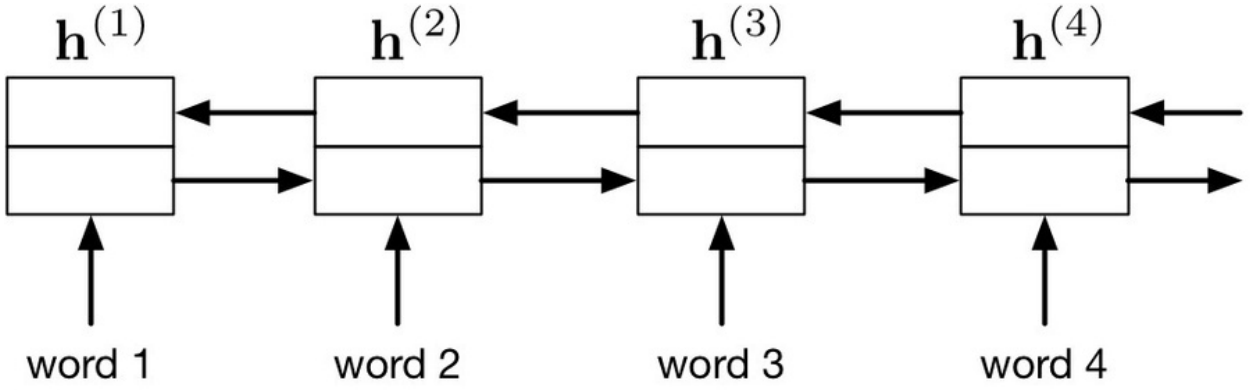
Idea clave:

Enseñe al modelo a centrarse selectivamente (o “prestar atención”) únicamente a las palabras de entrada relevantes al generar cada palabra de salida en lugar de procesar toda la secuencia de entrada de manera uniforme.

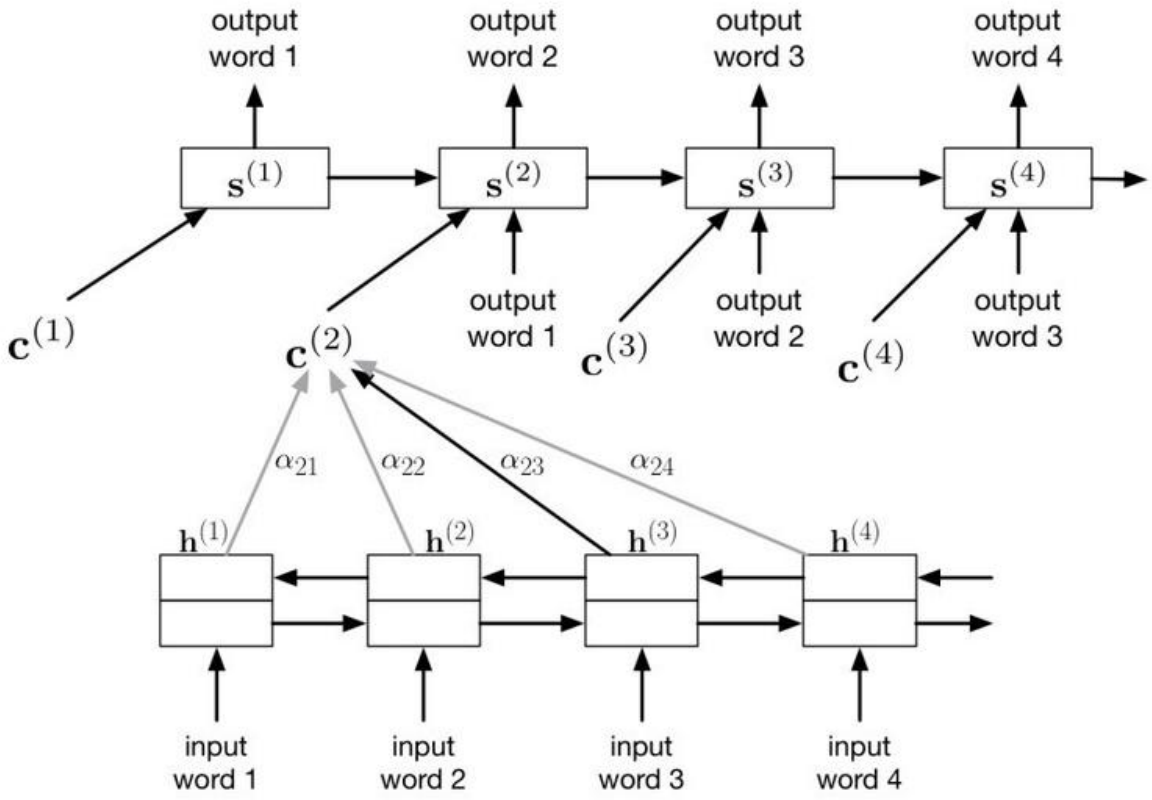
Diseño de codificador (encoder)

El codificador crea una anotación para cada palabra de entrada.

- Procesa cada palabra de entrada mediante un embedding a travez de una RNN bidireccional para proporcionar contexto.
- La anotación concatena los dos estados ocultos.
- Cada unidad oculta es una Unidad Recurrente con Puerta (similar a una unidad LSTM).
  - Utiliza puertas de actualización y reinicio para controlar el flujo de información a través de pasos de tiempo.



Diseño de decodificadores (decoder)



- Predice una palabra a la vez.
- Usa la salida como entrada.
- Recibe un contexto en cada **paso de tiempo, una suma ponderada** del codificador.

Diseño de decodificadores en la traducción automática basada en la atención

El decodificador también es una RNN que genera palabras de salida una a la vez, y cada predicción se devuelve como entrada para el siguiente paso.

Innovación clave: El decodificador recibe un vector de contexto en cada paso de tiempo. El contexto es una suma ponderada de las anotaciones del codificador. Esto permite al decodificador centrarse en diferentes partes de la oración de entrada al generar cada palabra de salida.



Matemáticas de la atención

Context

Attention Weights

Encoder's Annotations

Softmax Function

Decoder's State

Encoder's Annotation

$$c^{(i)} = \sum_j \alpha_{ij}\, h^{(j)},$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{j'} \exp(e_{ij'})},$$

$$e_{ij'} = a(s^{(i-1)}, h^{(j)})$$

$$a(s_{i-1}, h_j) = v_a^{\mathrm{T}} \tanh(W_a s_{i-1} + U_a h_j)$$

Direccionamiento basado en contenido

La función de anotación a depende del vector de anotación $h(\#)$ en lugar de la posición en la oración.

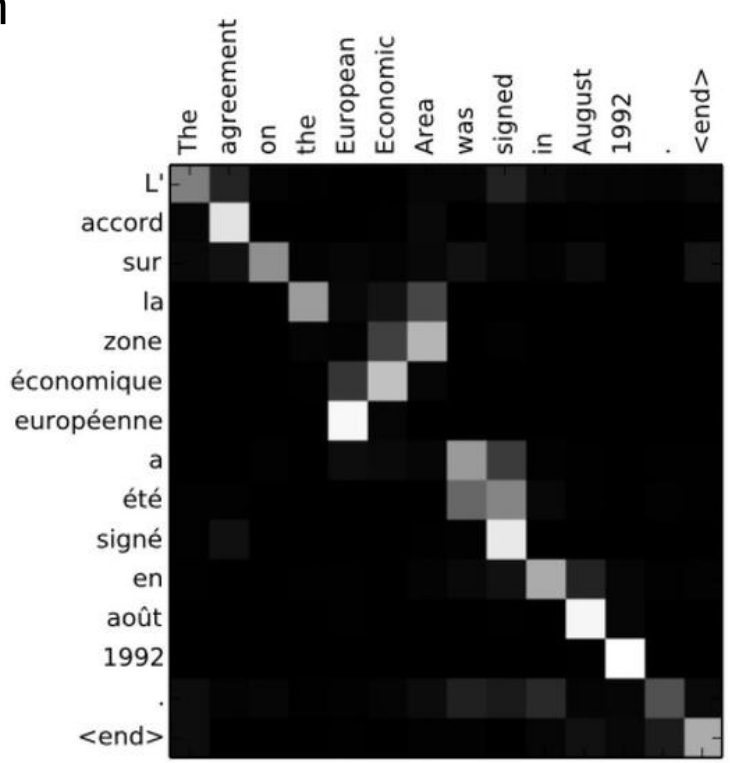
Se centra en el contenido semántico de las palabras de entrada en lugar de en las posiciones.

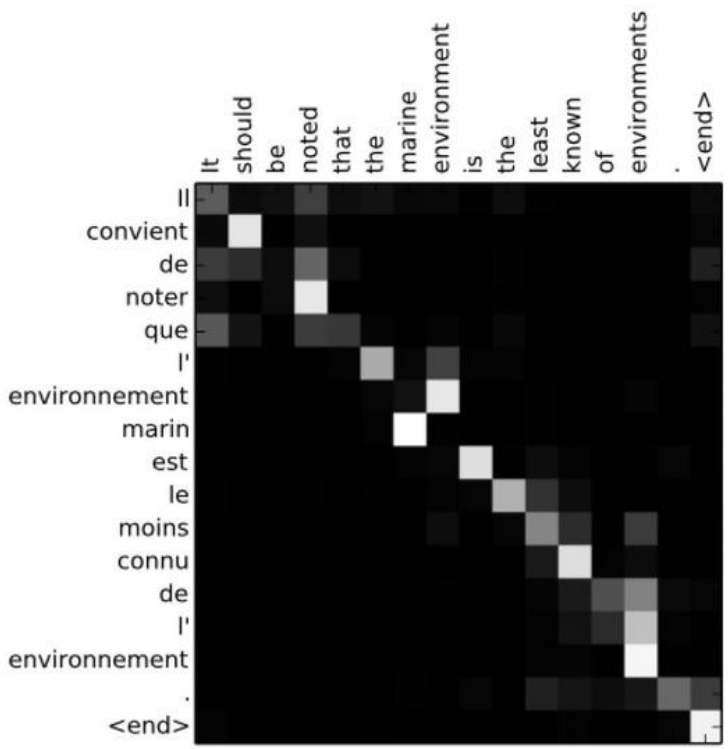
Ejemplo: al traducir "the beautiful house" → "la casa "..?

Si el modelo necesita generar un adjetivo en otro idioma, se centrará más en "beautiful" por su contenido/significado, no porque sea la segunda palabra de la oración.

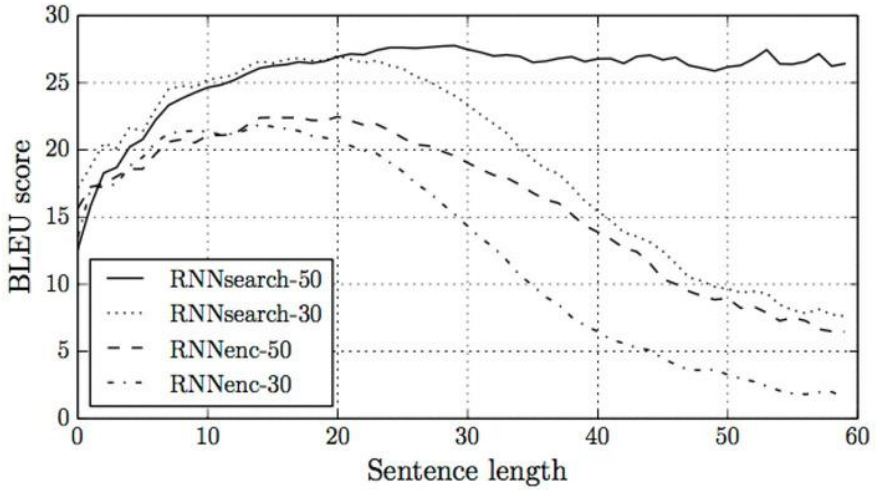
Patrones de atención diagonal en la traducción automática neuronal

Visualizando los mapas de atención para traducir entre francés e inglés
El modelo aprende naturalmente a alinear palabras entre idiomas sin programación explícita.
Los patrones diagonales: el modelo procesa las oraciones de manera lineal.
Excepciones para acomodar las diferencias lingüísticas (francés: adj después del sustantivo)
El modelo descubre patrones lingüísticos por sí mismo en lugar de tenerlos codificados manualmente.





La atención mantiene la calidad de la traducción para oraciones largas

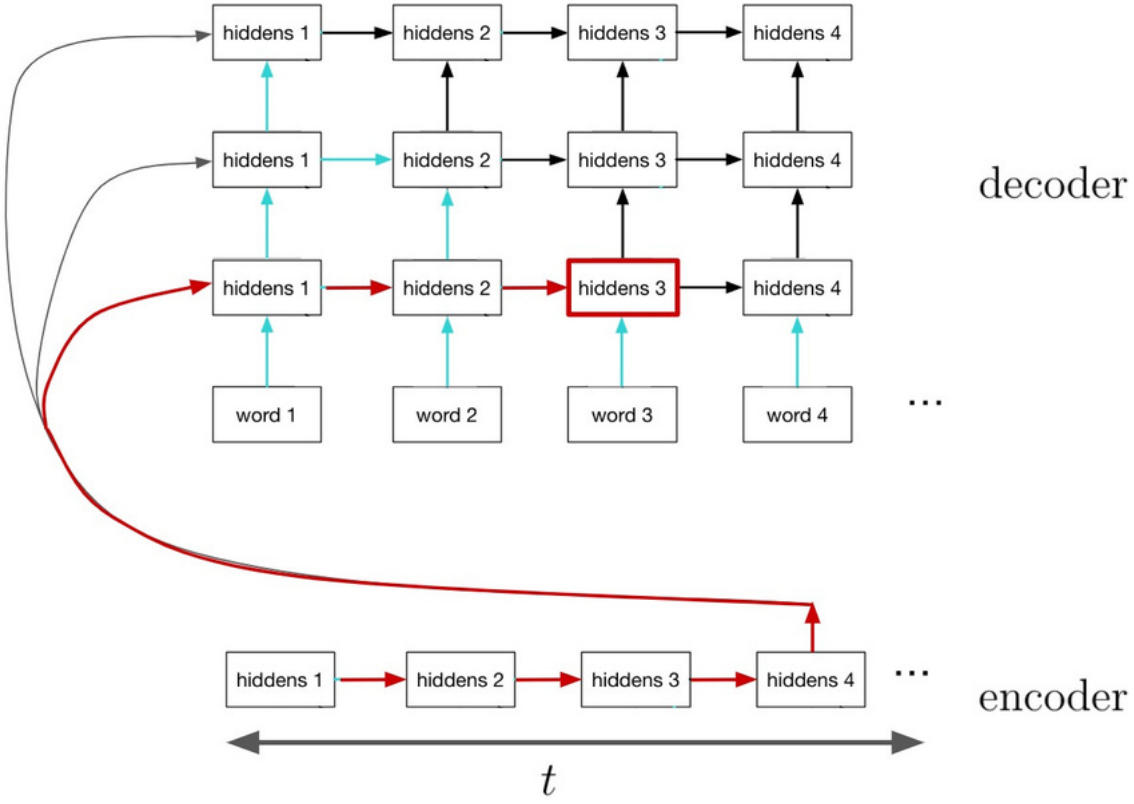


- RNNsearch permite una traducción mucho más confiable de oraciones largas que el modelo estándar RNNencdec. (Puntuación BLEU: una métrica para la calidad de traducción)
- Los modelos basados en atención (RNNsearch) mantienen un rendimiento consistentemente alto incluso cuando las oraciones se vuelven más largas (hasta 60 palabras)
- El enfoque estándar codificador-decodificador (RNNenc) se degradó significativamente con el aumento de la longitud de las oraciones.

Flujo de información en el codificador-decodificador estándar RNN

En el codificador-descodificador estándar RNN, la longitud máxima de la ruta a través del tiempo es proporcional al número de pasos de tiempo.

La información de la primera entrada debe recorrer muchos pasos para llegar a la tercera entrada, lo que crea largas dependencias.

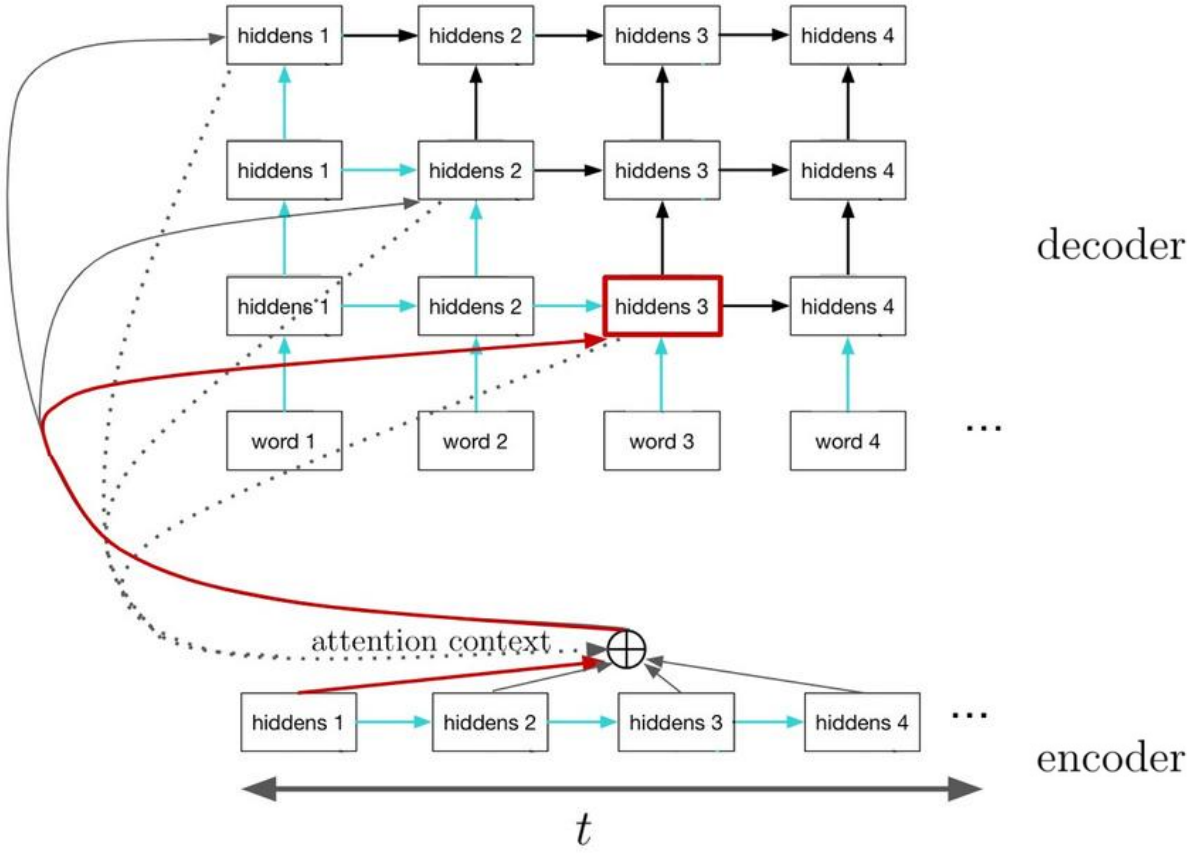


Flujo de información en RNN basadas en la atención

Lograr un direccionamiento eficiente basado en contenido al crear conexiones directas entre cualquier entrada y salida.

Recalcular el vector de contexto en cada paso de tiempo

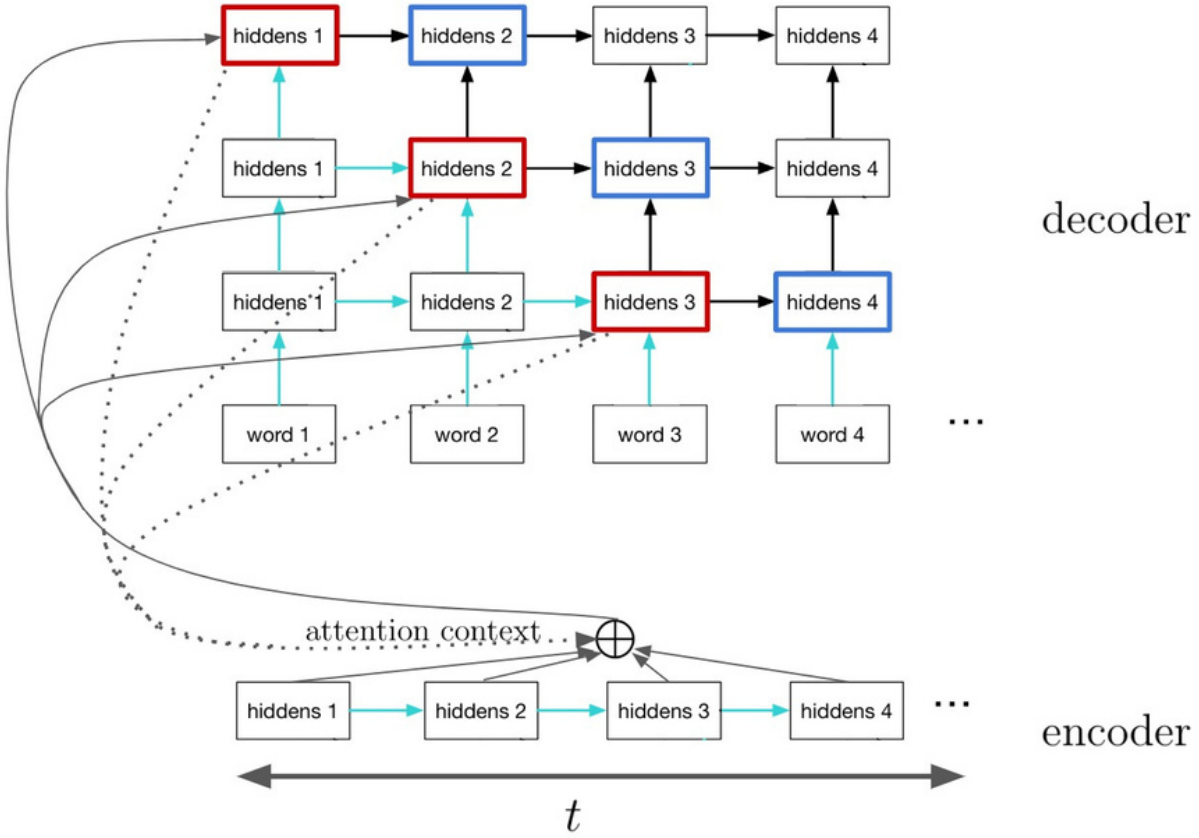
- agregando un costo t * k² en cada paso



Compensaciones computacionales en las RNN basadas en la atención

RNN basada en atención tiene una longitud de ruta constante entre las entradas del codificador y los estados ocultos del decodificador.

La atención crea un atajo desde la primera entrada hasta la tercera salida al omitir el procesamiento secuencial.





Resumen: Costo computacional y paralelismo de las RNN basadas en la atención

Training/Test Complexity and Memory

Model	Training Complexity	Training Memory	Test Complexity	Test Memory
RNN	$k^2 * d * t$	$k * d * t$	$k^2 * d * t$	$k * d$
RNN + attn.	$k^2 * d * t^2$	$k * d * t^2$	$k^2 * d * t^2$	$k * d * t$

Sequential Complexity

Model	Sequential operations	Max path length across time
RNN	t	t
RNN + attn.	t	1

Bahdanau (Aditiva) vs. Luong (Multiplicativa)

Aspecto	Bahdanau (Aditiva)	Luong (Multiplicativa)
Función score	`v_a tanh(W_a[s;h])`	`s_t . h_i (dot) o s_t.W_a.h_i (general)`
Red neuronal extra	Sí (MLP con tanh)	No (solo producto matricial)
Costo computacional	Mayor (MLP por cada par)	Menor (producto punto O(d))
Paralelismo GPU	Limitado por RNN secuencial	Mejor (base del Transformer)
Influencia en Transformers	Concepto de Cross-Attention	Scaled Dot-Product Attention (base)



Scaled Dot-Product Attention y Transformers

Transformers: una arquitectura de red neuronal revolucionaria

- Arquitectura dominante para el procesamiento del lenguaje natural, generación de código, y probablemente todos los chatbots de LLM
- Reemplazó las redes convolucionales en la visión por computadora
- Habilitó avances como AlphaFold en el plegamiento de proteínas.

Idea clave: utiliza la autoatención para capturar dependencias de largo alcance.

Attention Is All You Need
(Atención es Todo lo que Necesitas)

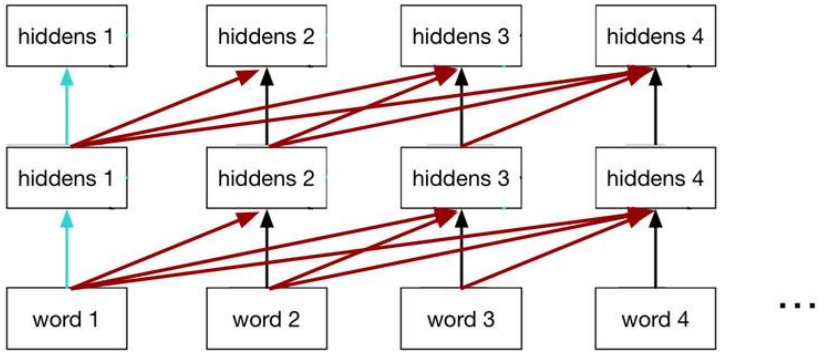


Transformers: Por qué los necesitamos

- **Las RNNs** tienen alta complejidad secuencial (d + t) y son costosas para secuencias muy largas.
- **Las CNNs** autoregresivas resuelven el cuello de botella secuencial pero tienen dificultades con dependencias de larga distancia.

Transformers

- permiten que cada posición haga referencia a toda la historia.
- hacen que el número de operaciones secuenciales sea independiente de la longitud de la secuencia.



Configuración del problema

•Considere la predicción **secuencial a secuencia** con datos de texto (por ejemplo, traducción automática).

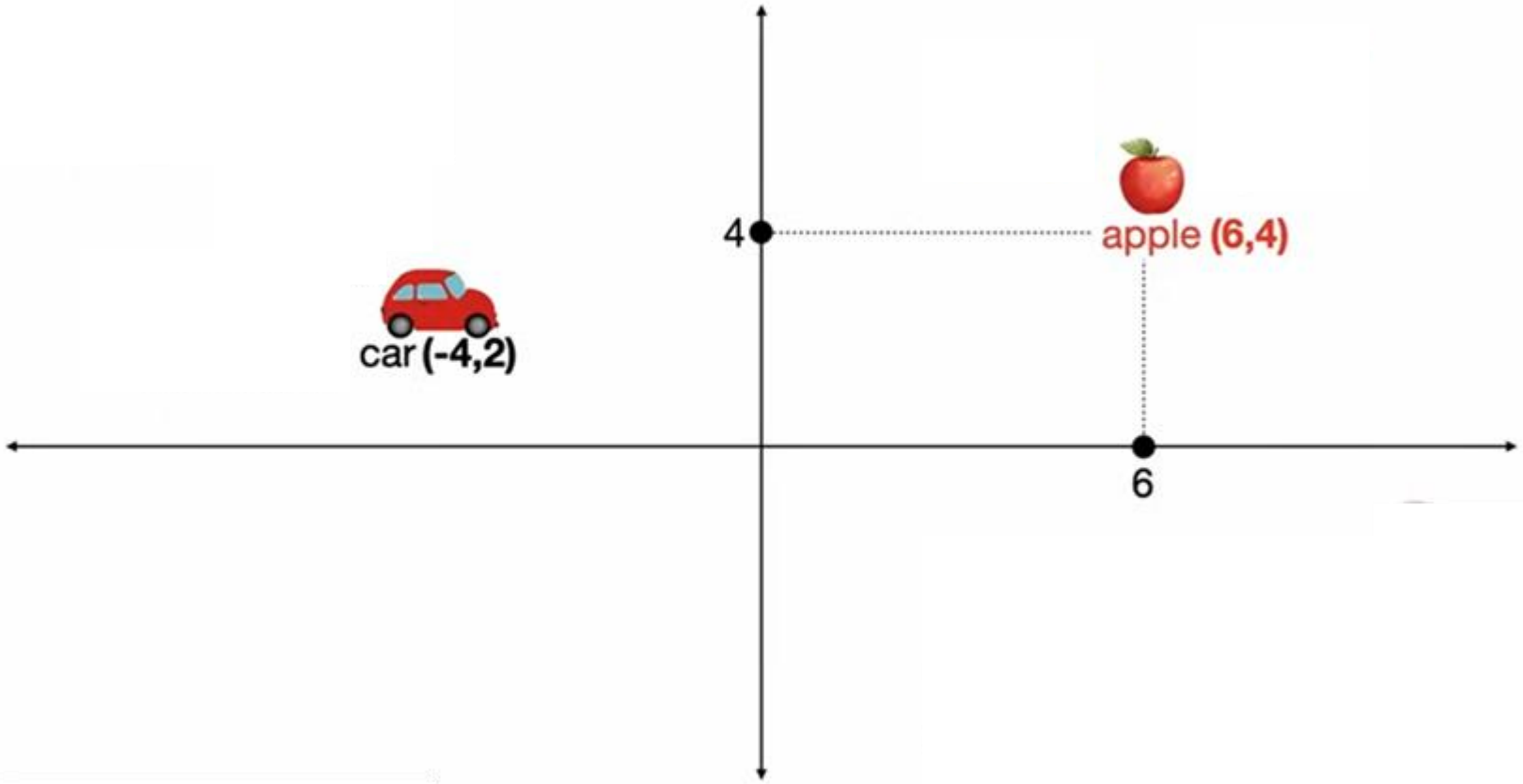
•El texto se divide en tokens (unidad semántica similar a las palabras).

•El **vocabulario** tiene decenas de miles de tokens seleccionados mediante compresión.

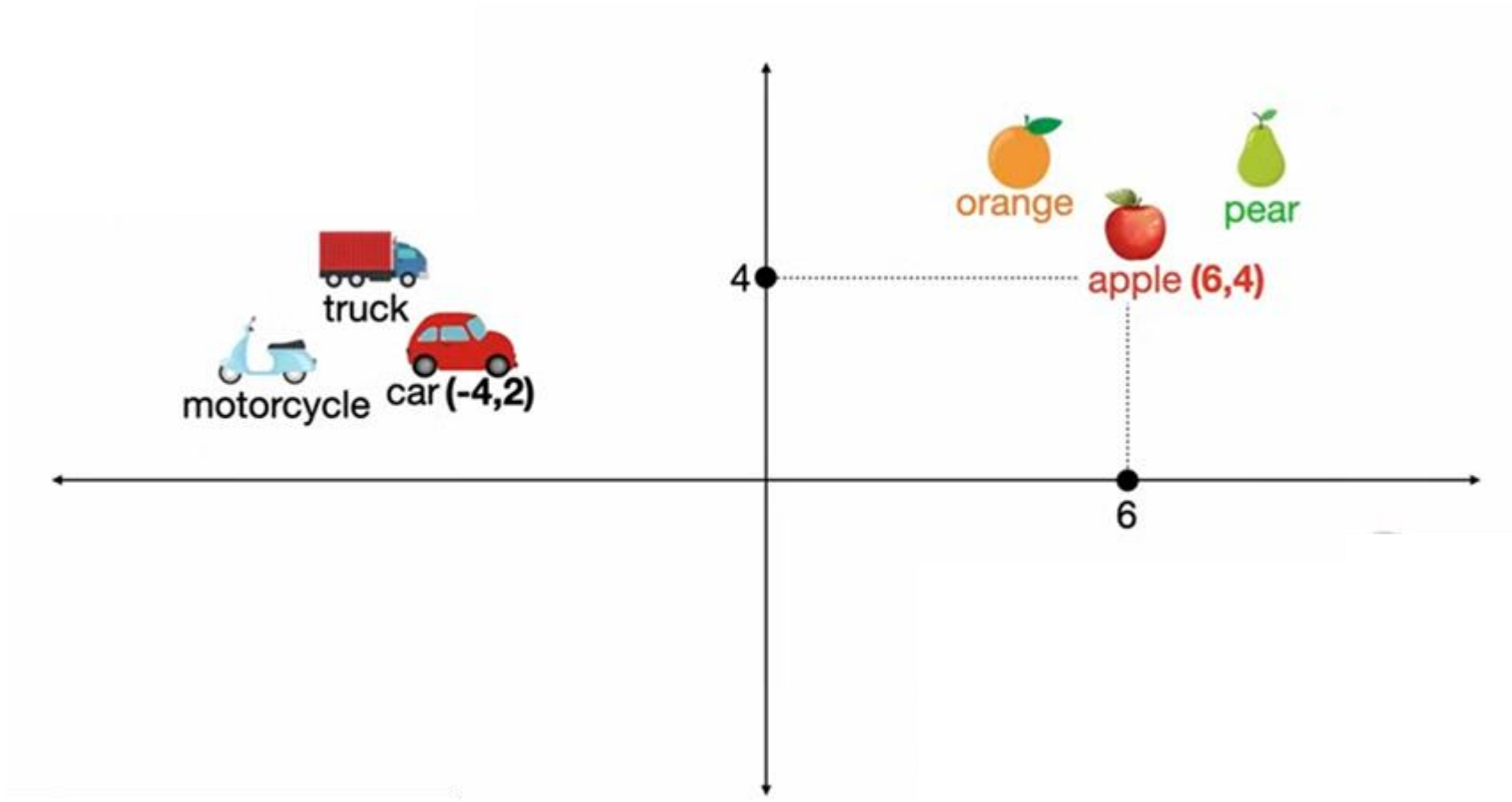
•La primera capa consta de ***embeddings*** **de tokens**

¿Qué era un *embedding*?

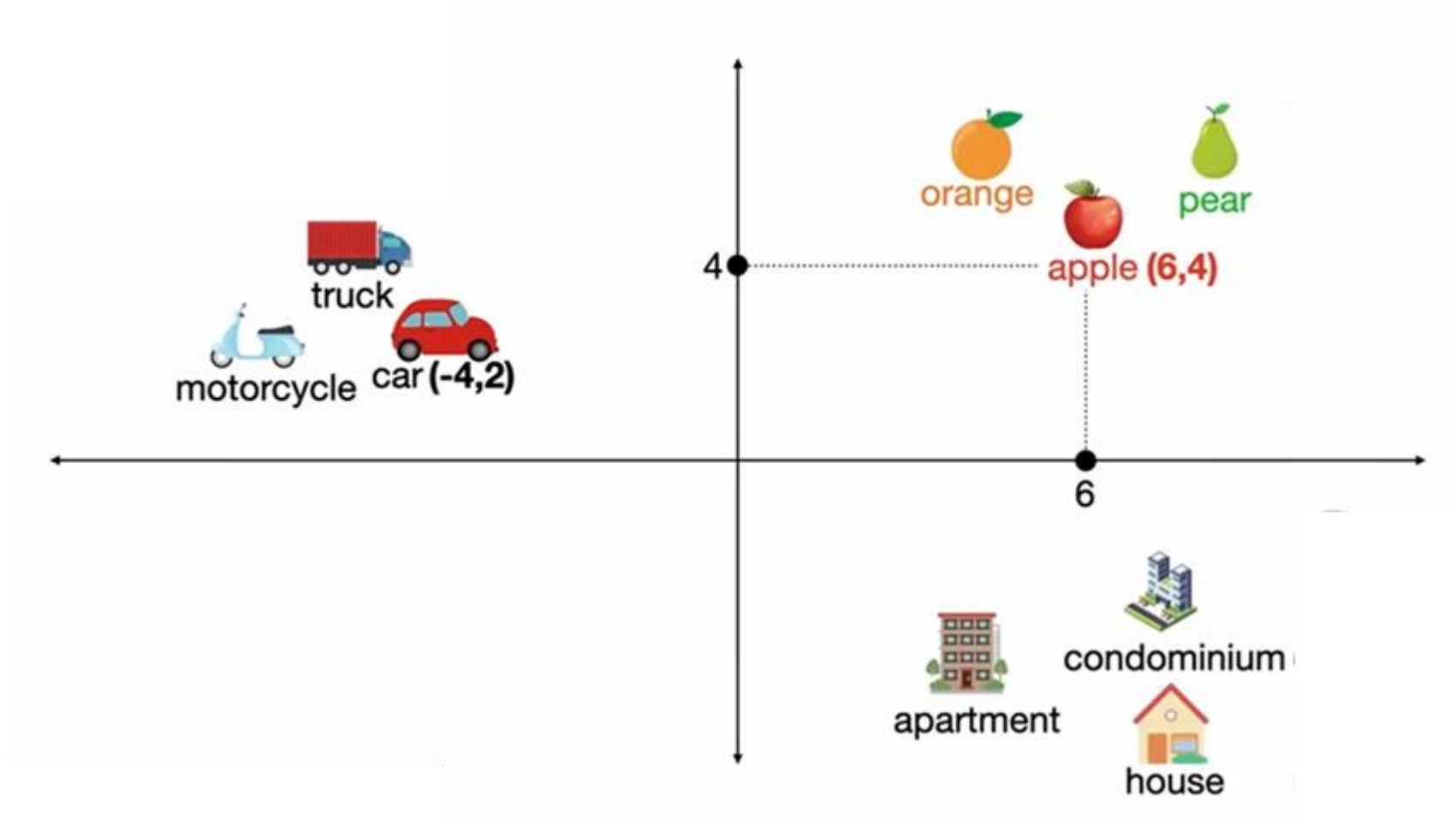
Embeddings: De Palabras a Números



Embeddings: De Palabras a Números



Embeddings: De Palabras a Números

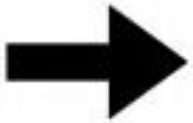


Embeddings de Palabras: Muchas Más Columnas

Word	Numbers	
Apple	5	5
Soccer	0	6
House	2	2
Car	6	0

Embeddings de Palabras: Muchas Más Columnas

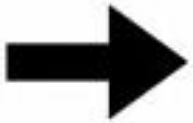
Word	Numbers	
Apple	5	5
Soccer	0	6
House	2	2
Car	6	0



Word		Numbers	
A	-0.82	-0.32	...
Aardvark	0.419	1.28	...
...			...
Zygote	-0.74	-1.02	...

Embeddings de Palabras: Muchas Más Columnas

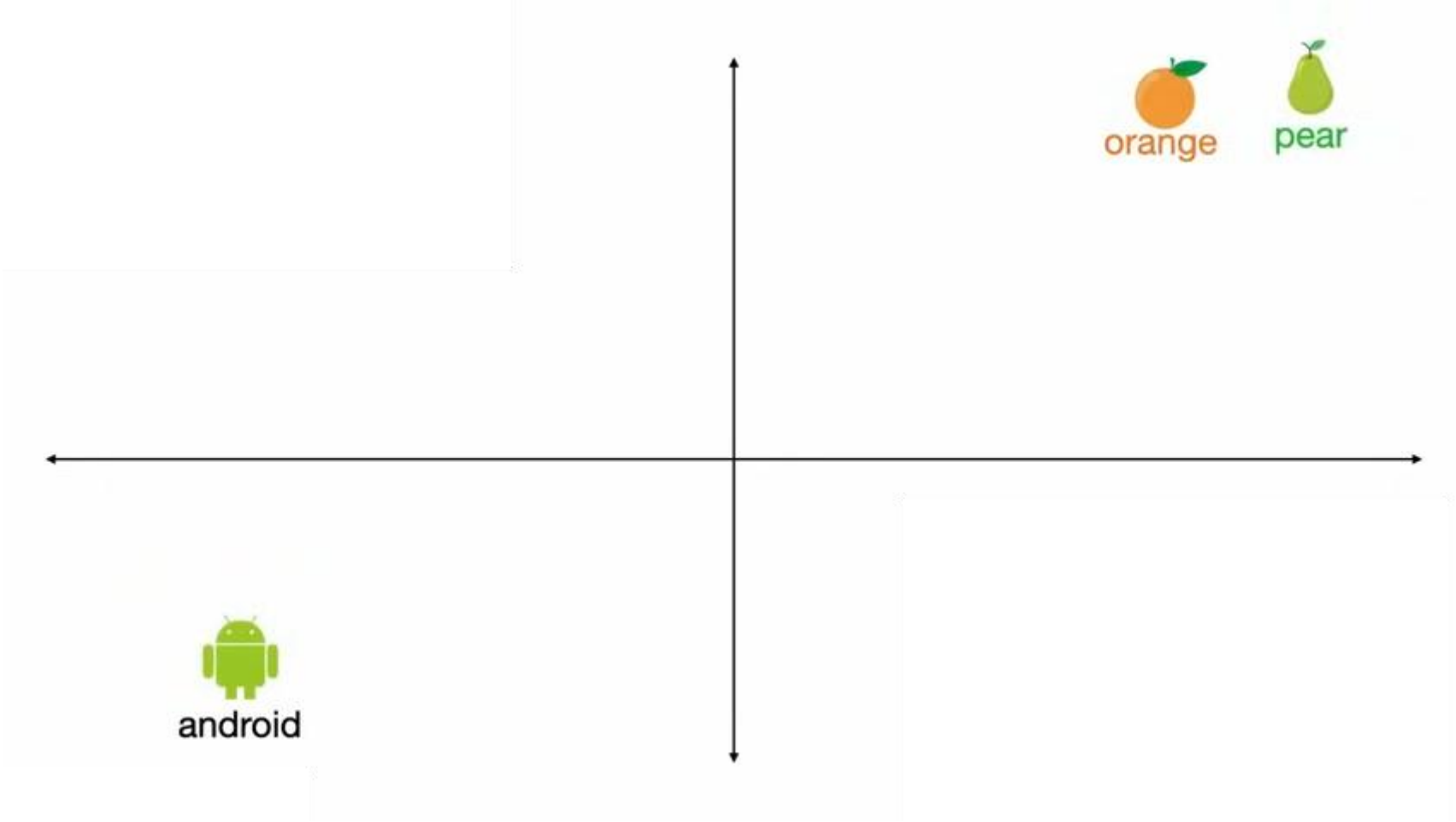
Word	Numbers	
Apple	5	5
Soccer	0	6
House	2	2
Car	6	0



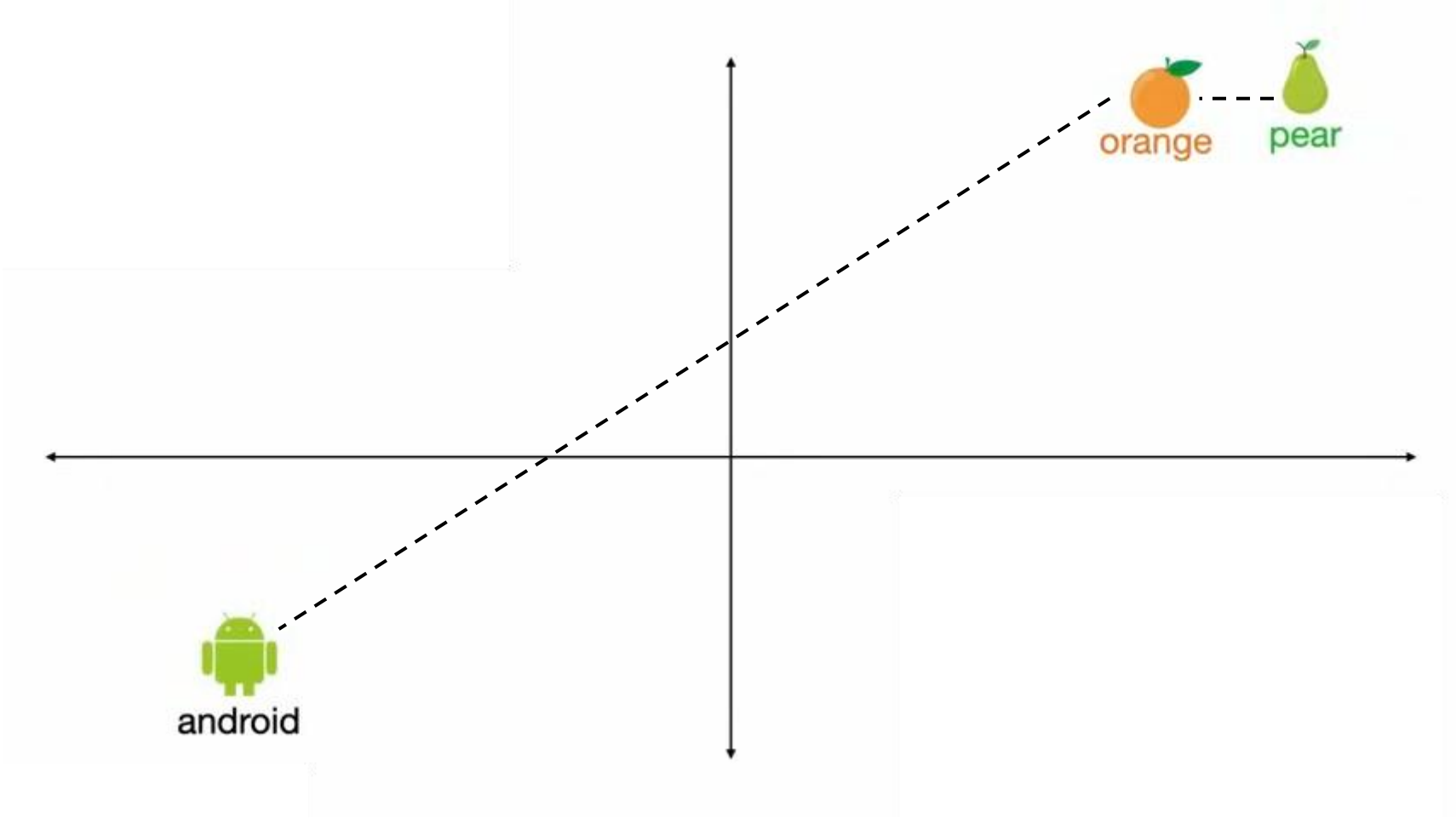
¿Qué nombre pueden tener?

Word		Numbers	
A	-0.82	-0.32	...
Aardvark	0.419	1.28	...
...			...
Zygote	-0.74	-1.02	...

Los Embeddings Definen la Similitud a Través de la Distancia

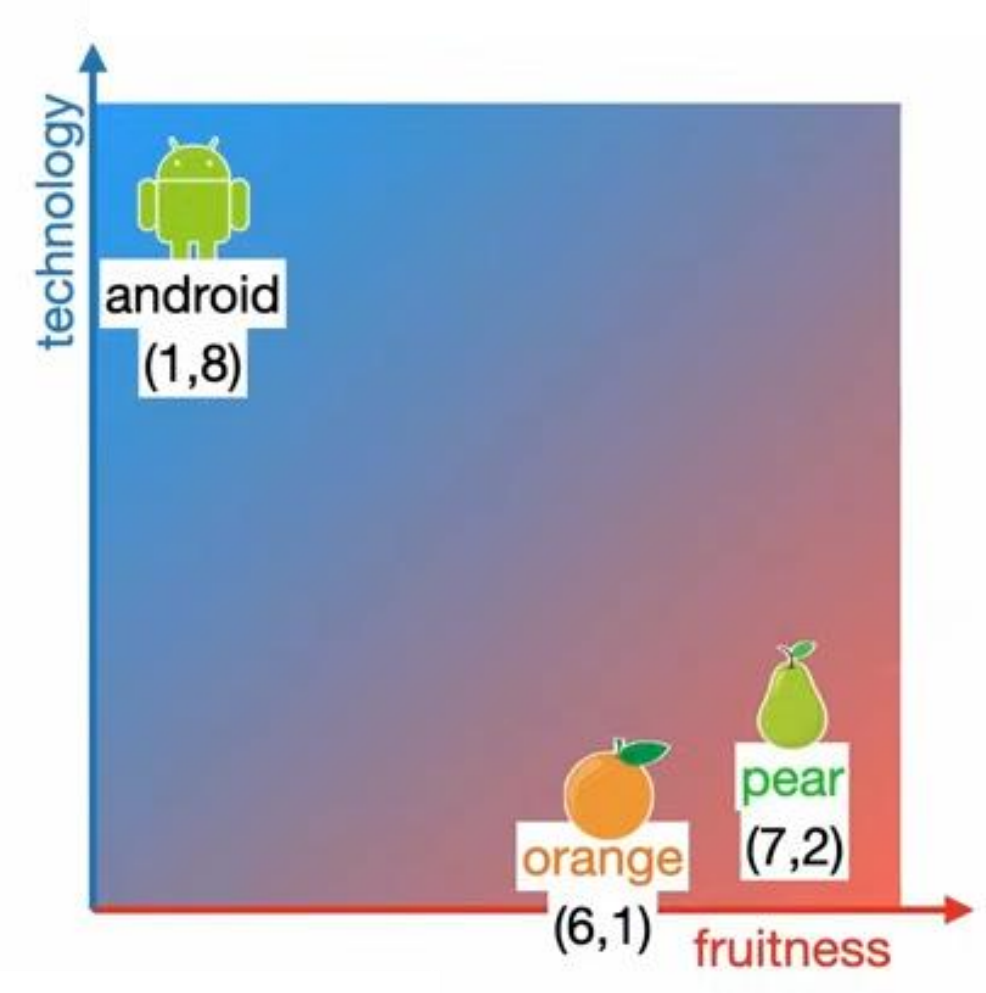


Los Embeddings Definen la Similitud a Través de la Distancia

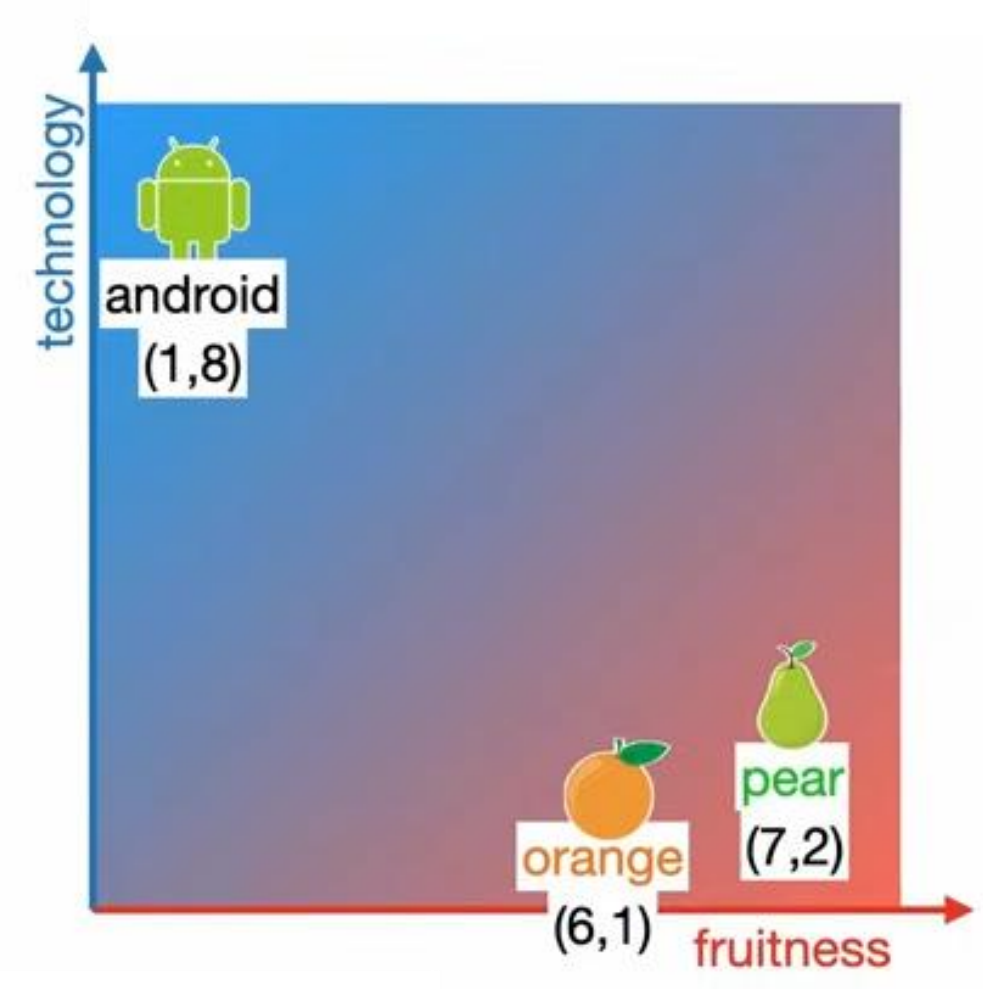


¿Cómo hallamos esta similitud?

Dos Nociones de Similitud: Producto Escalar



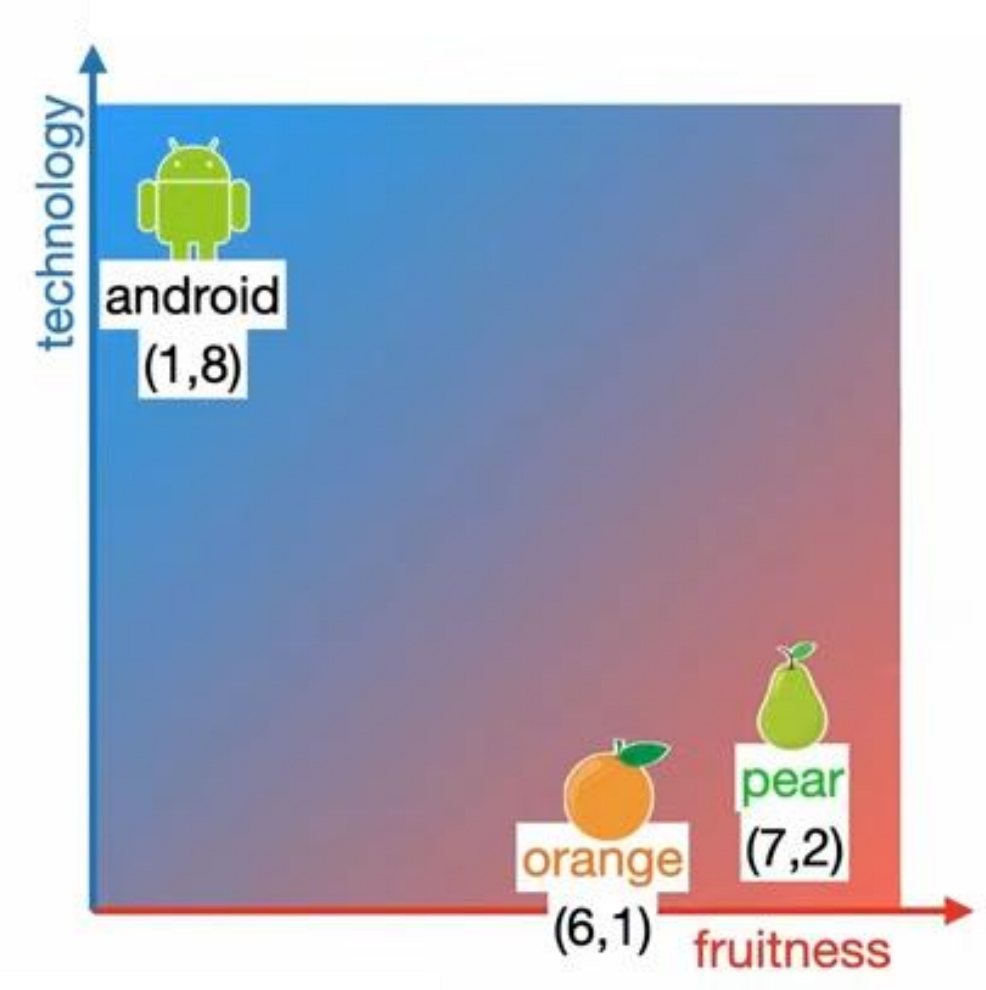
Dos Nociones de Similitud: Producto Escalar

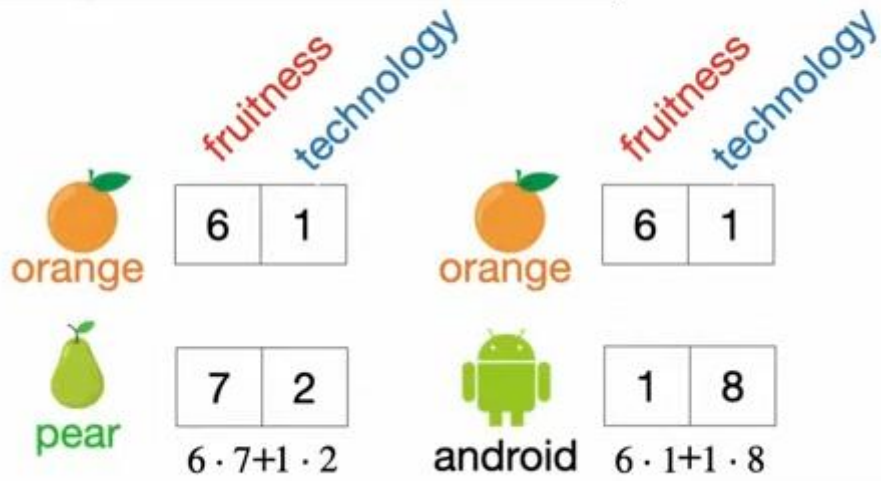


	fruitness	technology
orange	6	1
pear	7	2

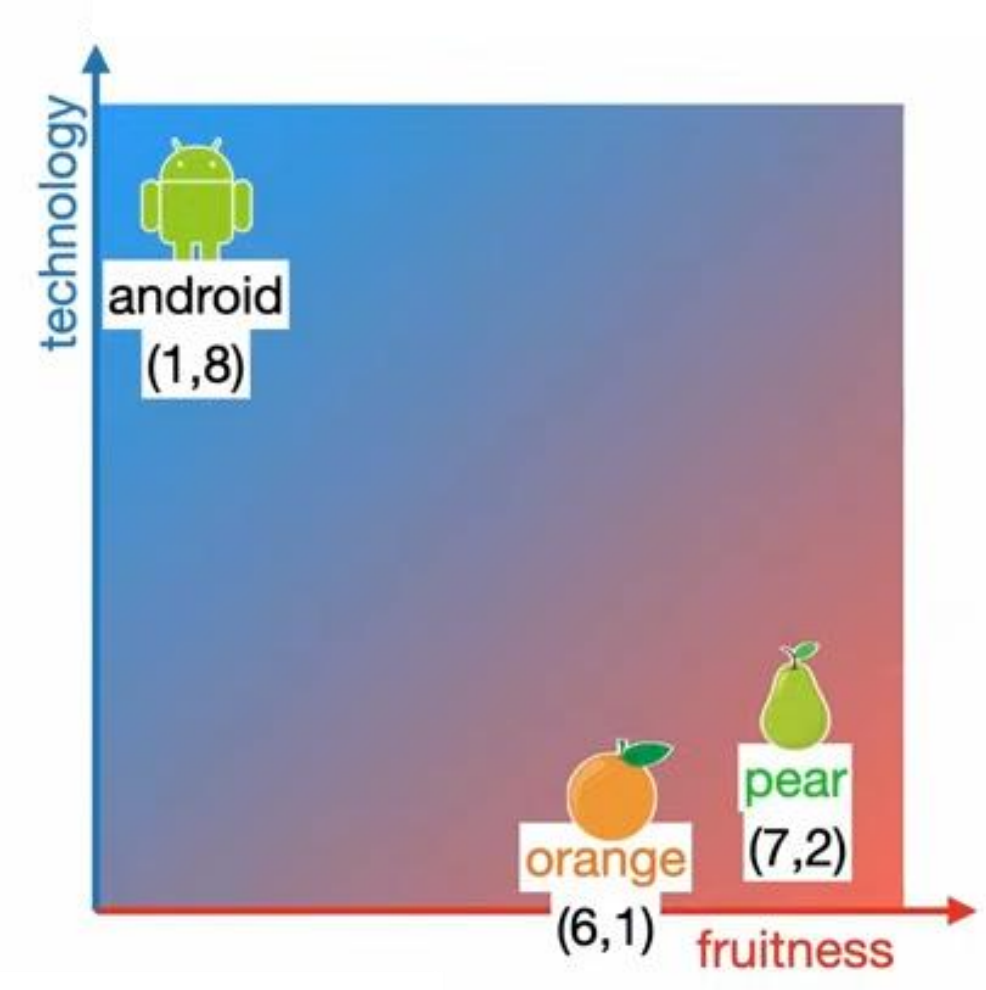
	fruitness	technology
orange	6	1
android	1	8

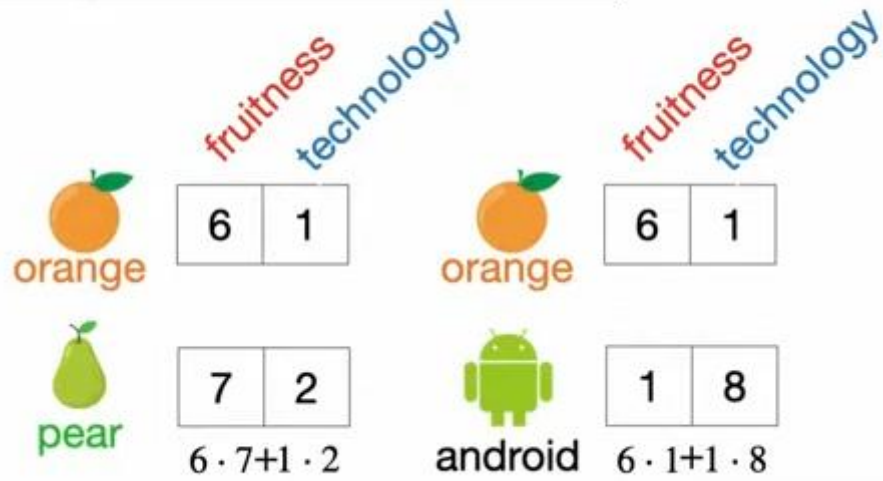
Dos Nociones de Similitud: Producto Escalar





Dos Nociones de Similitud: Producto Escalar

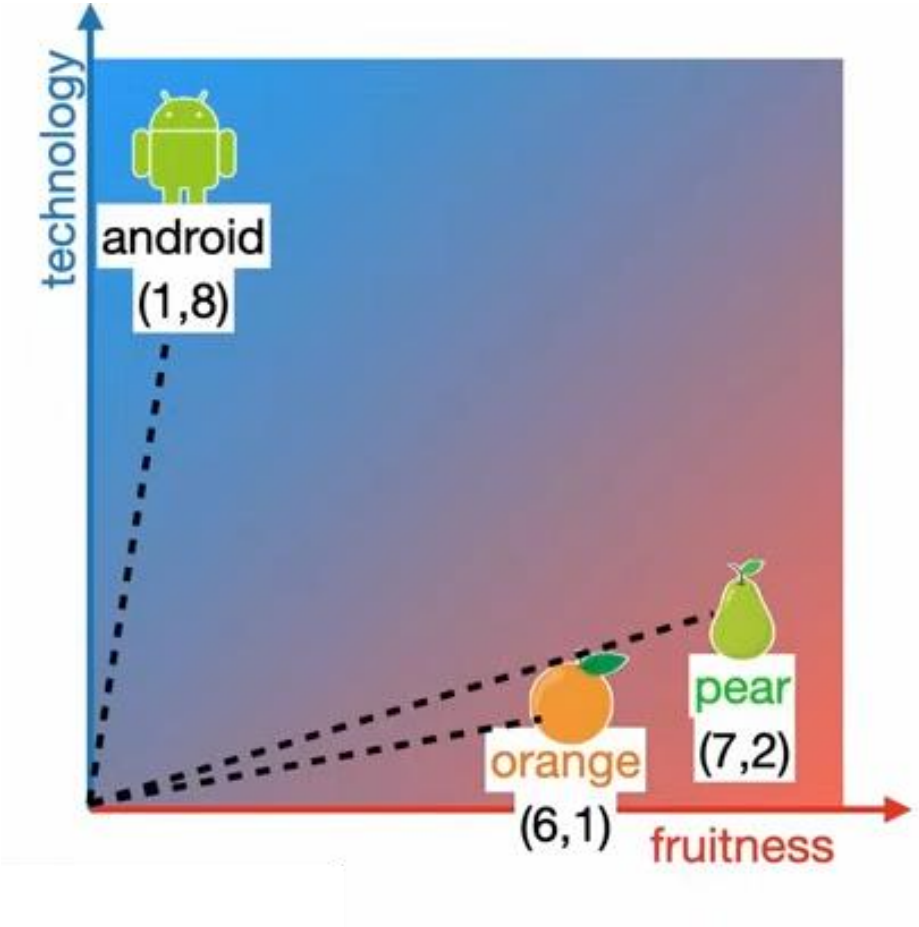




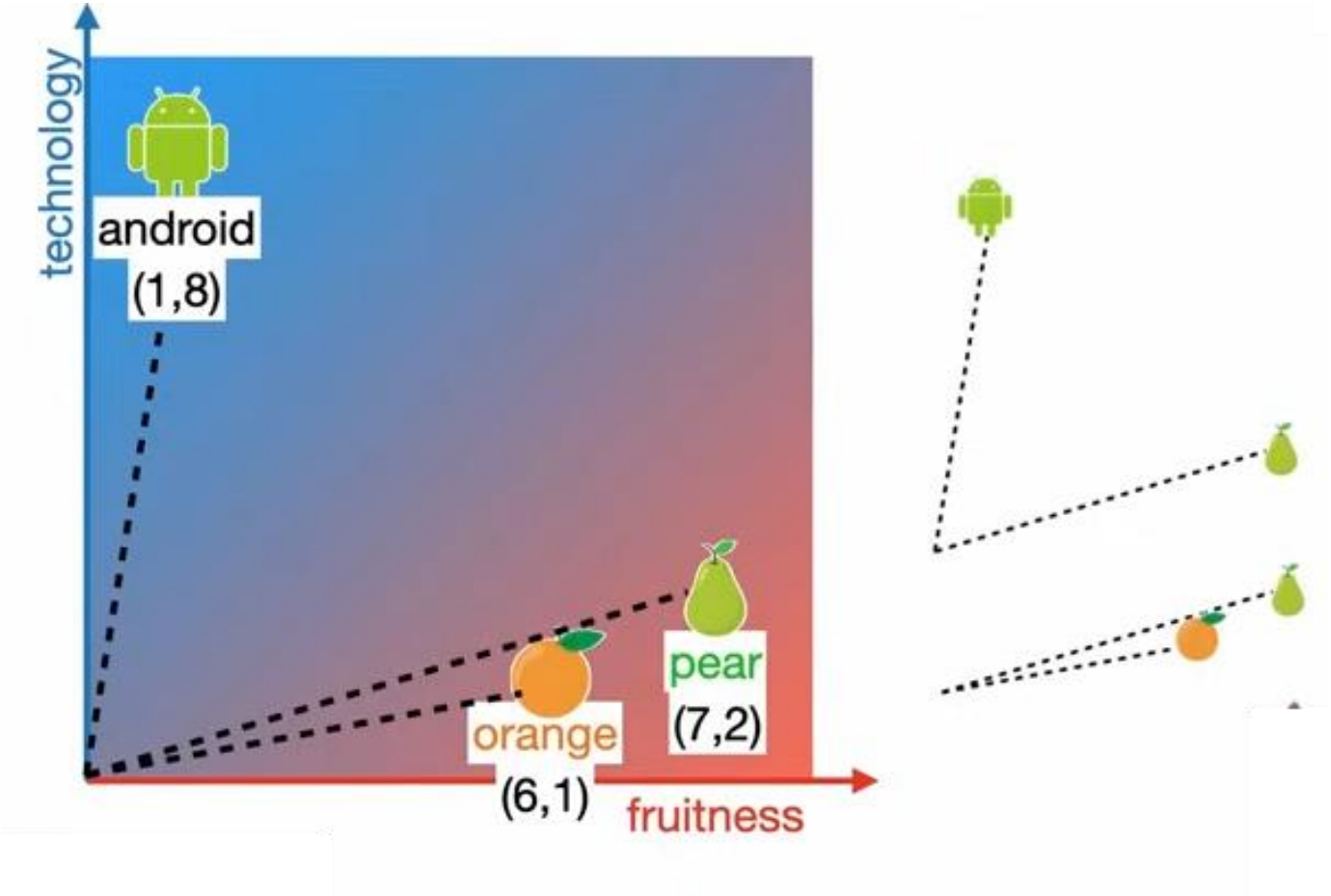
Producto punto grande — Producto punto pequeño

Gran similitud — Poca similitud

Dos Nociones de Similitud: Distancia del Coseno



Dos Nociones de Similitud: Distancia del Coseno



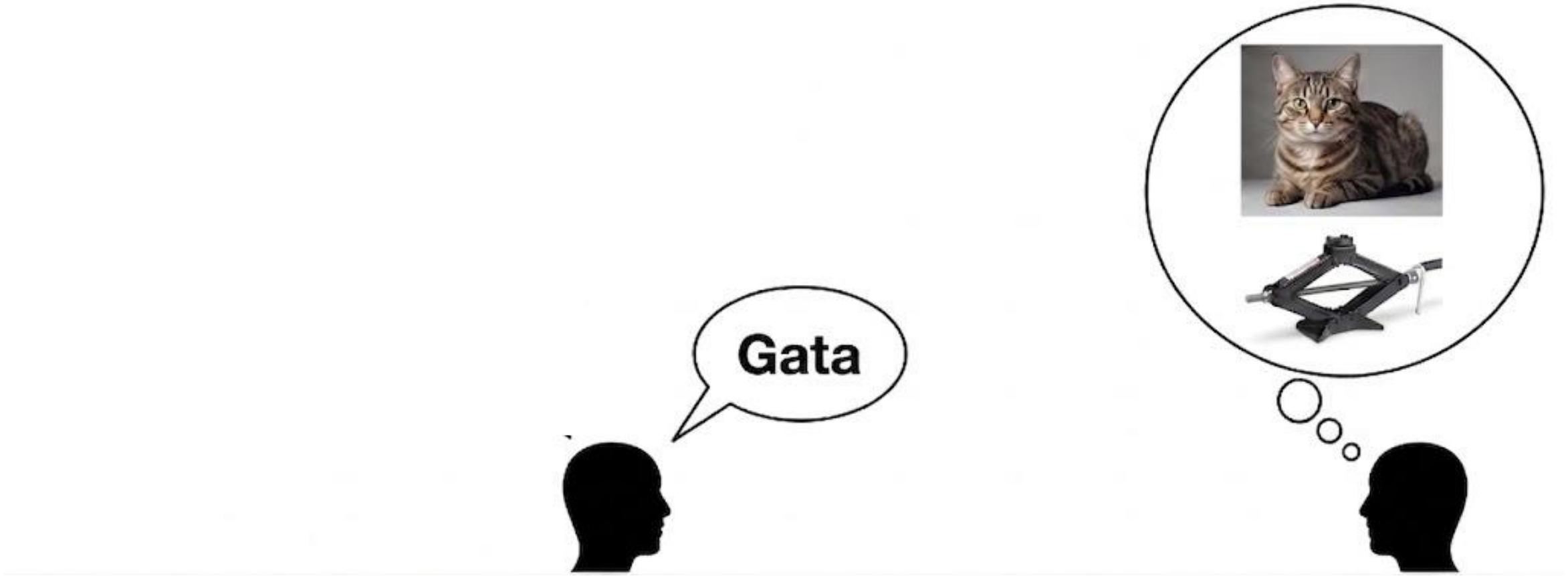
Ángulo grande
Coseno pequeño
Poca similitud

Ángulo pequeño
Coseno grande
Gran similitud



¿Siempre funcionara estos embeddings para todos los casos?

Entendiendo el Contexto con “Gata"



Entendiendo el Contexto con “Gata"



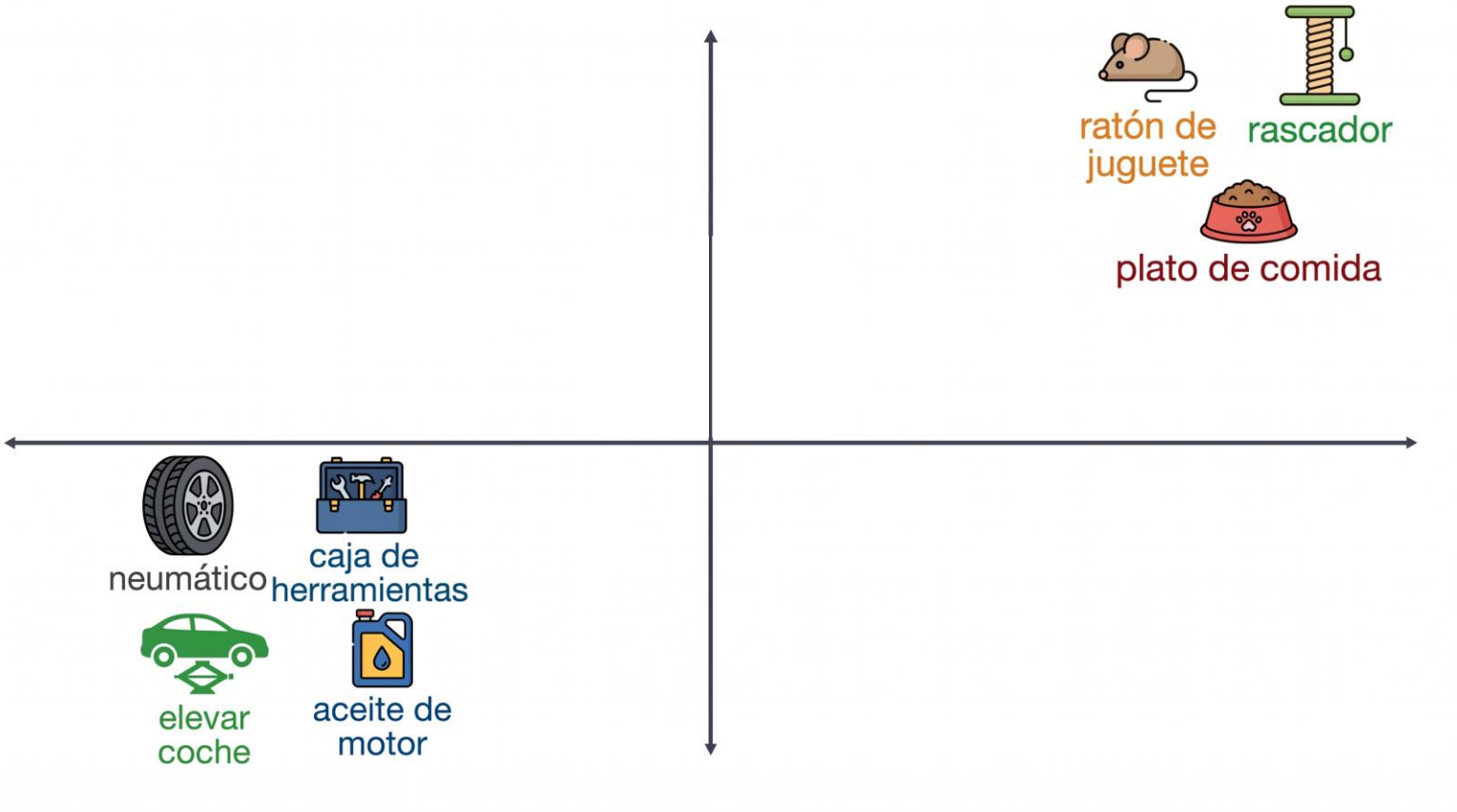
Entendiendo el Contexto con “Gata"



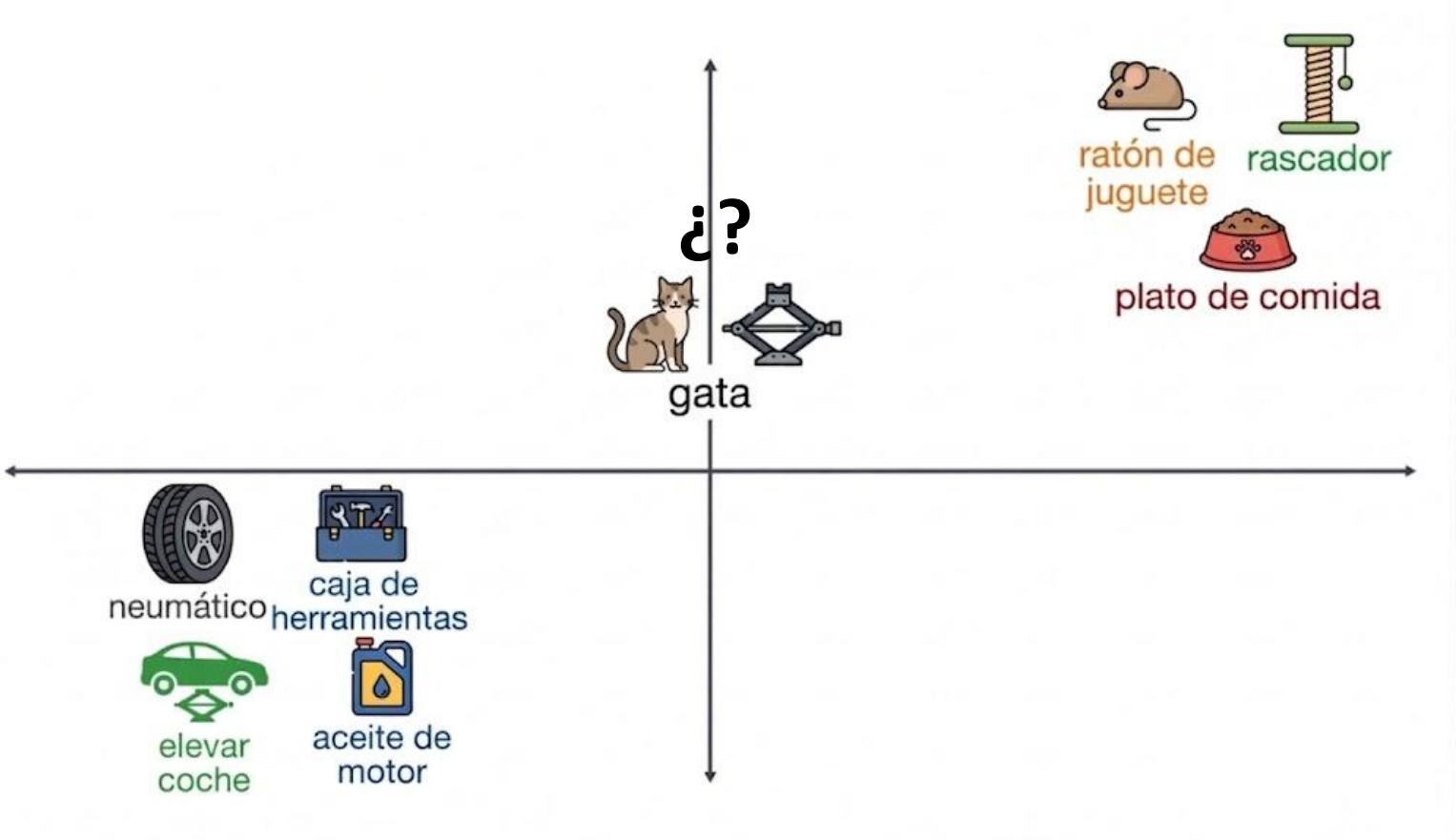
Presta más atención a las palabras más “similares”

Cuanto más similares sean las palabras, más 'atraen'.

¿Cuándo esto no funciona?

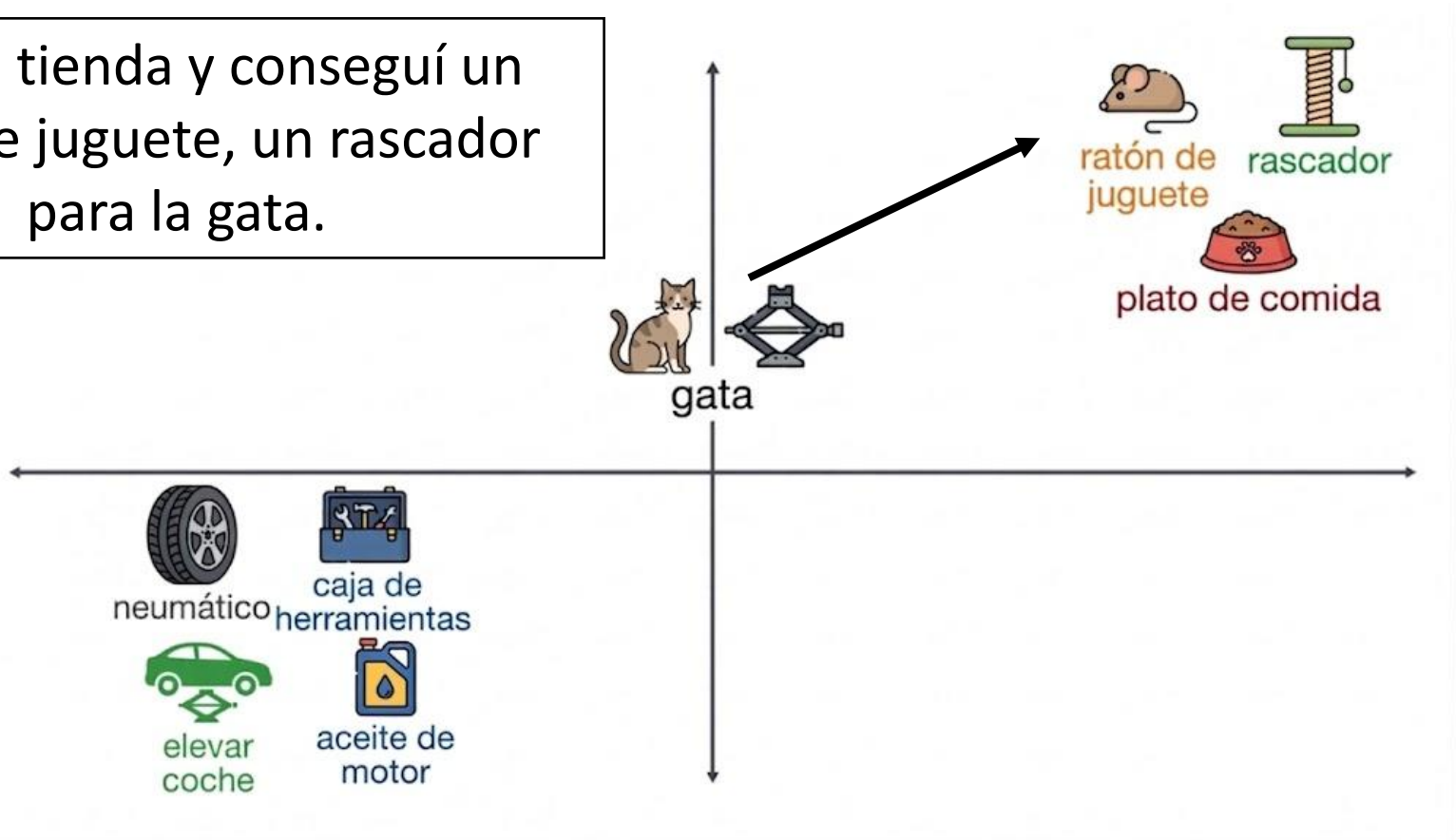


Los Embeddings tienen problemas: El Dilema de “gata"



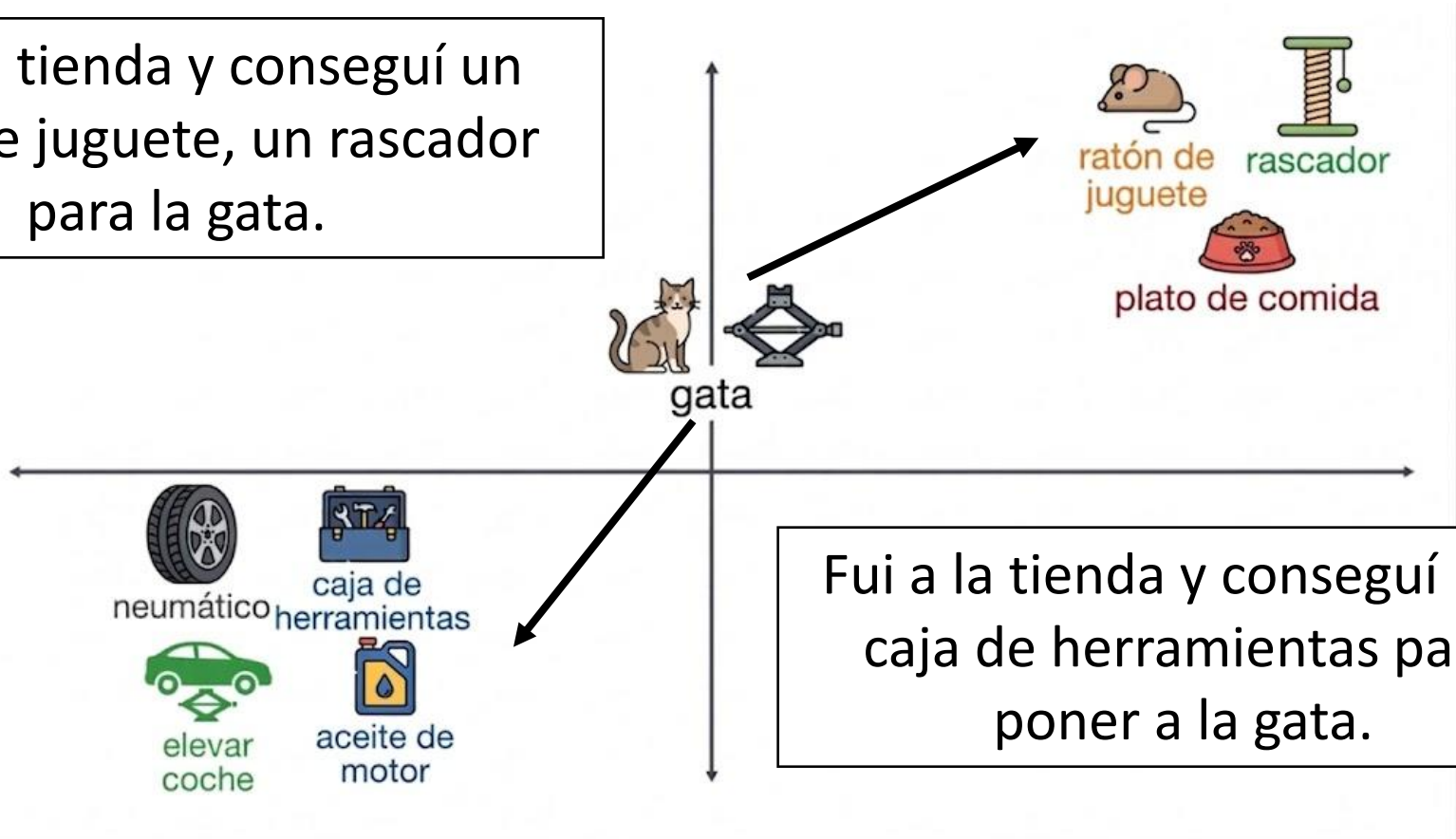
Los Embeddings tienen problemas: El Dilema de “gato"

Fui a la tienda y conseguí un ratón de juguete, un rascador para la gata.



Los Embeddings tienen problemas: El Dilema de “gato"

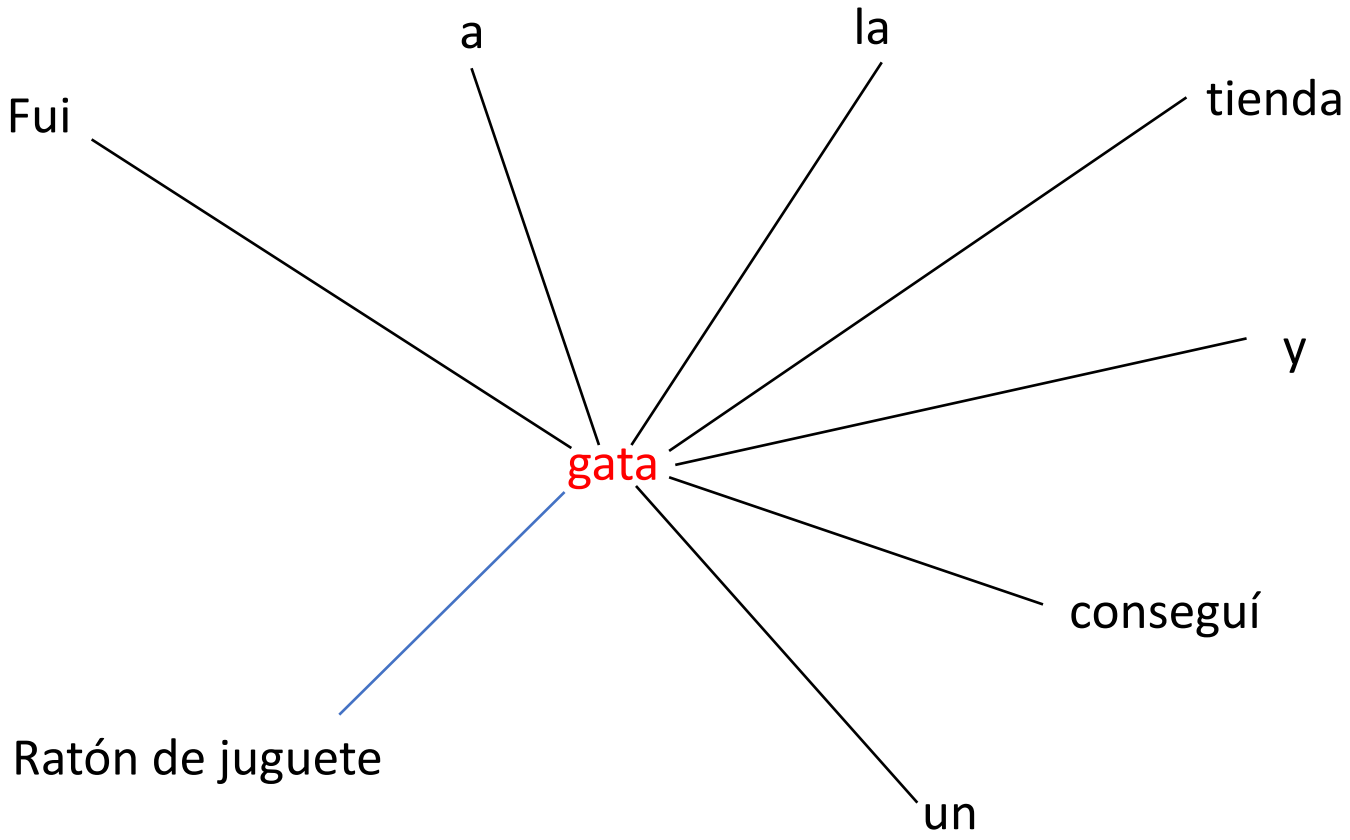
Fui a la tienda y conseguí un ratón de juguete, un rascador para la gata.



Fui a la tienda y conseguí una caja de herramientas para poner a la gata.

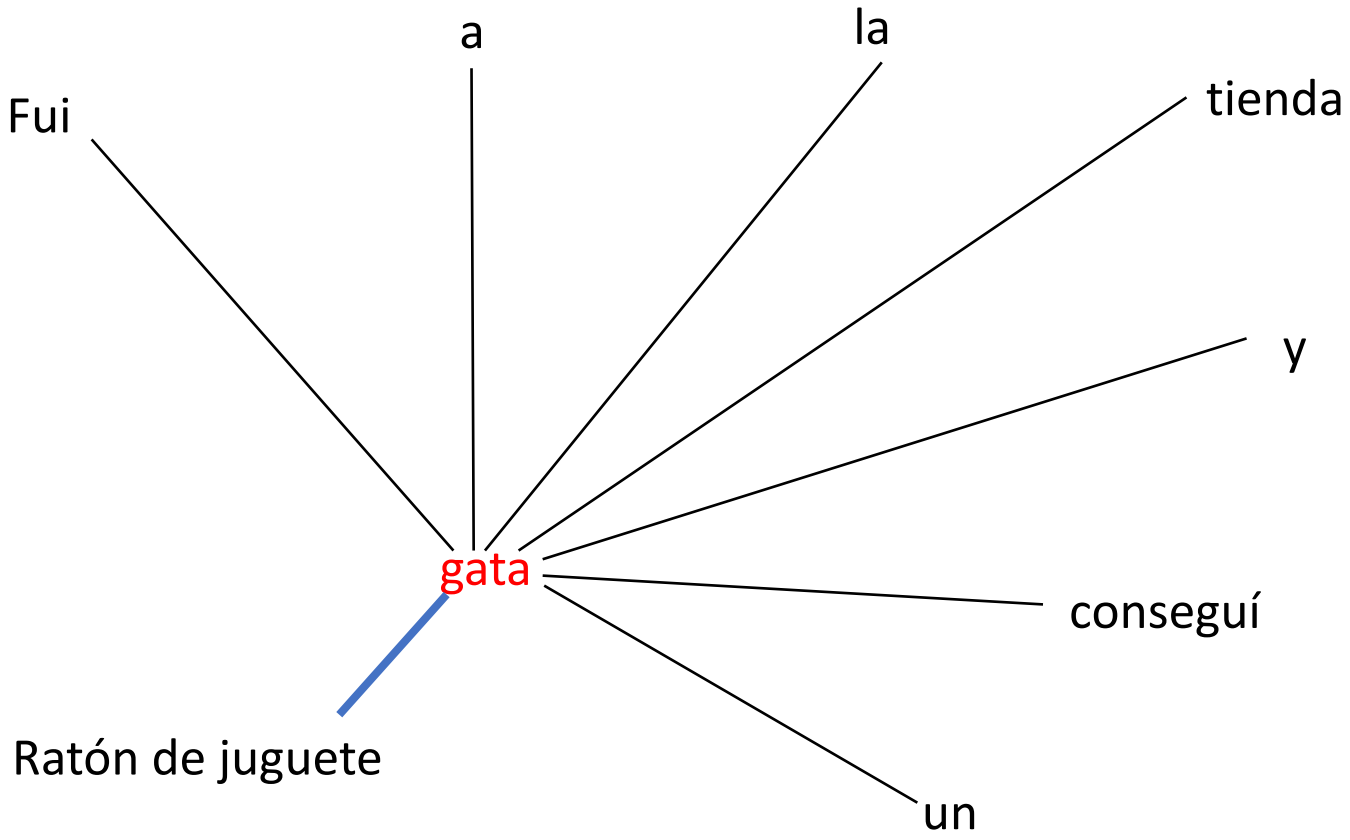
La Atención como "Atracción" Contextual

Atracción más Fuerte para Palabras más Similares



Fui a la tienda y conseguí un ratón de juguete, un rascador para la gata.

Atracción más Fuerte para Palabras más Similares



Fui a la tienda y conseguí un ratón de juguete, un rascador para la gata.

¿Cómo “movemos” los embeddings?

Transformaciones Matriciales

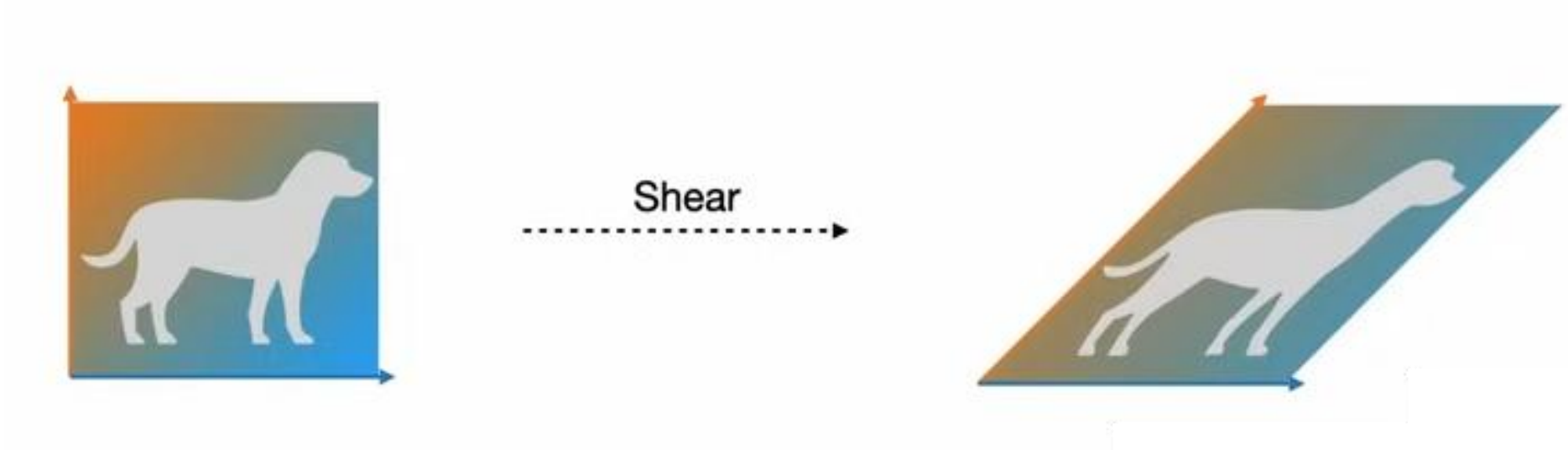


Por esto se llaman “Transformers”

Transformaciones Lineales:
Rotar, Estirar, Sesgar



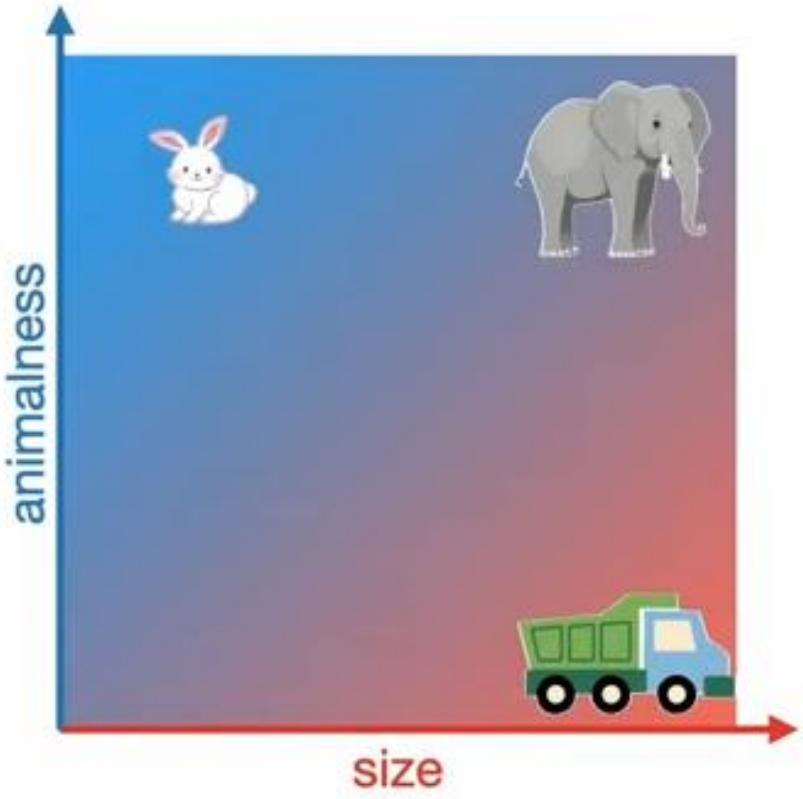
Matrices como Transformaciones Lineales



Matrices como Transformaciones Lineales

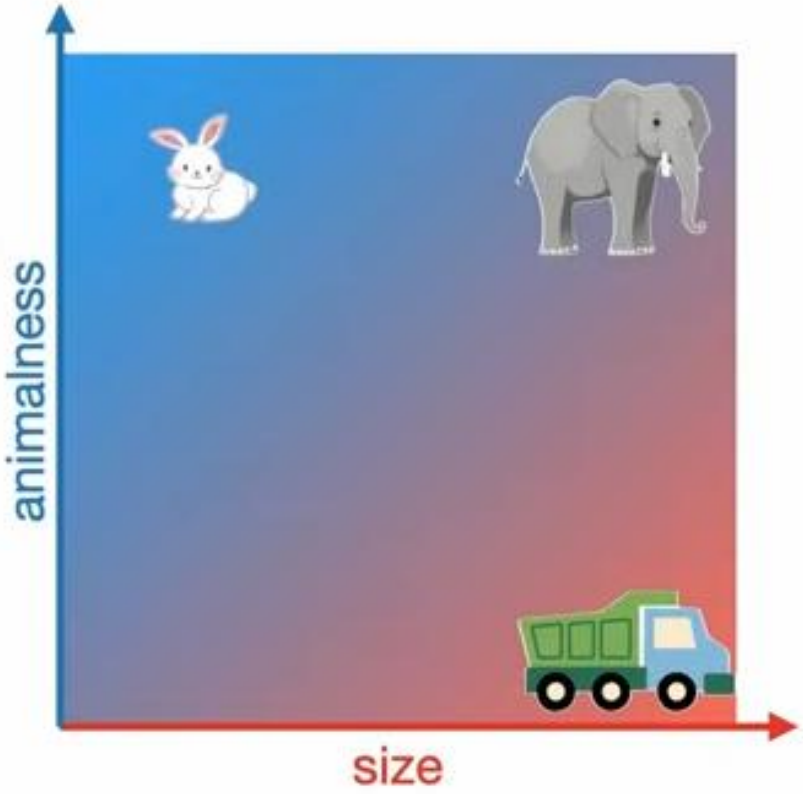


Transformando Embeddings Escalando Características



	size	animalness
Elephant	1	1
Rabbit	0.1	1
Truck	1	0

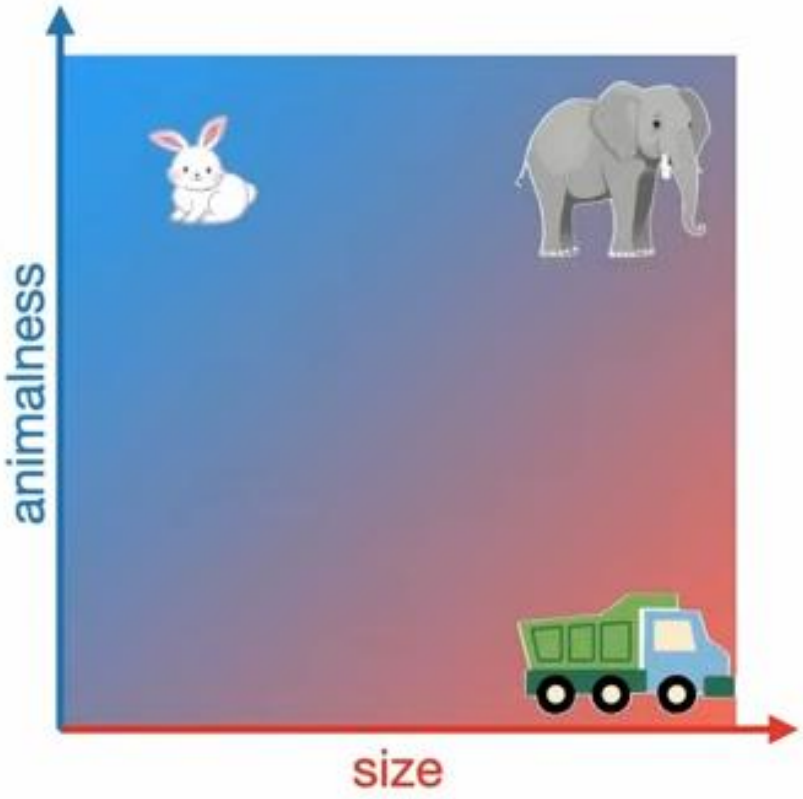
Transformando Embeddings Escalando Características



	size	animalness
Elephant	1	1
Rabbit	0.1	1
Truck	1	0

¿Cómo “juntamos” al conejo y al elefante?

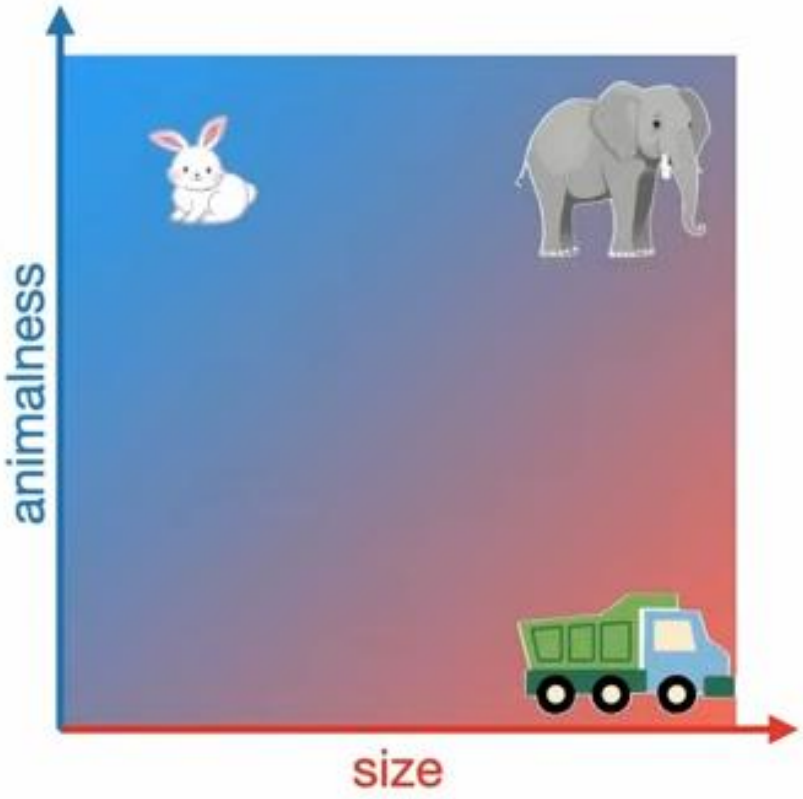
Transformando Embeddings Escalando Características



	size	animalness
Elephant	1	1
Rabbit	0.1	1
Truck	1	0

× 0.3 × 1

Transformando Embeddings Escalando Características



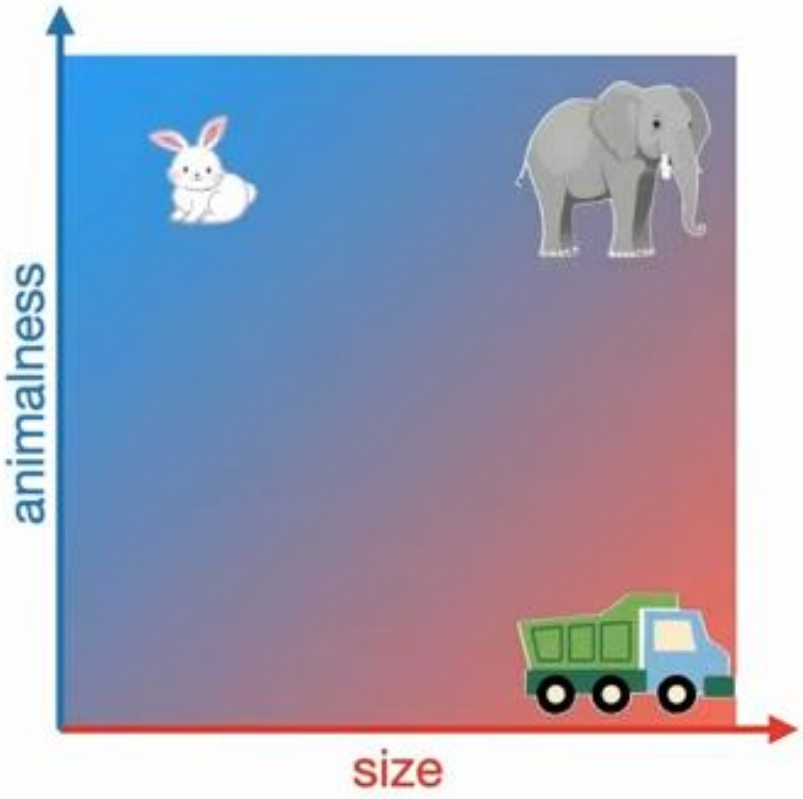
	size	animalness
Elephant	1	1
Rabbit	0.1	1
Truck	1	0

¿Qué matriz usamos?

× 0.3 × 1

	size	animalness
Elephant	0.3	1
Rabbit	0.03	1
Truck	0.3	0

Transformando Embeddings Escalando Características



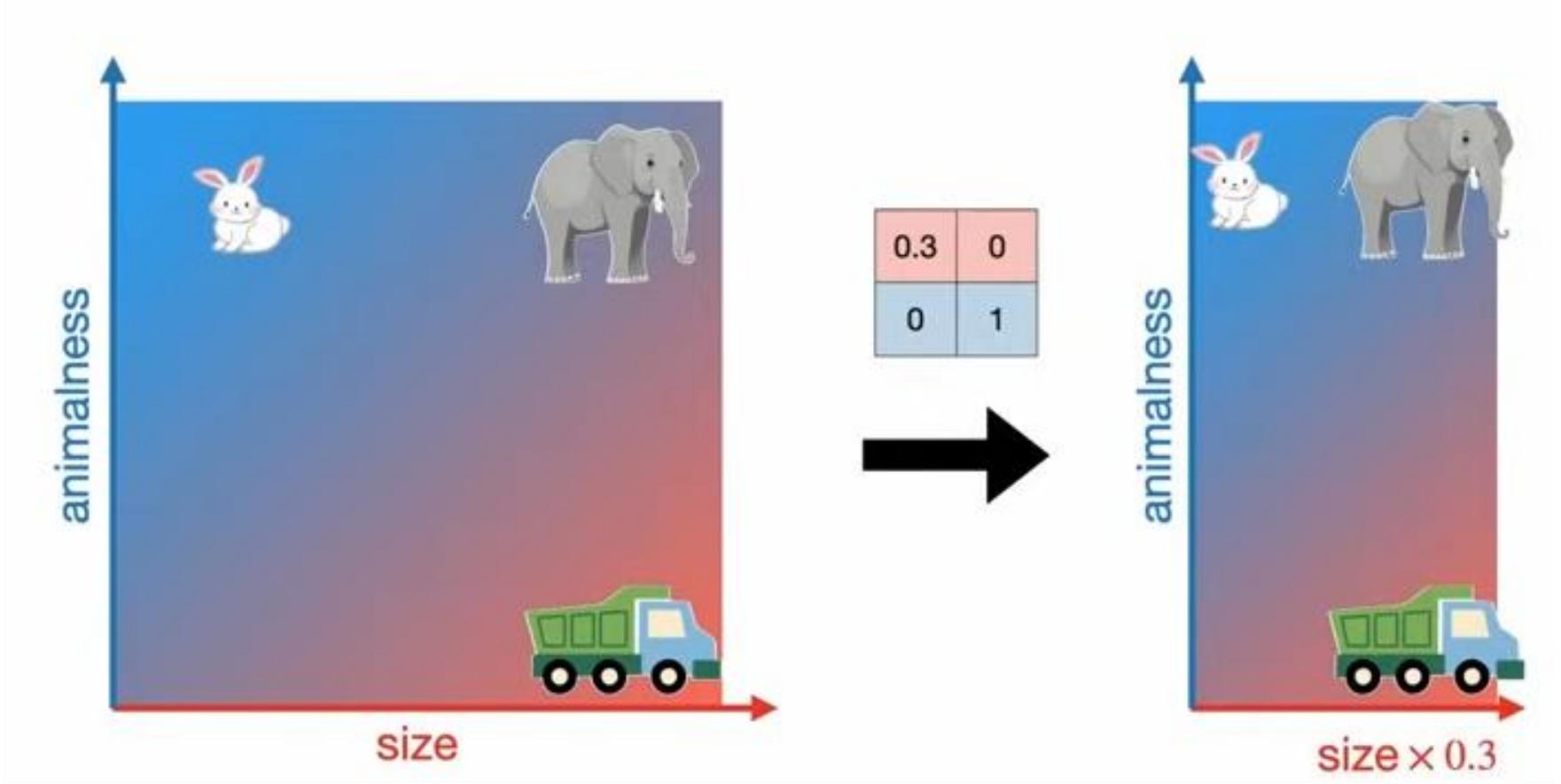
	size	animalness
Elephant	1	1
Rabbit	0.1	1
Truck	1	0

0.3	0
0	1

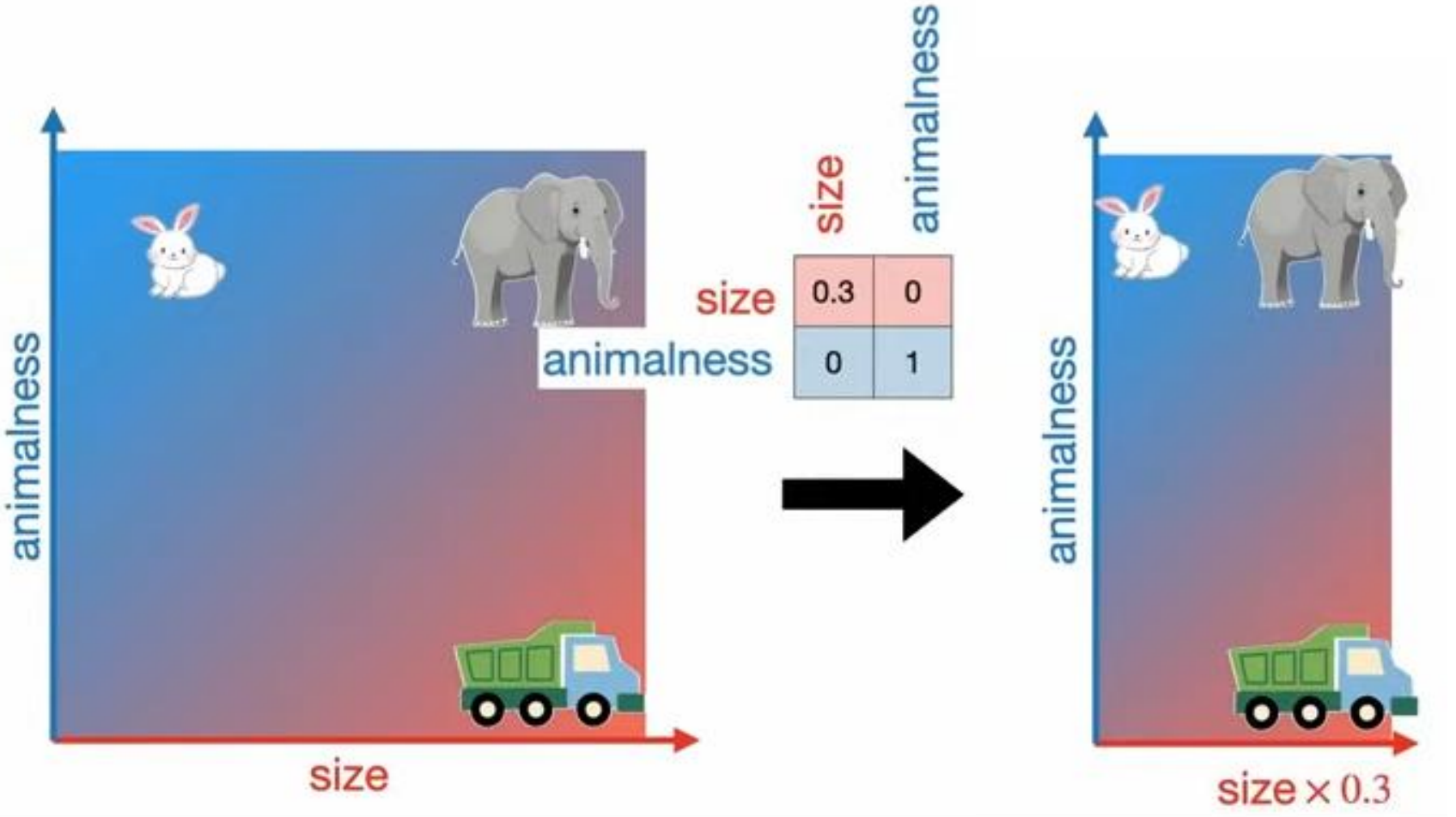
× 0.3 × 1

	size	animalness
Elephant	0.3	1
Rabbit	0.03	1
Truck	0.3	0

Transformando Embeddings Escalando Características

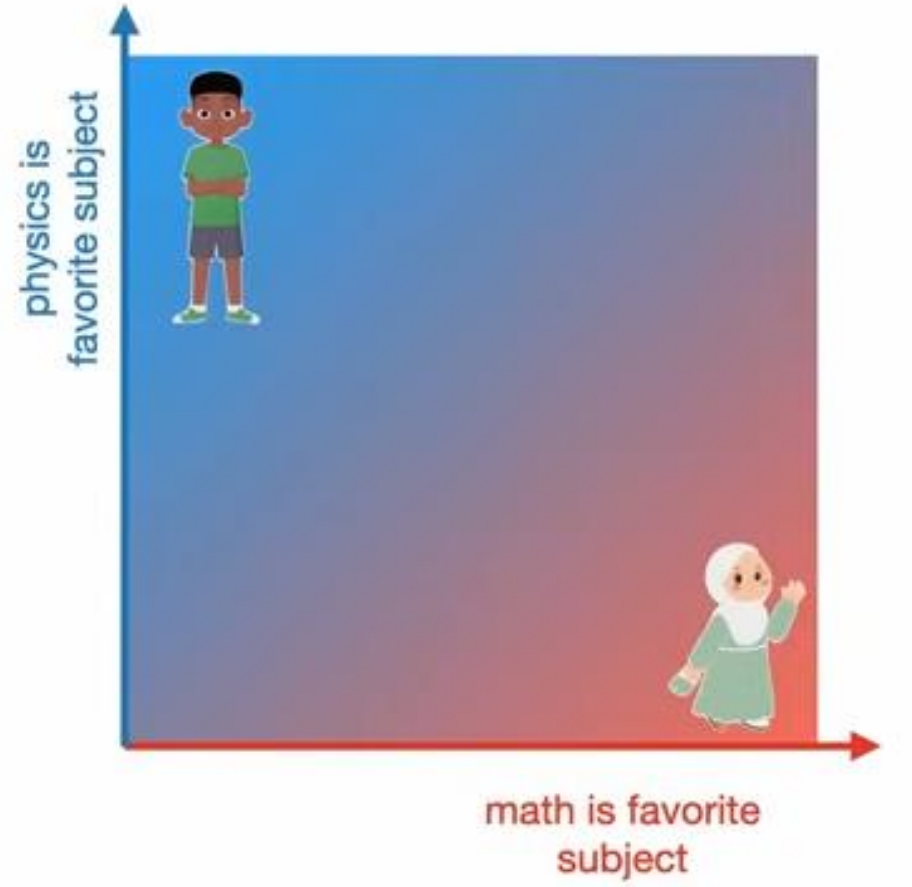


Transformando Embeddings Escalando Características



A Veces Hay que Combinar Dimensiones

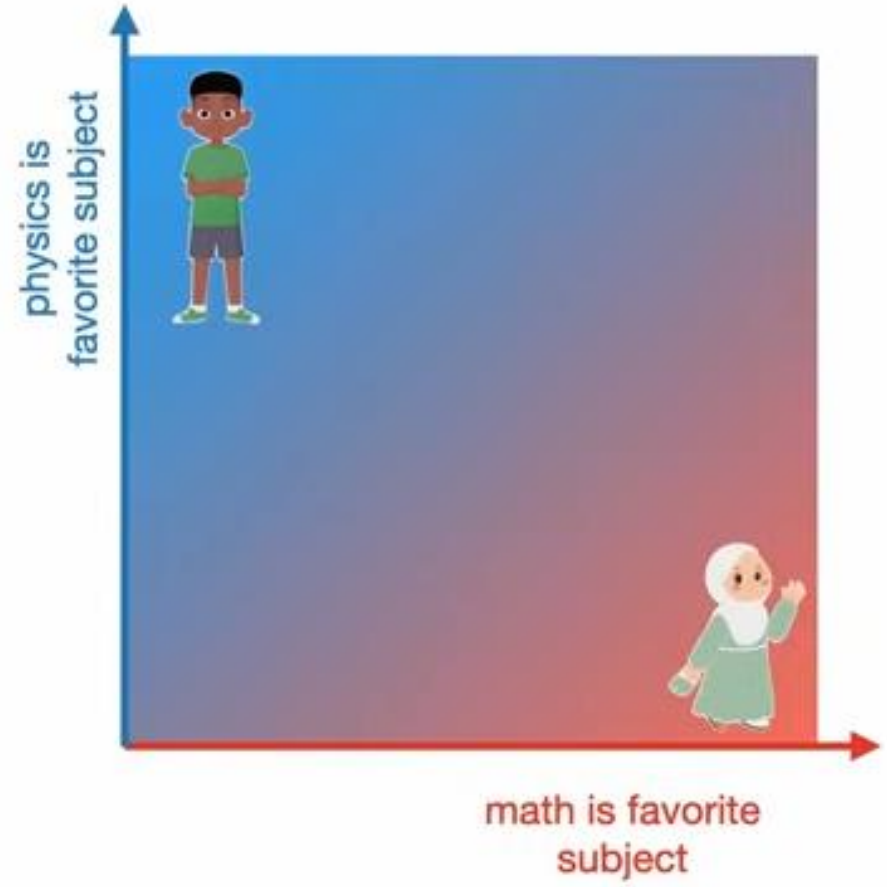
Juntar personas similares (Matemáticas/Física)



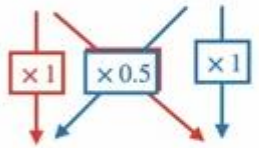
	math is favorite	physics is favorite
Aisha	1	0
Beto	0	1

¿Cómo “acercamos” a estos esos estudiantes?

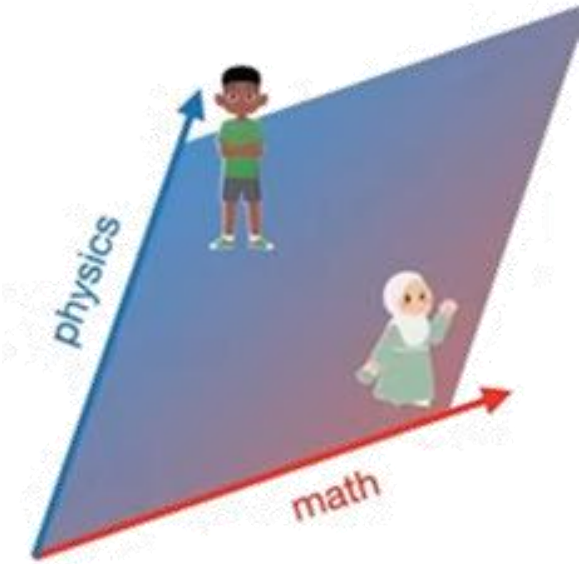
Juntar personas similares (Matemáticas/Física)



	math is favorite	physics is favorite
Aisha	1	0
Beto	0	1

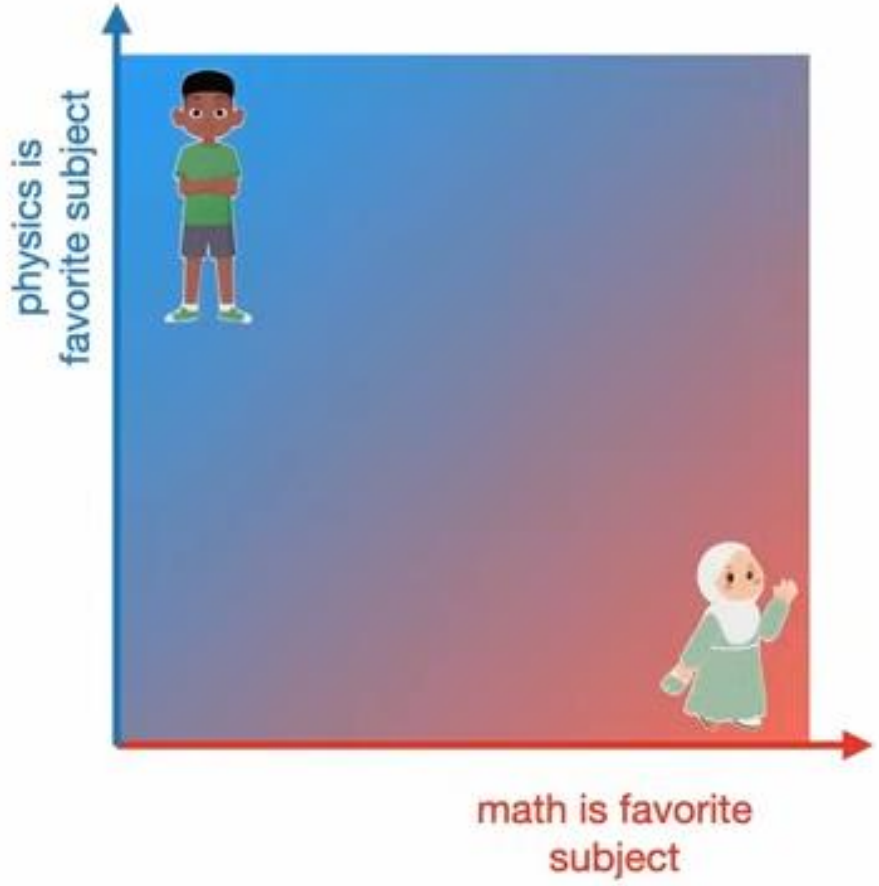


	math is favorite	physics is favorite
Aisha	1	0.5
Beto	0.5	1

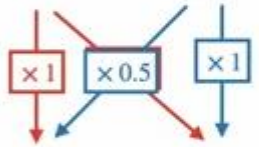


¿Qué matriz usamos?

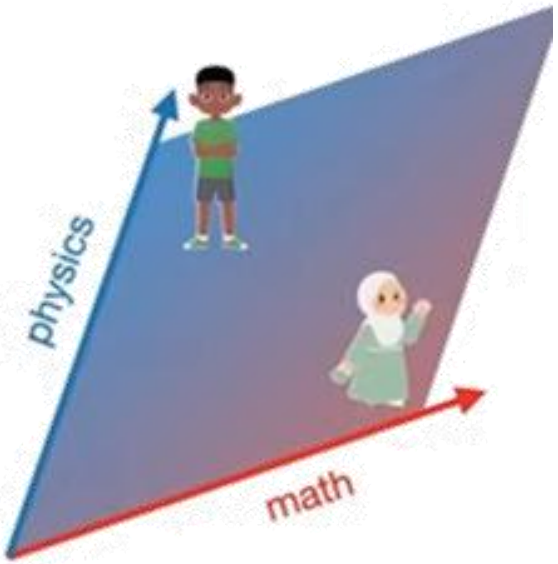
Juntar personas similares (Matemáticas/Física)



	math is favorite	physics is favorite
Aisha	1	0
Beto	0	1



	math is favorite	physics is favorite
Aisha	1	0.5
Beto	0.5	1



1	0.5
0.5	1

Ejemplo del financiero y el banco

Banquero



Banco



¿Queremos que se junten o separen las palabras? (dar ejemplo)

Ejemplo del Banquero y el Banco

BANQUERO





BANCO

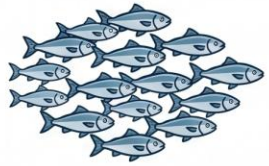


Banco de parque



Banco de dinero





Banco de peces



Banco de carpintero

¿Cuál es el problema con realizar una sola transformación?

Para transformar el espacio necesitamos…



Matrices de Claves y Consultas





¿Qué características son importantes al comparar palabras?
¿Qué características necesitan ser combinadas al comparar palabras?
¿Qué características atraen otras palabras? ¿Qué características hacen que la palabra sea traída?

Claves (Atrae) y Consultas (Es Atraído)

CLAVES

CONSULTAS

Banco

Banquero



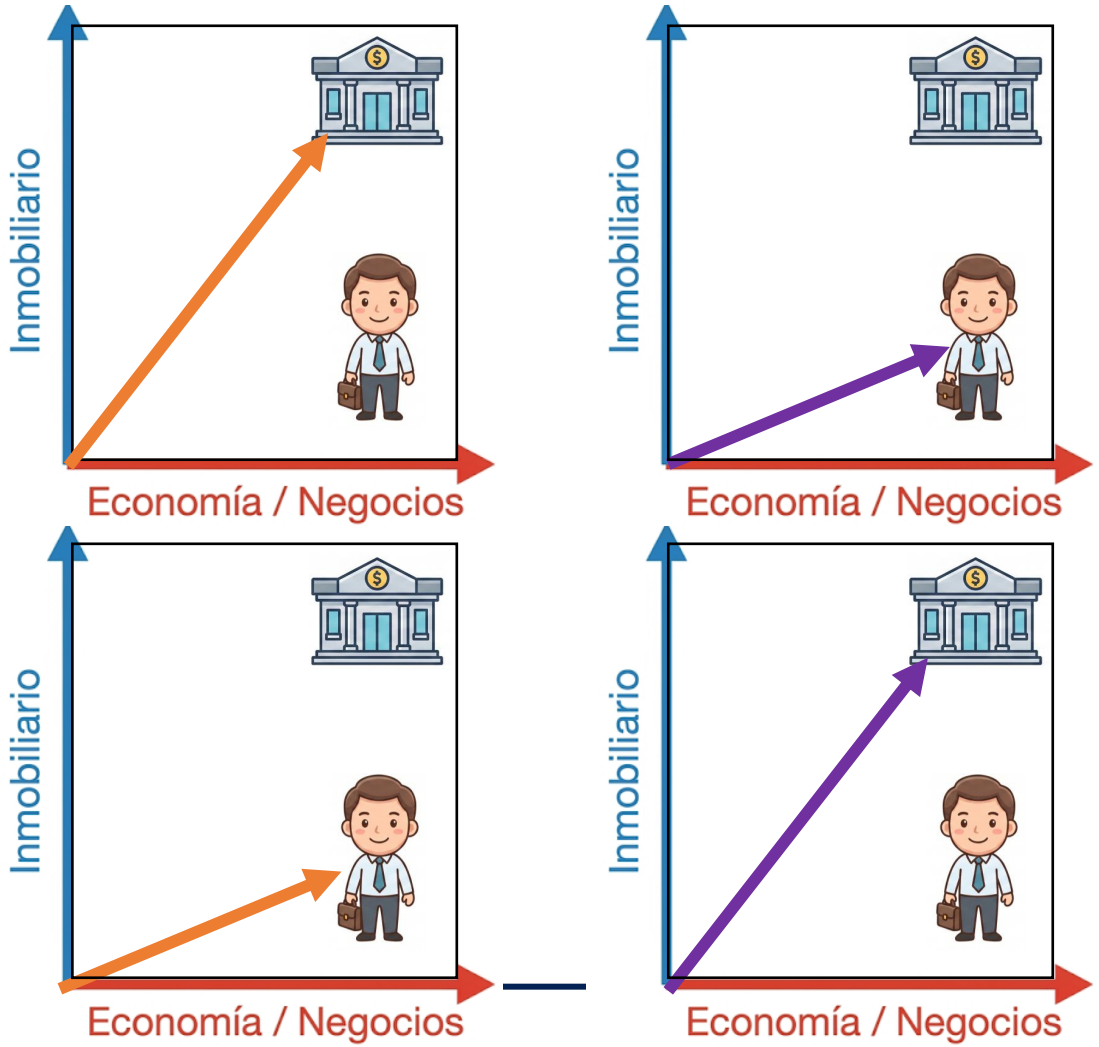
Banquero

Banco



CLAVES

CONSULTAS



Claves (Atrae) y Consultas (Es Atraído)

CLAVES

CONSULTAS

Banco

Banquero



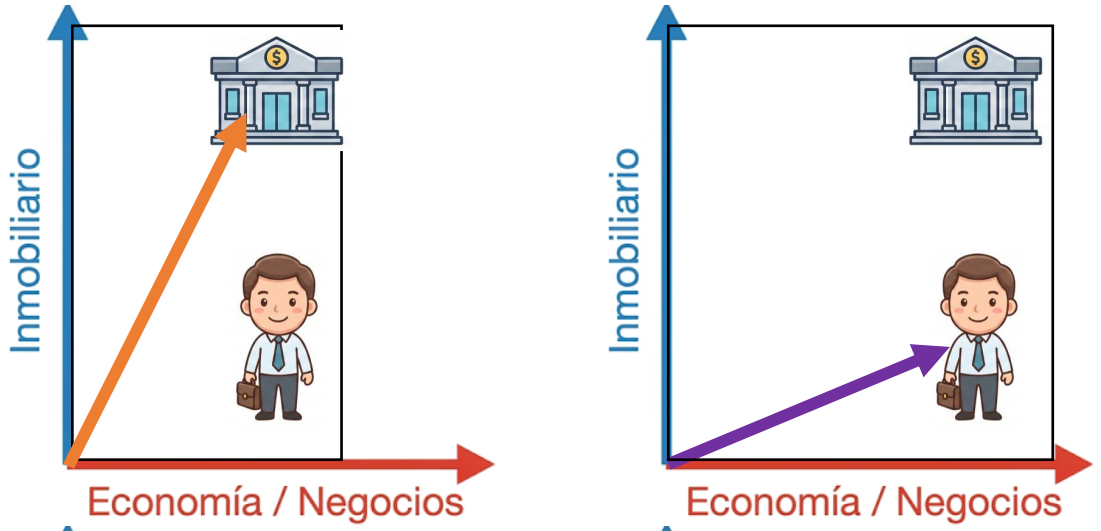
Banquero

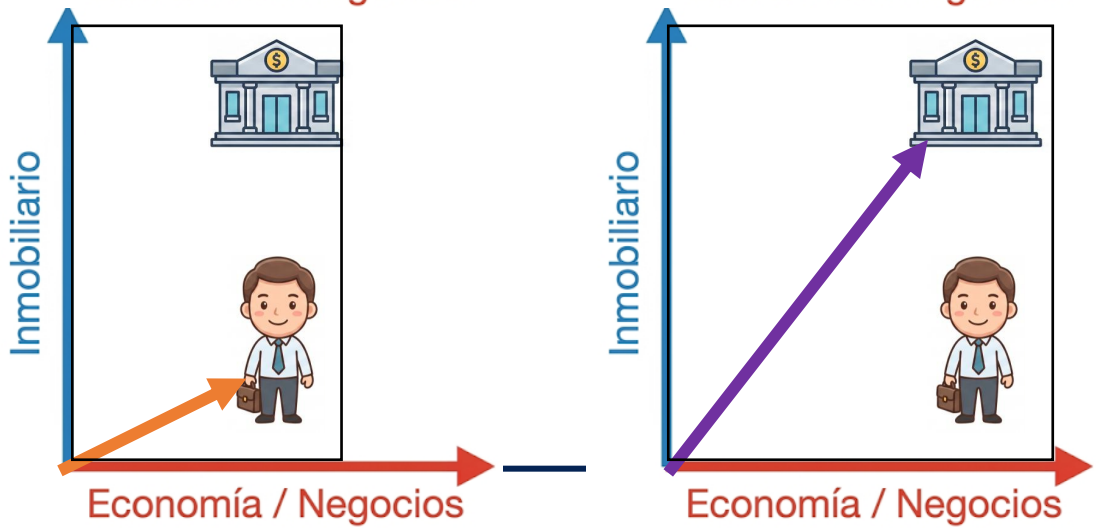
Banco



CLAVES

CONSULTAS





Claves (Atrae) y Consultas (Es Atraído)

CLAVES

CONSULTAS

Banco

Banquero

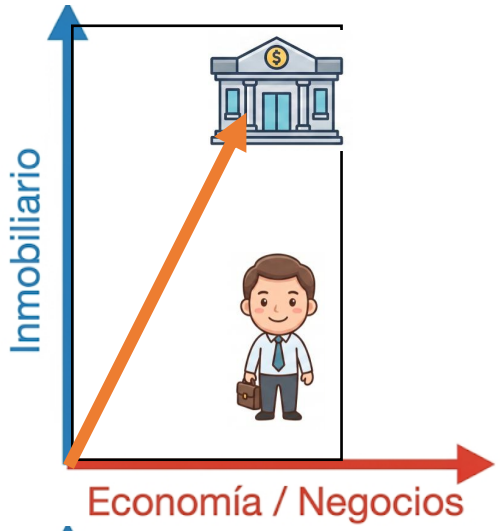


Banquero

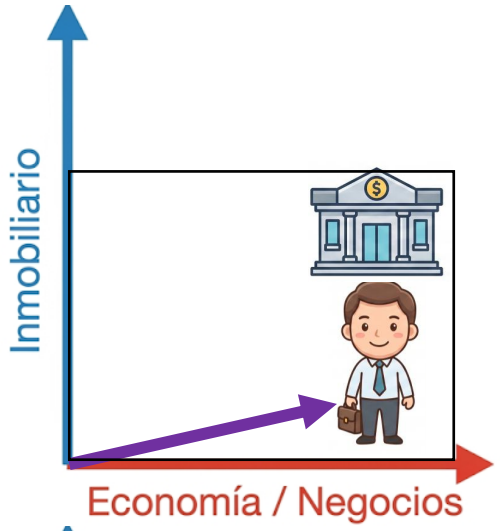
Banco

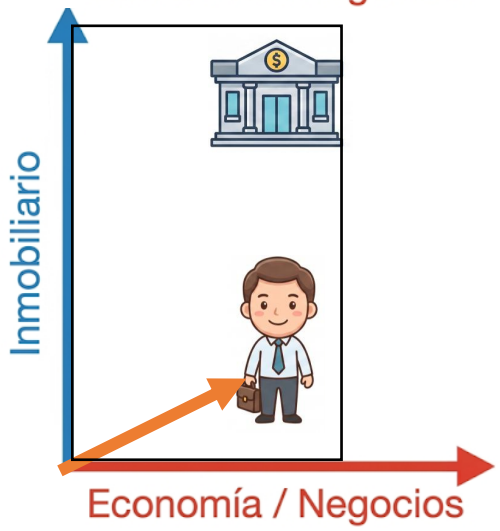


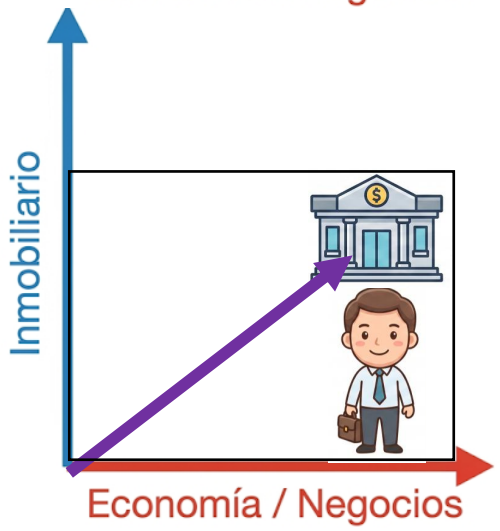
CLAVES



CONSULTAS



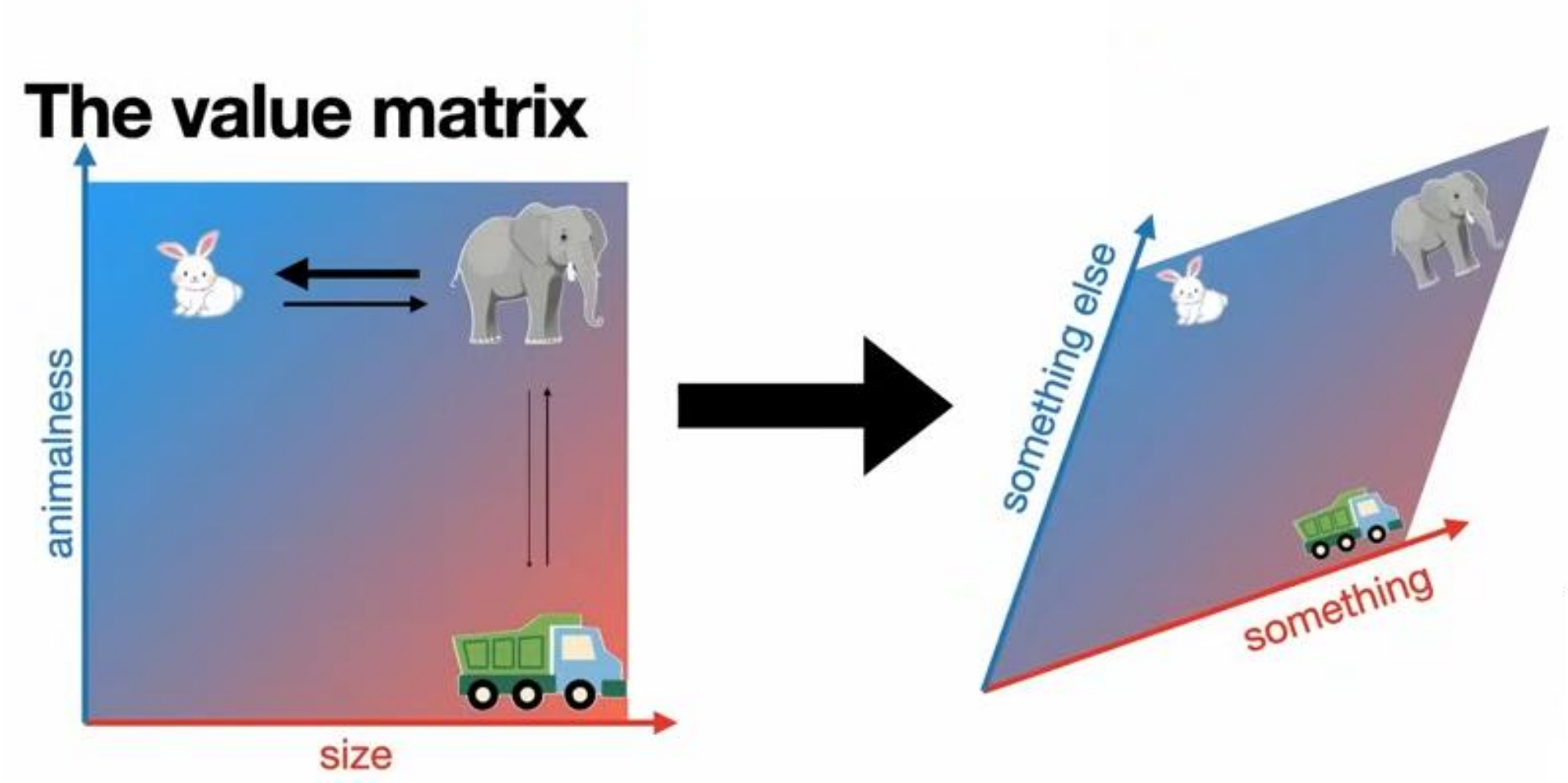




Claves (Key) y Consultas (Query)

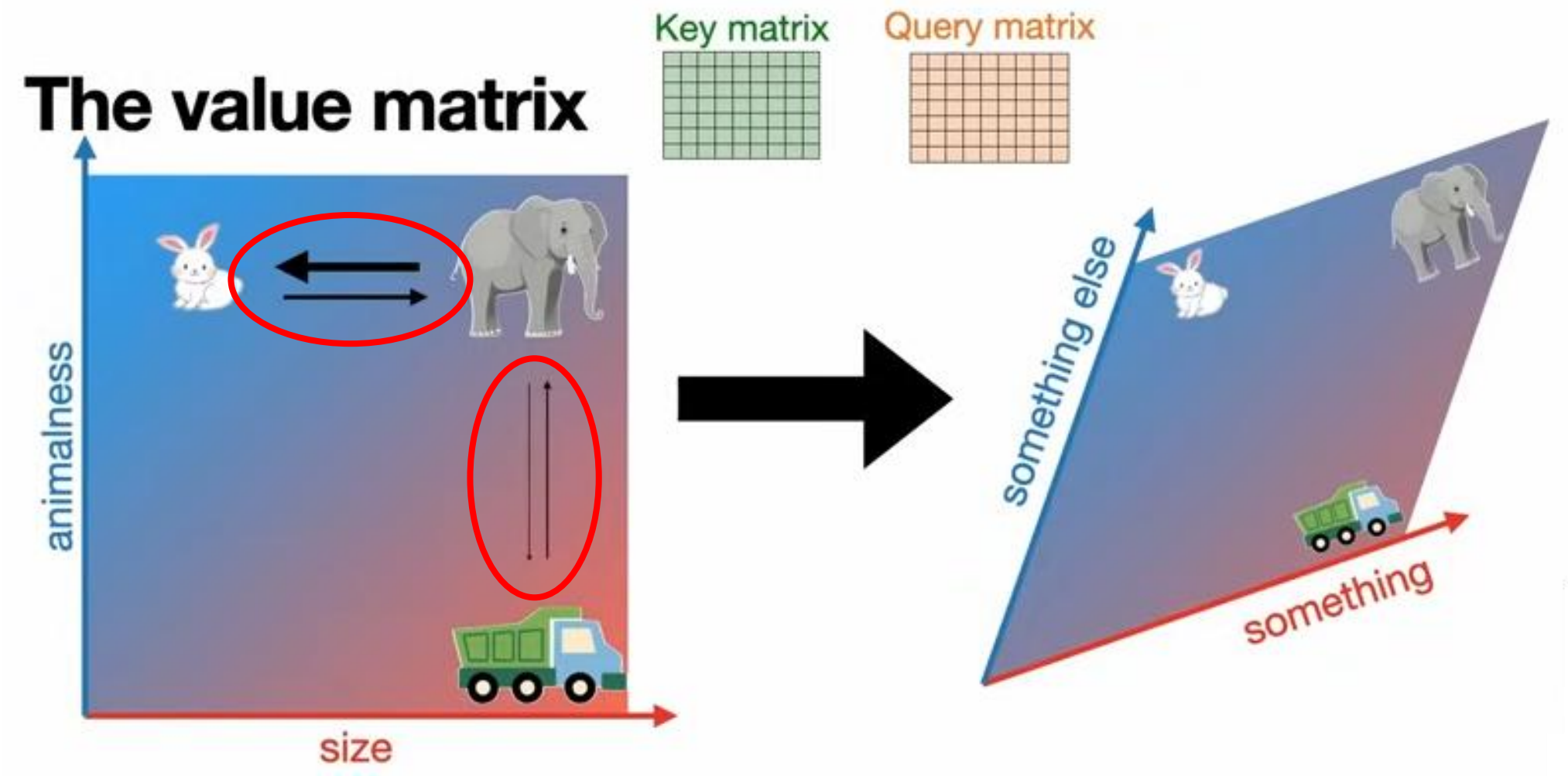


La Matriz de Valor (Value Matrix)



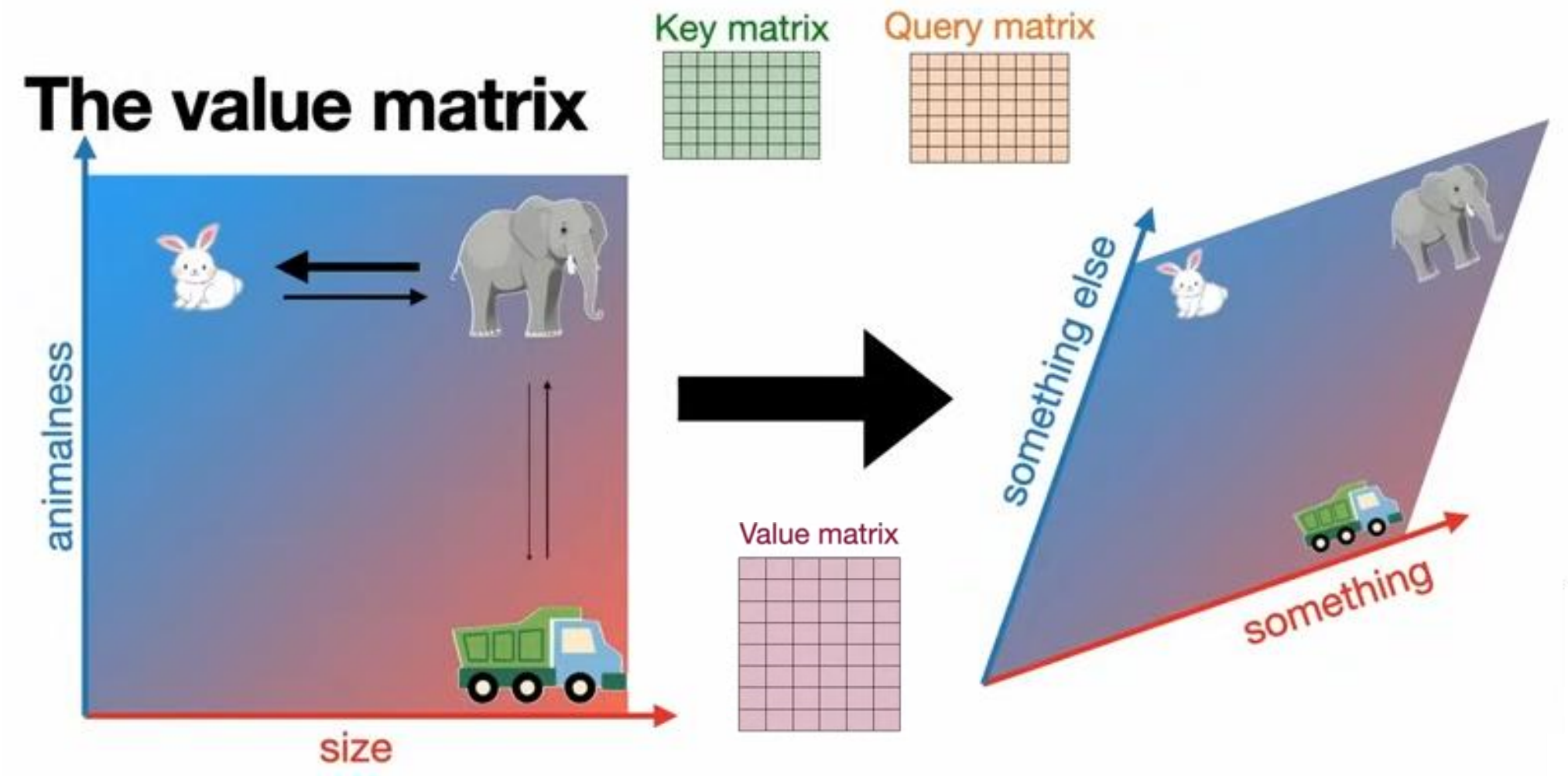
Embedding original

Embedding mejorado



Embedding original

Embedding mejorado



Embedding original

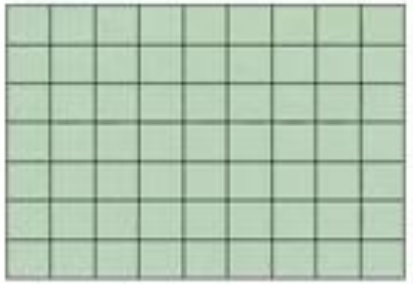
Embedding mejorado

Claves (Key), Consultas (Query) y Valores (Value)



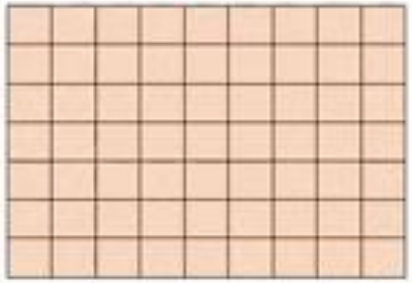
En Resumen

Key matrix



Selecciona características que atraen otras palabras

Query matrix



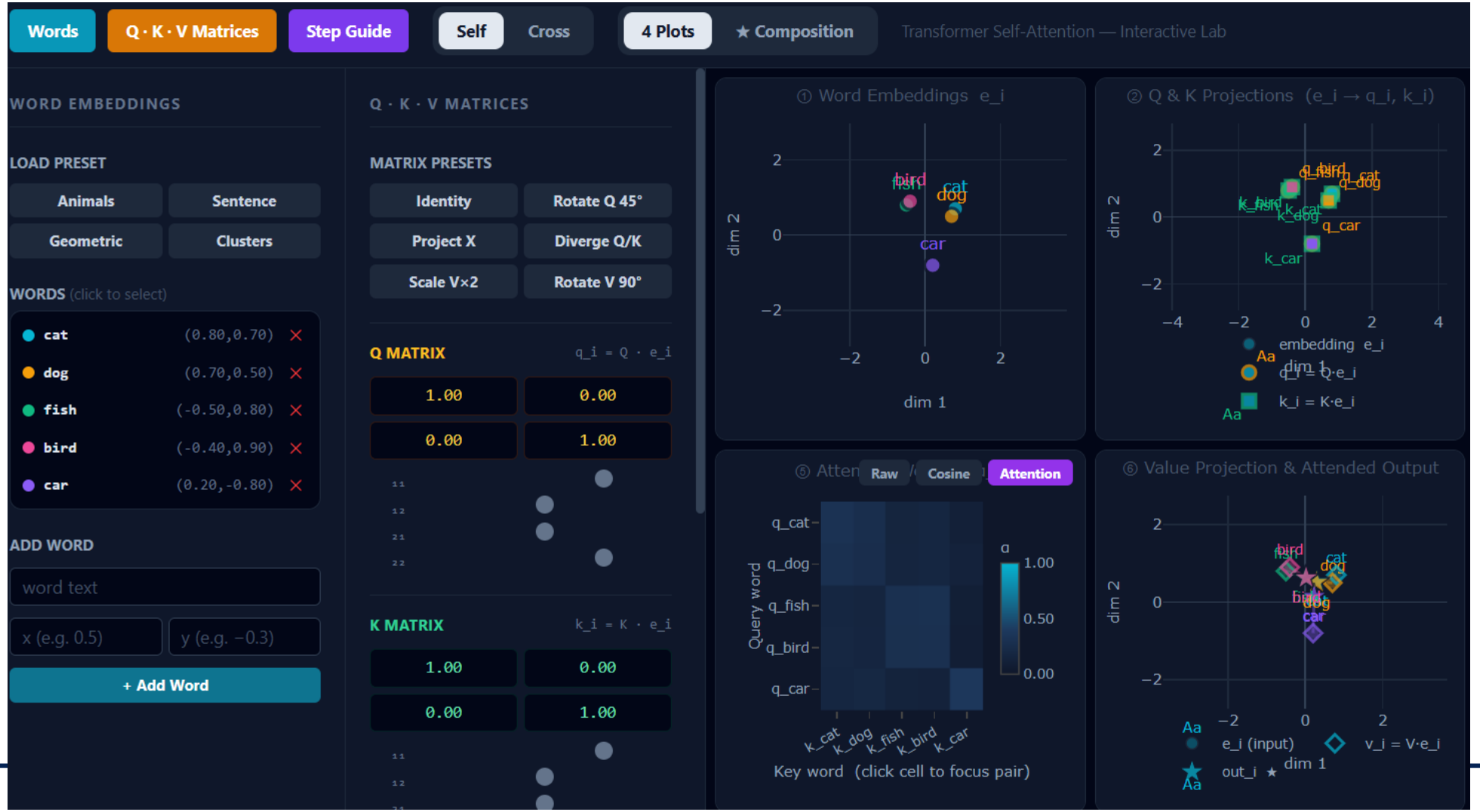
Selecciona características que ayudan a la palabra a ser atraída por otras palabras

Value matrix

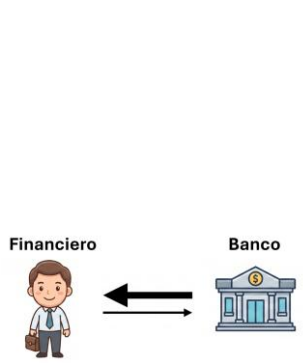


Selecciona características que son útiles para hablar

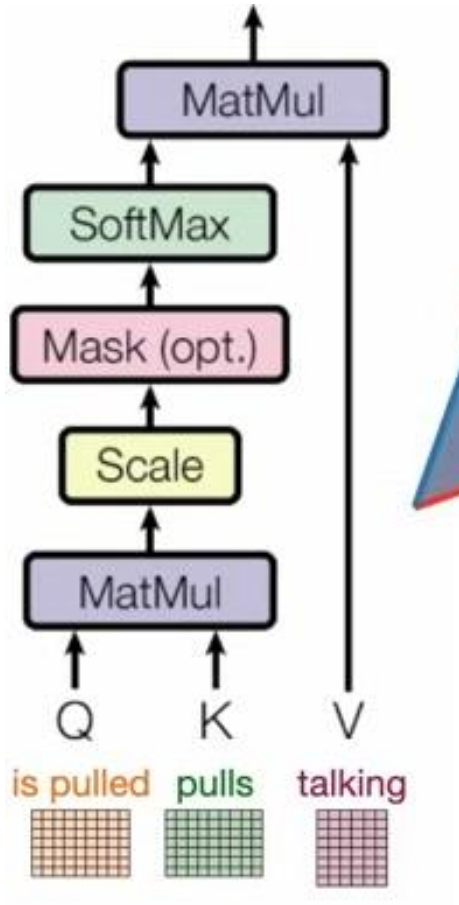
Ejemplo Interactivo

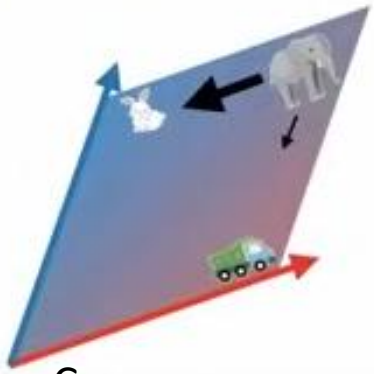


La atención en un transformer (Self-Attention)



Calcula similitudes

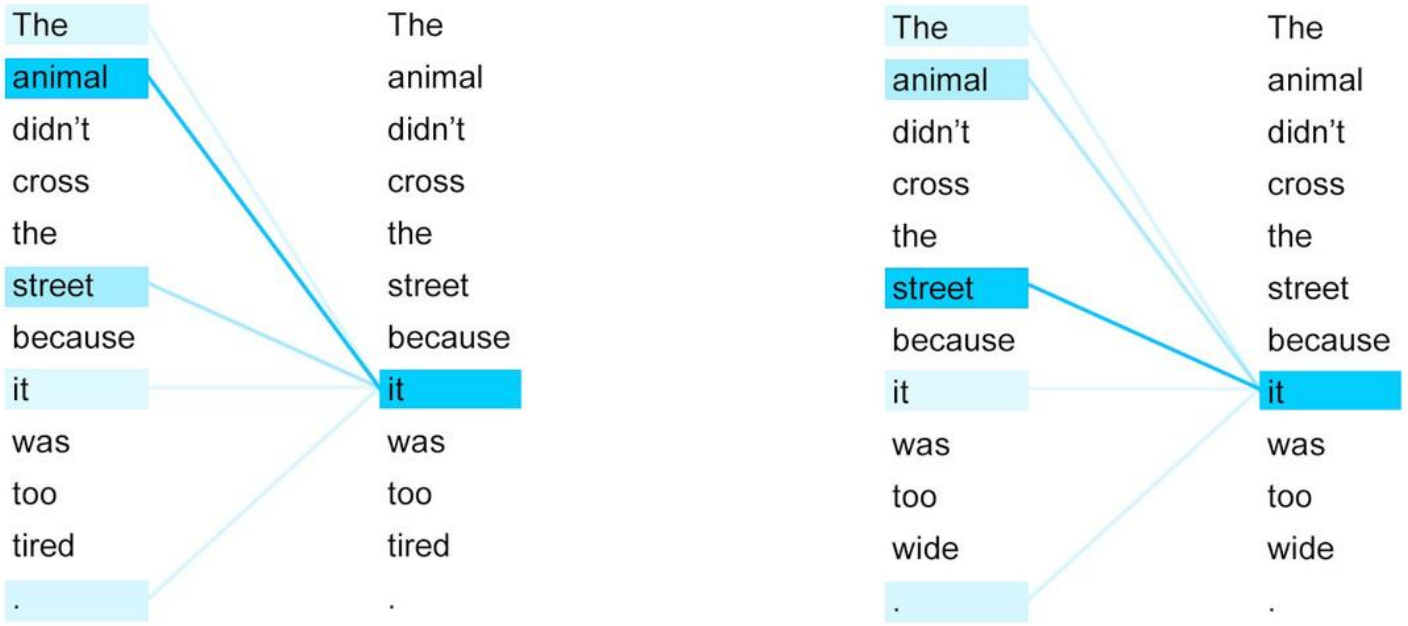




Crea un nuevo espacio de embeddings

Lo que Self-Attention puede aprender

Self-Attention aprende que “eso” podría referirse a diferentes entidades en diferentes contextos.

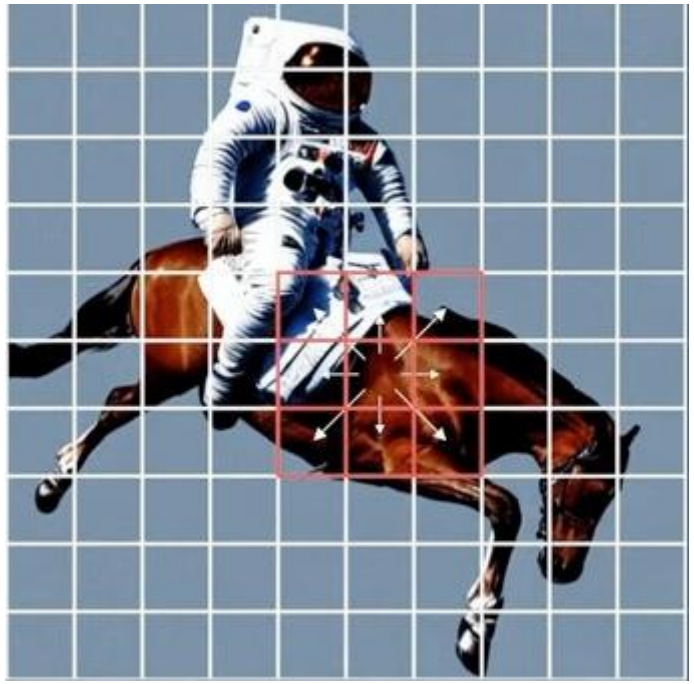


https://research.google/blog/transformer-a-novel-neural-network-architecture-for-language-understanding/

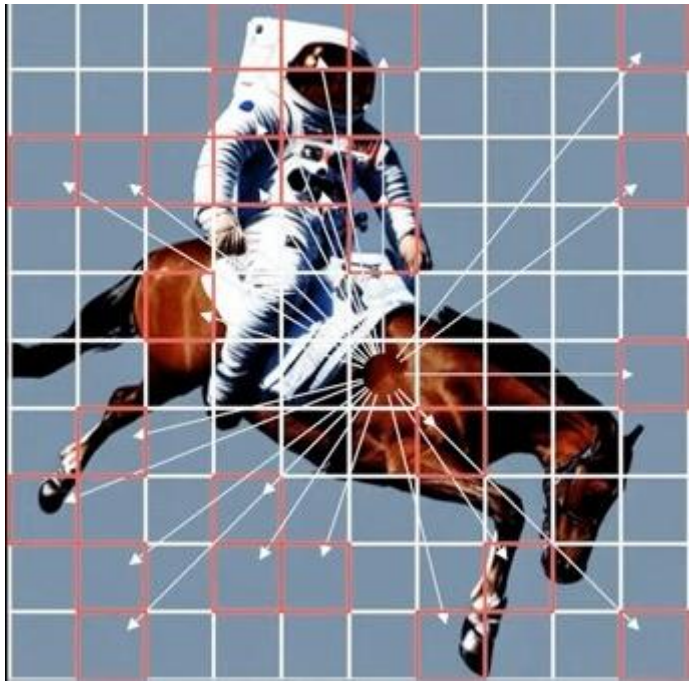
Comparación Visual

En vez de enfocarse en ventanas específicas, la atención expande el enfoque en áreas particulares del dataset.

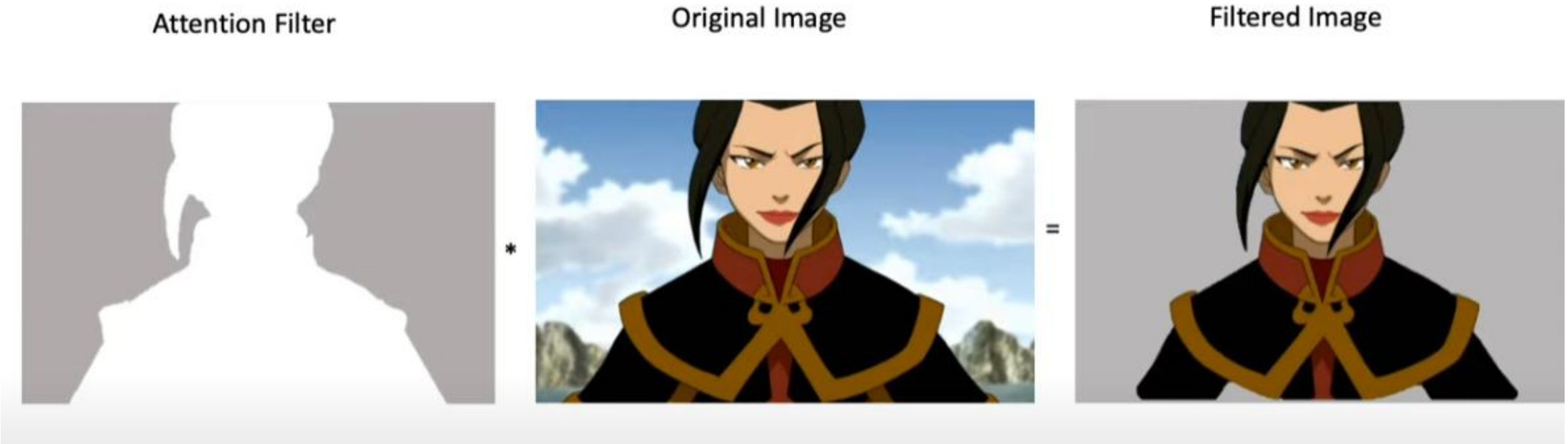
Convolución



Atención



La atención filtra la información necesaria a ser multiplicada



Visual Guide to Transformer Neural Networks - (Episode 2) Multi-Head & Self-Attention

https://www.youtube.com/watch?v=mMa2PmYJlCo

Atención Cruzada (Cross Attention)

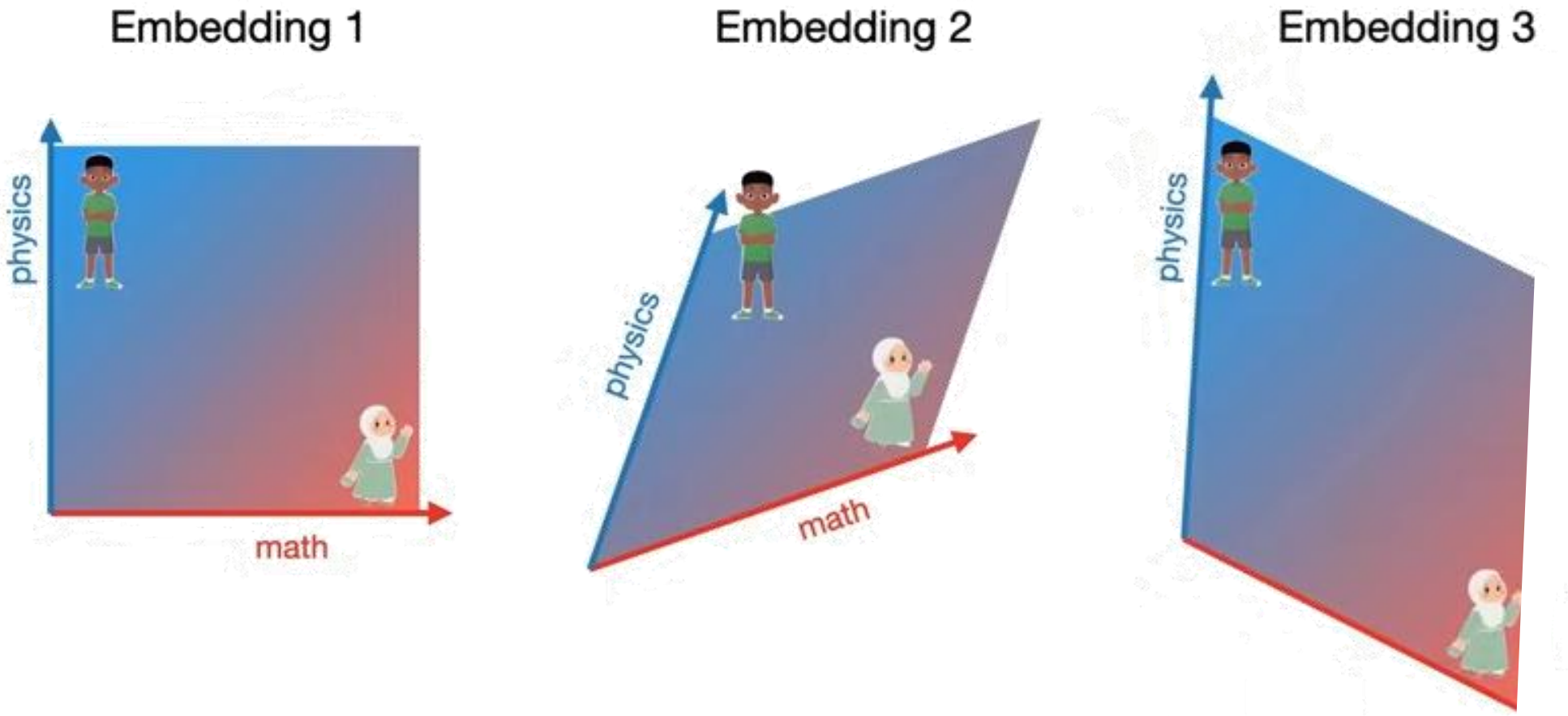


https://www.youtube.com/watch?v=aw3H-wPuRcw

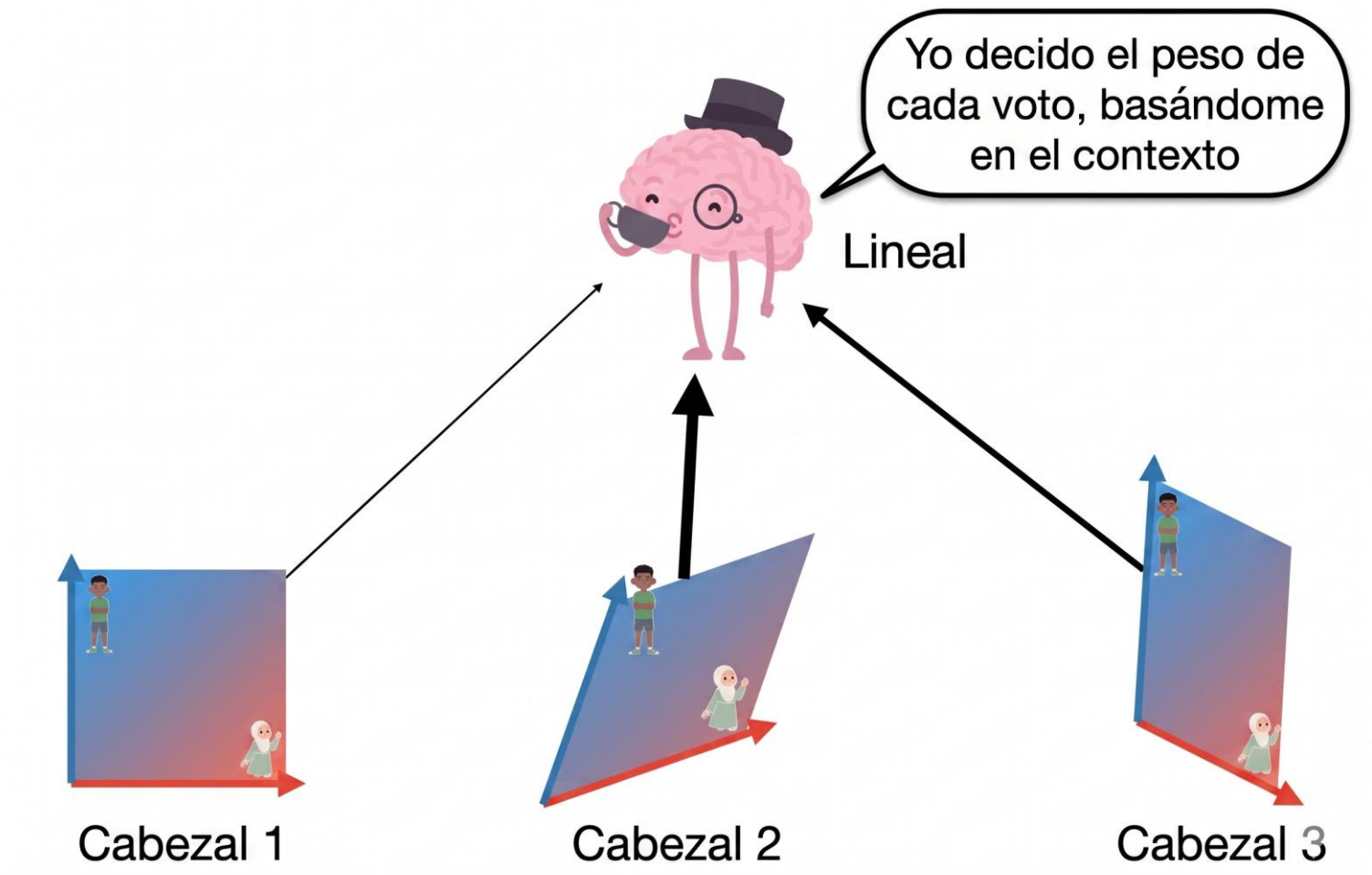


Atención de Múltiples Cabezas (Multi-Head Attention)

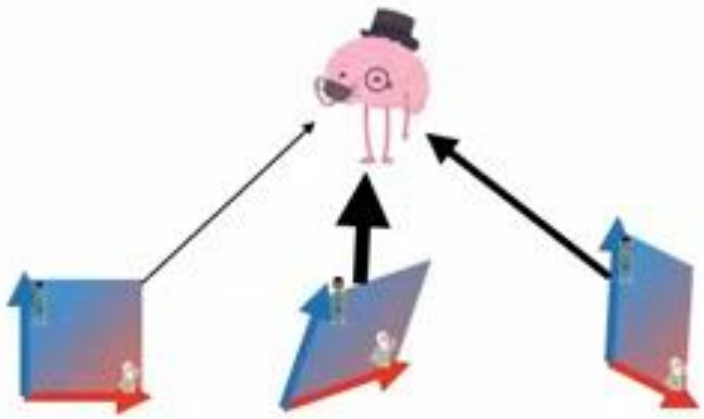
¿Qué Embedding es Mejor/Peor? (Matemáticas/Física)



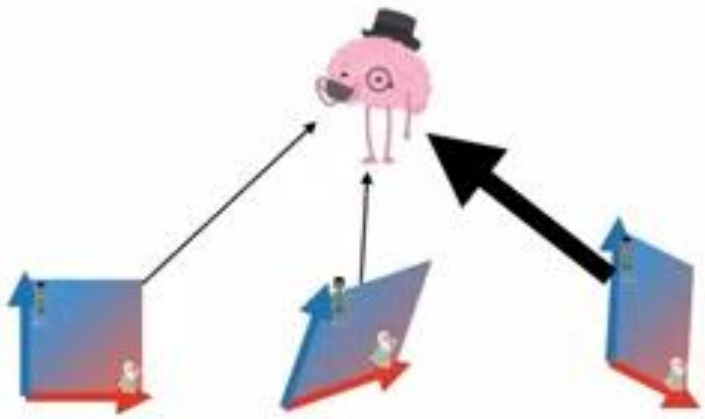
¡Tres Cabezas Piensan Más que Una!



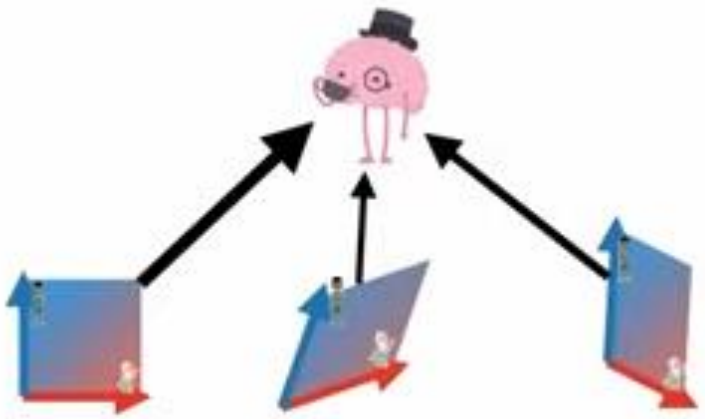
El contexto influye en los pesos de los cabezales



Contexto: Una feria de carreras universitarias

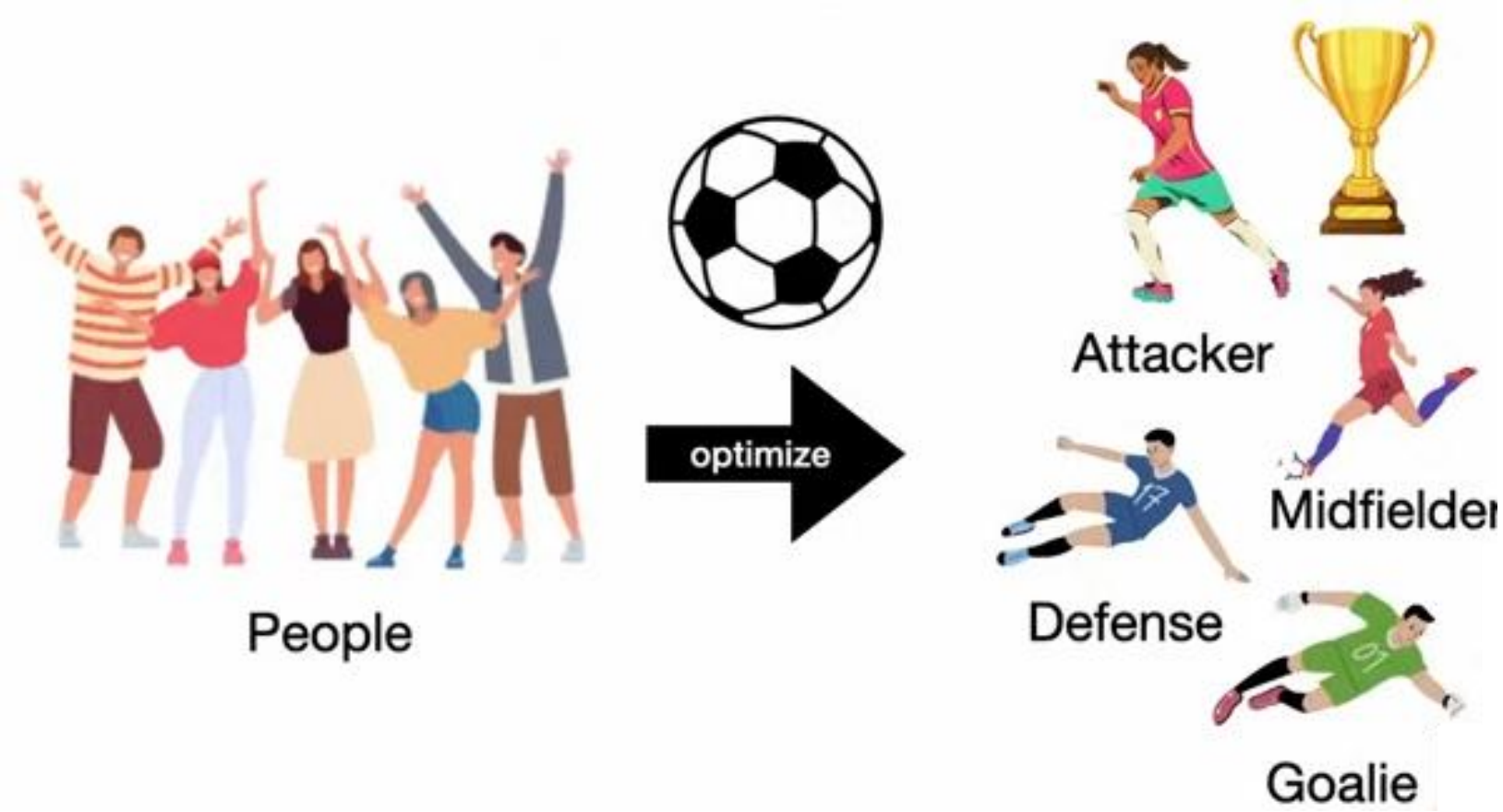


Contexto: Una conversación científica

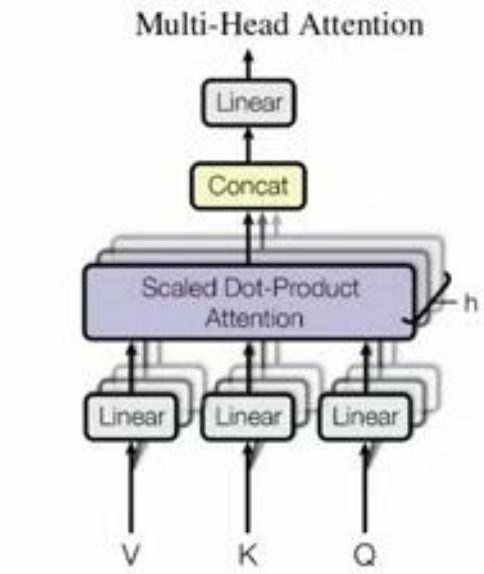


Contexto: Otros

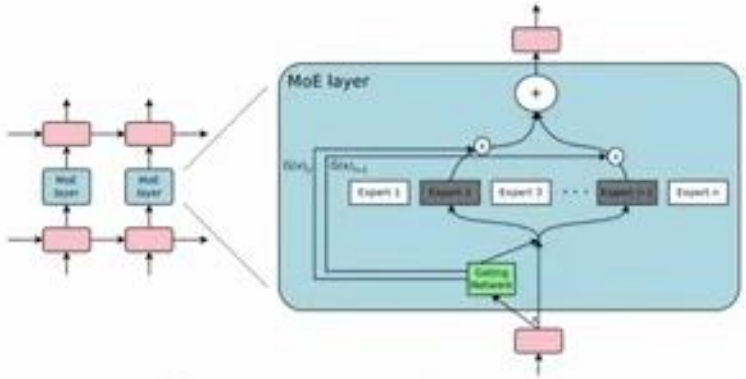
¿Por Qué Funcionan "Como un Equipo“?



Ejemplos de "Trabajo en Equipo" en el Aprendizaje Automático

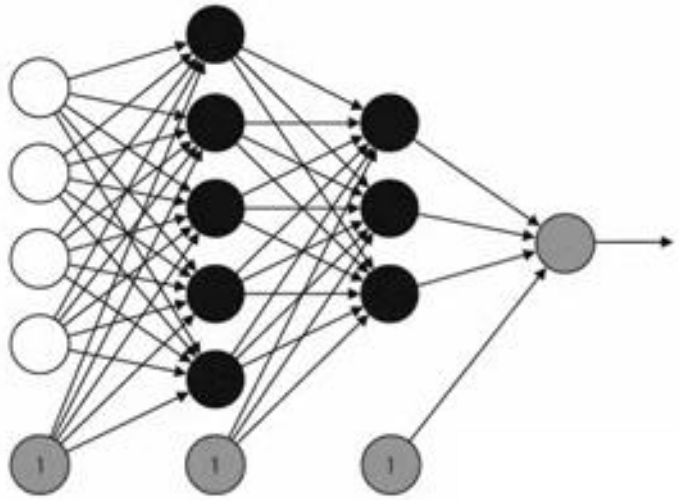


Multi-head Attention



Mixture of Experts

Image source: https://huggingface.co/blog/moe



NN



Arquitectura Transformer

Arquitectura Transformer Completa (Vaswani et al., 2017)

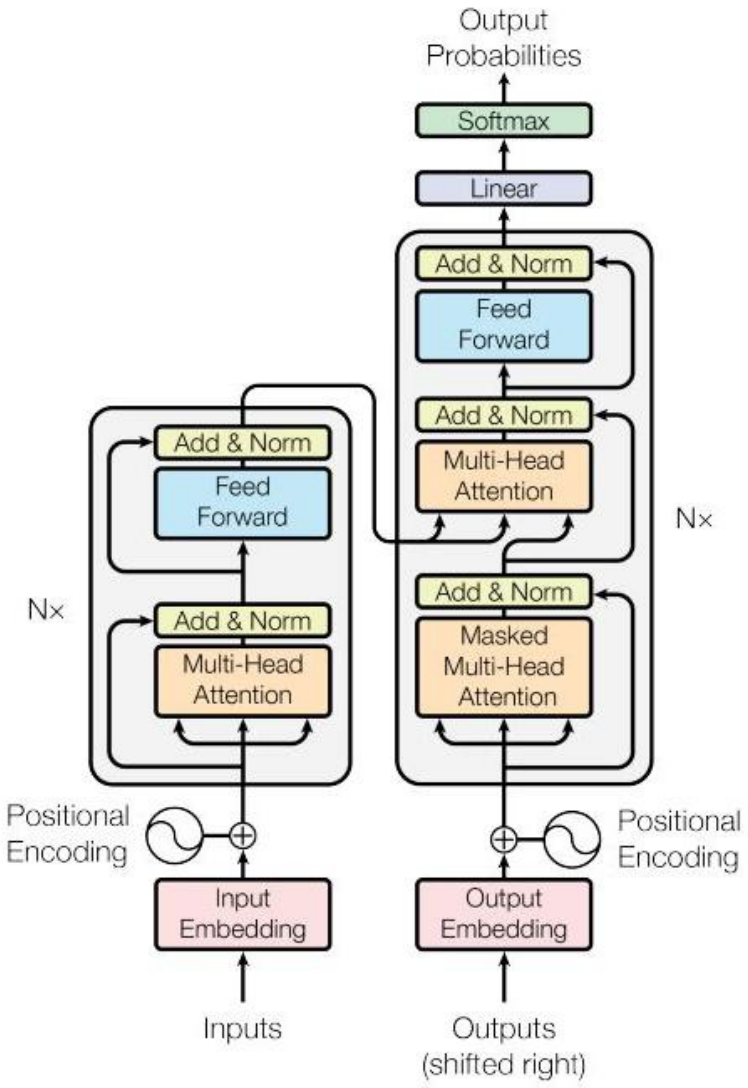
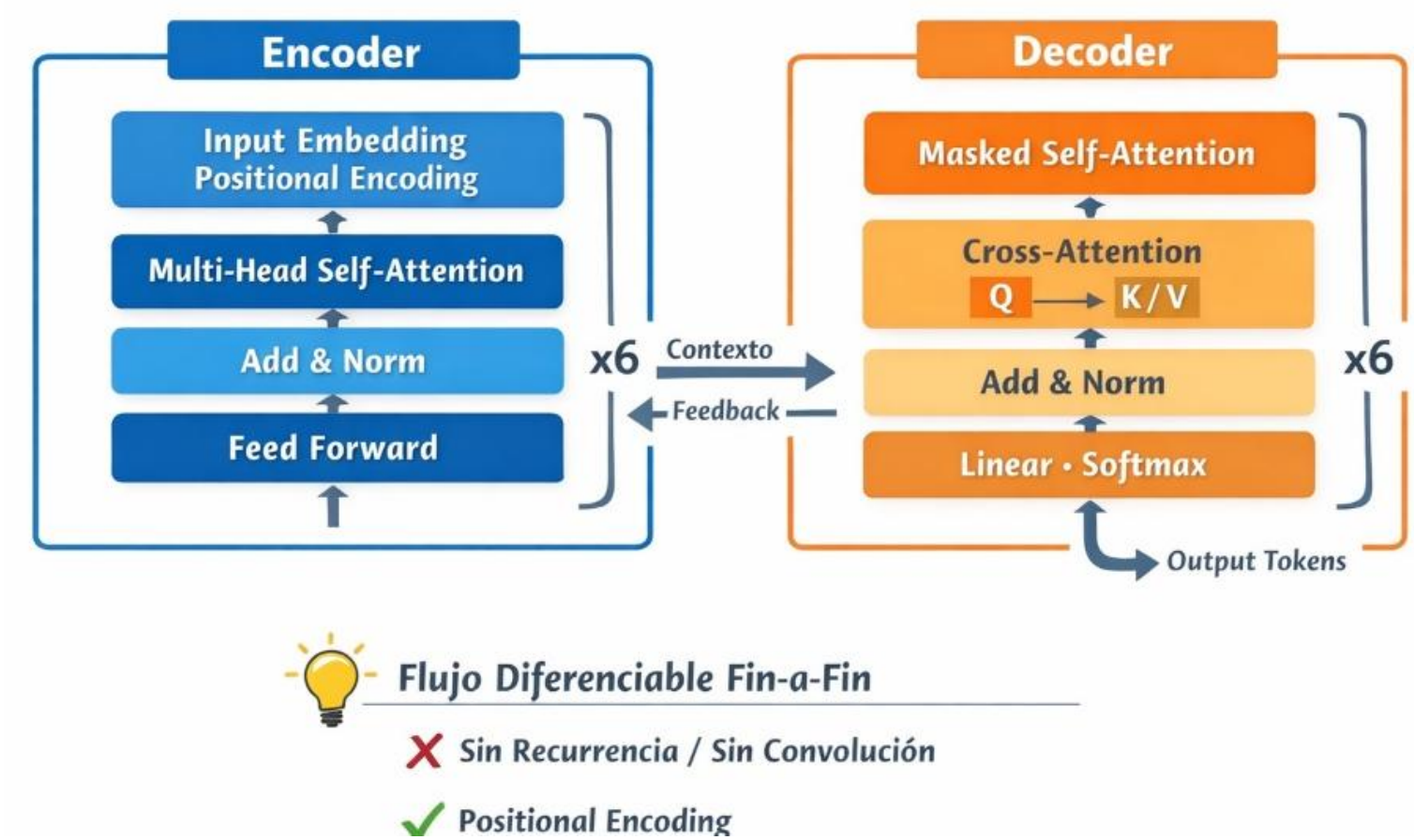


Figure 1: The Transformer - model architecture.



Codificación Posicional (Positional Encoding)

Problema: La Self-Attention es invariante al orden — no sabe en qué posición está cada token

Solución: Añadir info posicional

Se suma un vector de posición al embedding de cada token antes de entrar al Transformer:

```
input = Embedding(token) + PE(pos)
```

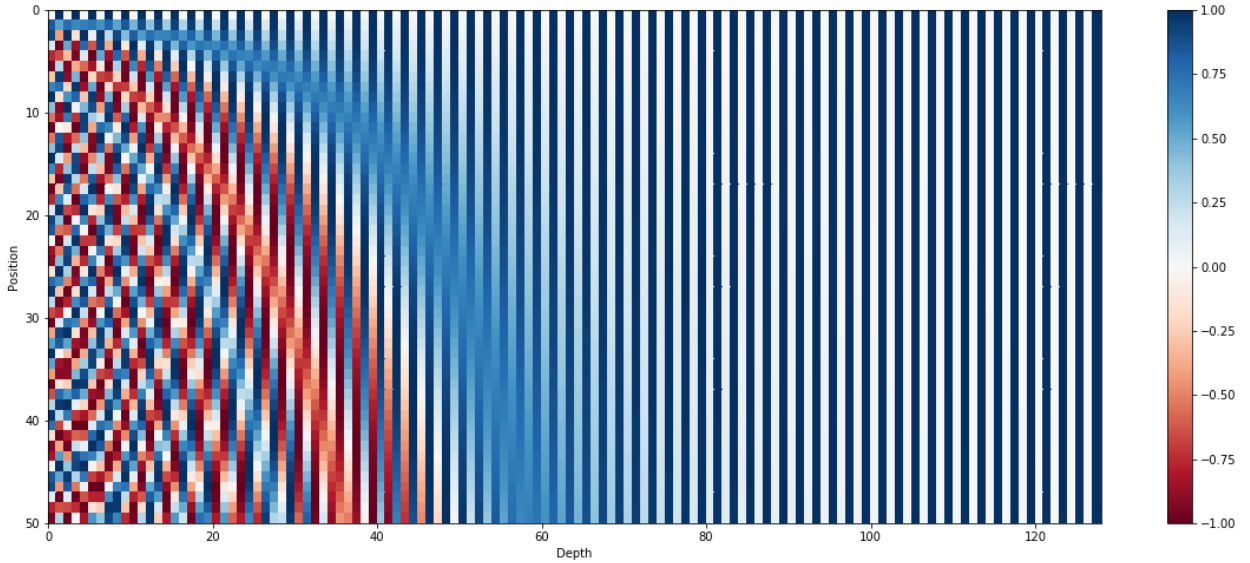
Vaswani et al. usan funciones sinusoidales fijas (no aprendidas):

```
PE(pos,2i)   = sin(pos/10000^(2i/d))
PE(pos,2i+1) = cos(pos/10000^(2i/d))
```

donde pos = posición del token, i = dimensión,
d = tamaño del embedding

Propiedades clave:

✓ **Posiciones cercanas = vectores similares**
✓ **Generaliza a secuencias más largas**
✓ **Captura distancias relativas**
✓ **Valores en [-1, 1] compatibles con embeddings**



Por qué el Transformer Supera a las RNNs en NLP

Aspecto	RNN / LSTM	Transformer
Paralelismo	❌ Secuencial (O(t) pasos)	✅ Totalmente paralelo en GPU (O(1) pasos)
Dependencias largas	❌ Se degradan (vanishing gradients)	✅ Ruta directa token-a-token (O(1))
Cuello de botella	❌ Vector de contexto único	✅ Acceso completo a todos los tokens
Escalabilidad	❌ Difcil escalar a secuencias largas	✅ GPT-4, LLaMA 3, Gemini (billones de params)
Complejidad por capa	$O(n \cdot d^2)$ [memoria $O(n \cdot d)$]	$O(n^2 \cdot d)$ [memoria $O(n^2)$] — cuello Flash Attn

Resumen: Mecanismos de Atención y Arquitectura Transformer

1. Problema RNN Encoder-Decoder: Cuello de botella — un solo vector de contexto para toda la secuencia. La calidad cae con oraciones largas.

2. Atención de Bahdanau (2015): Aditiva (MLP). Permite al decoder acceder directamente a los estados del encoder. Primer gran salto.

3. Atención de Luong (2015): Multiplicativa (producto punto). Más eficiente. Base del Scaled Dot-Product Attention del Transformer.

4. Self-Attention + Positional Encoding: Elimina la RNN. Cada token atiende a todos los demás. La codificación posicional (sin/cos) inyecta el orden.

5. Transformer (Vaswani et al., NeurIPS 2017): Encoder (6 capas) + Decoder (6 capas). Multi-Head Attention (h=8), FFN, Add & Norm. Totalmente paralelo.

6. LayerNorm + Residuals: Permiten entrenar redes muy profundas estabilizando gradientes. Patrón: LayerNorm(x + Sublayer(x)).

Preguntas del Vibe Quest

- Un compañero dice: "La Self-Attention es básicamente lo mismo que la atención de Bahdanau, solo más rápida." ¿Qué le corriges? ¿Puedes implementar ambos mecanismos en PyTorch y visualizar sus matrices de atención sobre la misma oración para mostrar en qué difieren?
- ¿Qué le pasaría al Transformer si eliminamos el Positional Encoding y le damos las frases "El gato persiguió al perro" y "El perro persiguió al gato"? ¿Puedes demostrarlo con código comparando los embeddings de salida con y sin codificación posicional, y visualizando por qué el orden se pierde?
- En el Decoder del Transformer hay dos tipos de atención por capa: Masked Self-Attention y Cross-Attention. ¿Puedes explicar de dónde vienen las Q, K y V en cada caso y diseñar una visualización que muestre qué tokens atiende el decoder en cada tipo durante la generación de una traducción?
- ¿Por qué el bloque Add & Norm aparece después de cada subcapa del Transformer? ¿Puedes entrenar un Transformer con y sin conexiones residuales sobre una tarea de secuencia simple y graficar las curvas de pérdida para demostrar qué problema concreto resuelven al escalar a 6 o más capas?